



MICROPROCESSOR & MICROCONTROLLER





Centre for Distance and Online Education

Online BCA Program

Microprocessor & Microcontroller

Semester: 4

Author

Dr. Twinkle Doshi,
Assistant Professor,
Department of Computer Applications,
Faculty of Science,
The Maharaja Sayajirao University.

Credits

Centre for Distance and Online Education, Parul University,
Post Limda, Waghodia,
Vadodara, Gujarat, India
391760.
Website: <https://paruluniversity.ac.in/>

Disclaimer

This content is protected by CDOE, Parul University. It is sold under the stipulation that it cannot be lent, resold, hired out, or otherwise circulated without obtaining prior written consent from the publisher. The content should remain in the same binding or cover as it was initially published, and this requirement should also extend to any subsequent purchaser. Furthermore, it is important to note that, in compliance with the copyright protections outlined above, no part of this publication may be reproduced, stored in a retrieval system, or transmitted through any means (including electronic, Mechanical, photocopying, recording, or otherwise) without obtaining the prior written permission from both the copyright owner and the publisher of this content.

Note to Students



These course notes are intended for the exclusive use of students enrolled in Online BCA Program. They are not to be shared or distributed without explicit permission from the University. Any unauthorized sharing or distribution of these materials may result in academic and legal consequences.



TABLE OF CONTENT

SUB LESSON 1.1..... 10

Definition and Role of a Microprocessor 10

SUB LESSON 1.2..... 13

EVOLUTION AND HISTORY OF MICROPROCESSORS 13

Sub lesson 2.1

Classification Based on Architecture (CISC vs RISC)..... 19

Sub lesson 2.2

Classification Based on Bit-width (8-bit, 16-bit, 32-bit, 64-bit) 25

Sub lesson 2.3

Popular Microprocessor Families (8085, 8086, Pentium, ARM)..... 32

Sub lesson 3.1 37

Definition and Importance of Instruction Set..... 37

Sub lesson 3.2

Types of Instructions (Data Transfer, Arithmetic, Logical, Control) 40

SUB LESSON 3.3

ASSEMBLY LANGUAGE VS MACHINE LANGUAGE..... 44

SUB Lesson 4.1 47

Types of Buses (Data Bus, Address Bus, Control Bus). 47

SUB Lesson 4.2 50

Bus Structure and Communication in a Microprocessor..... 50

SUB Lesson 4.3 53

Role of System Buses in Memory and I/O Interfacing..... 53



Sub lesson 5.1	55
Definition and Importance of Addressing Modes	55
Sub lesson 5.2	58
TYPES OF Addressing MODES (Immediate, Direct, Indirect, Register)	58
Sub lesson 5.3	61
Examples of Addressing Modes in 8085	61
SUB lesson 6.1	64
Registers in 8085 (Accumulator, General-Purpose Registers).....	64
SUB lesson 6.2	68
SPECIAL PURPOSE REGISTERS (PROGRAM COUNTER, STACK POINTER, FLAGS)	68
SUB lesson 6.3	73
Memory Organization and Data Flow in 8085).....	73
SUB LESSON 7.1.....	77
Microprocessor ARCHITECTURE and it's Operation	77
SUB LESSON 7.2.....	83
Pin diagram of 8085	83
SUB LESSON 7.3.....	89
Block Diagram of the 8085 Microprocessor	89
SUB LESSON 8.1.....	95
Internal Registers Organization of 8085 Microprocessor	95
SUB LESSON 8.2.....	100
Memory and I/O devices.....	100
SUB LESSON 8.3.....	104
Memory Interfacing in 8085 Microprocessor	104



SUB LESSON 8.4.....	109
Interfacing I/O devices	109
SUB LESSON 9.1.....	115
Flag Registers of 8085	115
SUB LESSON 9.2.....	120
Looping, Counting and Indexing	120
SUB LESSON 9.3.....	127
Logic Operations Rotate and Compare	127
SUB LESSON 10.1	133
Addition and Subtraction of Two 8-bit Numbers	133
SUB LESSON 10.2	139
Multiplication and Division OF TWO NUMBERS USING REPEATED ADDITION/ sUBTRACTION .	139
SUB LESSON 10.3	145
Finding the Largest and Smallest Number in an Array	145
SUB LESSON 10.4	152
Delay Generation Using Looping Techniques	152
SUB LESSON 11.1	160
Concept of Stack in 8085 (PUSH and POP Instructions)	160
SUB LESSON 11.2	169
Role of Stack Pointer and Stack Memory Management.....	169
SUB LESSON 11.3	176
Subroutines and their Implementation Using CALL and RET Instructions	176
SUB LESSON 11.4	182
RST Instructions	182



SUB LESSON 11.5	188
Return Instructions and Stack Operations during Function Calls	188
SUB LESSON 11.6	194
Binary to BCD and Decimal Conversion	194
SUB LESSON 12.1	201
INTRODUCTION	201
SUB LESSON 12.2	205
Types of Interrupts.....	205
SUB LESSON 12.3	211
Processing of Interrupts.....	211
SUB LESSON 12.4	217
Interrupt Service Routine.....	217
SUB LESSON 13.1	223
Introduction	223
SUB LESSON 13.2	231
Register Transfer Language	231
SUB LESSON 13.3	237
Memory organization	237
SUB LESSON 13.4	244
Bus Design Using Multiplexer and Tri-State Buffer	244
SUB LESSON 13.5	251
Memory transfer.....	251
SUB LESSON 14.1	256
Arithmetic Micro Operation.....	256



SUB LESSON 14.2 263

Logical Micro Operation..... 263

SUB LESSON 14.3 270

Shift Micro Operation 270

SUB LESSON 14.4 275

Arithmetic Logic shift unit..... 275



INTRODUCTION TO MICROPROCESSORS





SUB LESSON 1.1

DEFINITION AND ROLE OF A MICROPROCESSOR

OBJECTIVES OF CHAPTER 1 (INTRODUCTION TO MICROPROCESSORS)

- 1) To explain the classification of microprocessors based on architecture and bit-width.
- 2) To provide an understanding of instruction sets, types of instructions, and assembly language programming.
- 3) To describe microprocessor bus organization and its role in data communication and interfacing.
- 4) To explore addressing modes and the programming model of the 8085 microprocessor in detail.

DEFINITION AND ROLE OF A MICROPROCESSOR

DEFINITION:

A microprocessor is an essential component in modern computing, often referred to as the "brain" of a computer system. It is a multipurpose, clock-driven, and register-based digital integrated circuit designed to accept binary data as input, process it according to pre-stored instructions, and deliver the output results. Essentially, a microprocessor functions as the central processing unit (CPU) of a computer, incorporating millions of transistors connected via intricate wiring on a single chip.

ROLE:

The microprocessor executes arithmetic and logic operations using small storage areas called registers. These operations typically include fundamental mathematical computations such as addition and subtraction, logical comparisons, and data movement between different memory



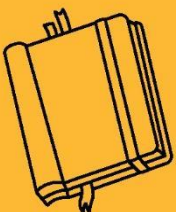
locations. Microprocessors serve as the core component of various computing devices, from simple embedded systems to complex computer architectures.

KEY TAKEAWAYS

- A microprocessor is the CPU of a computer, responsible for executing instructions and processing data.
- It uses registers for arithmetic and logic operations, handling data storage and movement.
- It's essential in a range of systems from embedded devices to full-fledged computers.



INTRODUCTION TO MICROPROCESSORS





SUB LESSON 1.2

EVOLUTION AND HISTORY OF MICROPROCESSORS

EVOLUTION AND HISTORY OF MICROPROCESSORS

The microprocessor is a remarkable invention that revolutionized the world of electronics and computing. Acting as the brain of digital devices, it is responsible for executing instructions and performing calculations that power everything from calculators to supercomputers. The history of microprocessors is marked by continuous innovation, miniaturization, and increasing complexity. This chapter explores the origins, development, and major milestones in the evolution of microprocessors.

THE CONCEPT AND EARLY FOUNDATIONS

Before the invention of the microprocessor, computers were built using vacuum tubes and later, discrete transistors. These early machines were large, expensive, and consumed significant power. The idea of integrating multiple functions onto a single chip was not initially obvious, but advances in semiconductor technology, especially the invention of the integrated circuit (IC) in 1958 by Jack Kilby and Robert Noyce, laid the groundwork for microprocessors.

The integrated circuit allowed multiple electronic components—transistors, resistors, and capacitors—to be fabricated on a single piece of silicon. This advancement significantly reduced the size and cost of electronic systems, leading to the development of more compact and efficient computers.

THE BIRTH OF THE FIRST MICROPROCESSOR: INTEL 4004

The world's first commercially available microprocessor was the Intel 4004, introduced in 1971. Developed by Federico Faggin, Ted Hoff, and Stanley Mazor at Intel, the 4004 was originally designed for a Japanese calculator company called Busicom. Instead of building multiple custom chips, Intel proposed a general-purpose processor that could be programmed for different tasks.



The Intel 4004 was a 4-bit processor with 2,300 transistors, a clock speed of 740 kHz, and could execute approximately 92,000 instructions per second. It could address up to 640 bytes of memory. Although limited by today's standards, the 4004 marked the beginning of a new era in computing.

MULTI-CORE ERA

As clock speeds increased, power consumption and heat generation became limiting factors. Rather than increasing clock speed indefinitely, processor manufacturers began integrating multiple cores into a single chip. A multi-core processor can execute multiple threads simultaneously, improving multitasking and parallel computing performance.

Intel and AMD both launched dual-core processors in the mid-2000s, followed by quad-core, hexa-core, and octa-core processors. Today, processors with 16, 32, or even more cores are available, especially in server and high-performance computing markets.

MODERN MICROPROCESSOR TRENDS

In recent years, several key trends have defined microprocessor development:

a) System on Chip (SoC):

A SoC integrates the CPU with other components like GPU, memory controller, and I/O interfaces on a single chip. SoCs are widely used in smartphones, tablets, and embedded systems. Companies like Apple (M-series chips), Qualcomm (Snapdragon), and MediaTek dominate this space.

b) Energy Efficiency:

With the growing demand for mobile and edge computing, microprocessors are being designed to consume less power without compromising performance. ARM processors, known for their energy efficiency, are now used in smartphones, wearables, and even some laptops (e.g., Apple's M1 and M2 chips).



c) AI and Specialized Processing Units:

Modern microprocessors are incorporating AI acceleration capabilities. GPUs and dedicated AI processors such as Tensor Processing Units (TPUs) or Neural Processing Units (NPUs) are optimized for machine learning and neural network computations.

d) Heterogeneous Computing:

Contemporary systems combine different types of cores (big and little) for optimized performance and power consumption—a concept known as heterogeneous computing or big.LITTLE architecture.

KEY MILESTONES IN MICROPROCESSOR EVOLUTION

Year	Microprocessor	Key Features
1971	Intel 4004	First 4-bit microprocessor
1974	Intel 8080	8-bit, used in early personal computers
1978	Intel 8086	16-bit, started x86 architecture
1982	Intel 80286	Introduced protected mode
1985	Intel 80386	32-bit architecture, virtual memory
1993	Intel Pentium	Superscalar architecture
2001	AMD Athlon 64	Successful 64-bit architecture
2005	Intel Dual Core	Start of multi-core era
2020	Apple M1	ARM-based SoC with high performance and efficiency

FUTURE OF MICROPROCESSORS

- The future of microprocessor technology is promising and continues to evolve with cutting-edge innovations:
- Quantum Computing: Though still in early development, quantum processors could solve problems beyond the capabilities of classical computers.



- Neuromorphic Computing: Mimicking the human brain's neural architecture to achieve more efficient cognitive processing.
- As artificial intelligence, robotics, and autonomous systems grow, microprocessors will continue to play a critical role in enabling smarter, faster, and more energy-efficient computing solutions.

KEY TAKEAWAYS

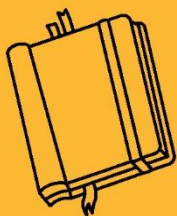
- Microprocessors are the core processing units that execute instructions and control system operations, evolving significantly from early 8-bit designs (like the 8085) to modern 64-bit architectures used in advanced computing systems.
- They are classified based on architecture (CISC vs RISC) and bit-width (8-bit, 16-bit, 32-bit, 64-bit), which influences their performance, complexity, and application areas.
- A microprocessor's instruction set, which includes data transfer, arithmetic, logical, and control instructions, is fundamental to its operation and determines how software interacts with the hardware.



OUTCOMES OF CHAPTER 1 (INTRODUCTION TO MICROPROCESSORS)

By the end of this chapter, students will be able to:

- **Explain the historical development** of microprocessors, from early computing technologies like vacuum tubes and transistors to modern integrated circuits.
- **Identify key milestones** in the evolution of microprocessors, including the introduction of Intel 4004 and the transition through 8-bit, 16-bit, 32-bit, and 64-bit architectures.
- **Describe the technological advancements** that led to the creation of the first microprocessor and the role of key innovators in this field.
- **Understand the transition to multi-core processors** and how it addressed limitations in speed, heat, and power consumption.
- **Analyze current trends** in microprocessor development, including System on Chip (SoC) design, energy efficiency, AI integration, and heterogeneous computing.
- **Discuss the future of microprocessors**, including emerging areas like quantum computing and neuromorphic architectures.
- **Recognize the classification and features** of microprocessors based on architecture (CISC vs. RISC) and bit-width.
- **Appreciate the significance of instruction sets** in determining how microprocessors interact with software to perform computing tasks.



INTRODUCTION TO MICROPROCESSORS



SUB LESSON 2.1

CLASSIFICATION BASED ON ARCHITECTURE (CISC VS RISC)

OBJECTIVES OF CHAPTER 2 (CLASSIFICATION OF MICROPROCESSOR)

To explain the classification of microprocessors based on architecture and bit-width.

To understand microprocessor families.

CLASSIFICATION BASED ON ARCHITECTURE (CISC VS RISC)

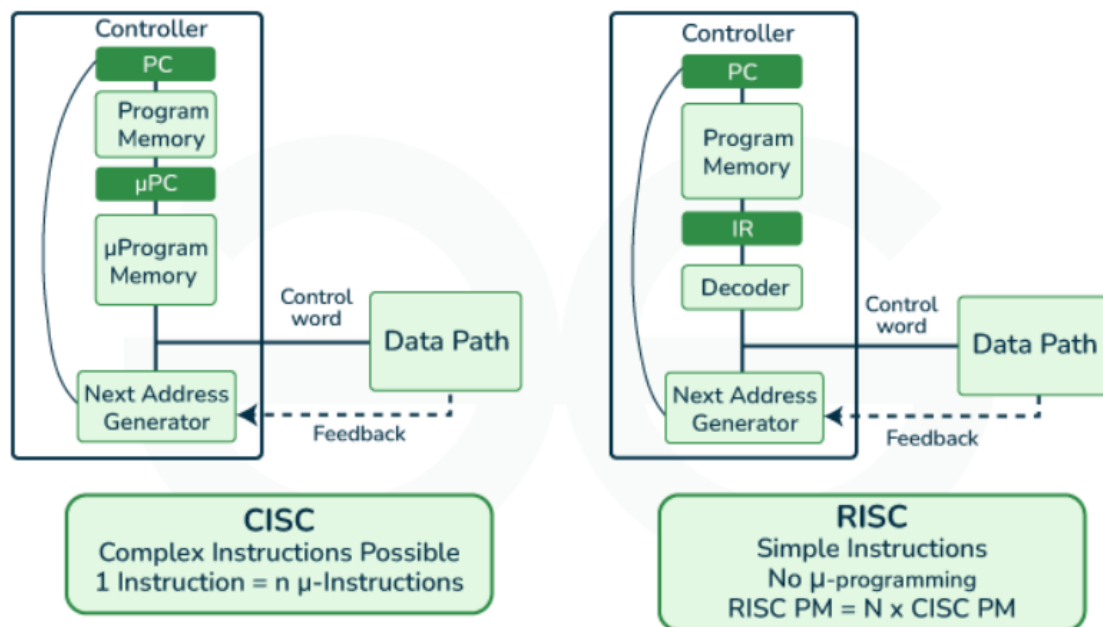


FIGURE 2.1 COMPARISON OF RISC AND CISC

The figure 2.1 illustrates a comparative structural representation of CISC (Complex Instruction Set Computing) and RISC (Reduced Instruction Set Computing) architectures. These are two fundamental approaches in processor design, each with its own advantages and trade-offs. Below is a detailed description of both architectures based on the image.



CISC ARCHITECTURE (COMPLEX INSTRUCTION SET COMPUTING)

- On the left side of the figure 2.1, the CISC architecture is depicted. The Controller consists of several key components:
- PC (Program Counter): Maintains the current execution address.
- Program Memory: Stores the machine-level instructions.
- Microprogram Memory (μ Program Memory) and Microprogram Counter (μ PC):
- In CISC, each high-level instruction is broken down into multiple micro-instructions.
- These micro-instructions are stored in the microprogram memory, which acts as an intermediary step between the program instructions and execution.
- Next Address Generator: Determines the next instruction to be fetched.
- Data Path: Executes instructions based on control word signals sent by the controller.
- Control Word & Feedback Loop: Provides execution feedback to ensure proper sequencing.

CHARACTERISTICS OF CISC:

- Complex Instructions:
- CISC processors support a large and complex set of instructions, some of which can perform multi-step operations (e.g., memory access and computation) in a single instruction.
- Micro programming:
- Instructions are broken down into multiple micro-instructions stored in microprogram memory.
- Lower Program Memory Requirements:
- Since each instruction performs more tasks, fewer instructions are needed to execute a program.
- Slower Execution Speed:
- Due to the need for decoding complex instructions, execution time per instruction is generally higher.



RISC ARCHITECTURE (REDUCED INSTRUCTION SET COMPUTING)

- On the right side of the Figure 2.1, the RISC architecture is depicted. The Controller structure differs from CISC:
- PC (Program Counter): Keeps track of the current instruction's address.
- Program Memory: Stores simpler, fixed-length instructions.
- Instruction Register (IR) and Decoder:
- In RISC, instructions are directly decoded without requiring microprogram memory.
- The Decoder translates instructions into control signals.
- Next Address Generator: Controls sequential instruction execution.
- Data Path: Executes simple operations with direct control signals.
- Control Word & Feedback Loop: Ensures proper data flow and execution.

CHARACTERISTICS OF RISC:

- Simple Instructions:
- Each instruction performs a small task (e.g., a single arithmetic or memory operation).
- No Microprogramming:
- Instructions are directly executed by hardware, reducing overhead.
- Larger Program Memory Requirement:
- More instructions are needed to accomplish a task compared to CISC.
- Faster Execution Speed:
- Each instruction executes in a single clock cycle, leading to higher performance.

KEY DIFFERENCES BETWEEN CISC AND RISC



Feature	CISC (Complex Instruction Set Computing)	RISC (Reduced Instruction Set Computing)
Instruction Complexity	Complex, multi-step instructions	Simple, single-cycle instructions
Microprogramming	Required (uses micro-instructions)	Not required (hardware-decoded)
Execution Speed	Slower (variable execution time)	Faster (fixed execution time)
Memory Usage	More efficient (fewer instructions)	Requires more instructions (larger memory)
Control Unit	Microcoded control	Hardwired control

- proved 3D graphics and Internet capabilities.
- Pentium 4 (2000-2008): Featured Hyper-Threading for better multitasking.

ARM MICROPROCESSORS

Introduction

ARM (Advanced RISC Machine) processors are based on the Reduced Instruction Set Computing (RISC) architecture. Unlike Intel's x86 processors, ARM CPUs focus on power efficiency and performance per watt, making them ideal for mobile and embedded systems.

Architecture and Features

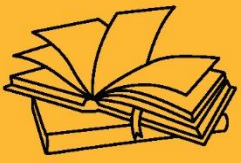
- 32-bit and 64-bit Architecture: Modern ARM processors support both 32-bit (ARMv7) and 64-bit (ARMv8 and ARMv9) architectures.
- RISC-based Design: Executes simpler instructions with fewer clock cycles, improving efficiency.
- Low Power Consumption: Designed for battery-powered devices.



- Scalability: ARM cores can be found in smartphones, tablets, IoT devices, and even servers.
- Integrated GPUs and AI Acceleration: Many ARM processors include dedicated graphics and AI processing units.
- Multiple Cores: Supports multi-core processing for better performance.

KEY TAKEAWAYS

- They are classified based on architecture (CISC vs RISC) and bit-width (8-bit, 16-bit, 32-bit, 64-bit), which influences their performance, complexity, and application areas.
- A microprocessor's instruction set, which includes data transfer, arithmetic, logical, and control instructions, is fundamental to its operation and determines how software interacts with the hardware.



INTRODUCTION TO MICROPROCESSORS





SUB LESSON 2.2

CLASSIFICATION BASED ON BIT-WIDTH (8-BIT, 16-BIT, 32-BIT, 64-BIT)

CLASSIFICATION BASED ON BIT-WIDTH (8-BIT, 16-BIT, 32-BIT, 64-BIT)

Classification of Microprocessors Based on Bit-Width

Microprocessors are classified based on their bit-width, which defines how much data they can process at a time and how large an addressable memory space they can access. The bit-width of a processor directly impacts its performance, speed, and computational capabilities. Over the decades, microprocessors have evolved from 8-bit designs to 64-bit and beyond, enabling higher processing power, efficiency, and advanced computing applications.

This classification includes 8-bit, 16-bit, 32-bit, and 64-bit microprocessors, each playing a crucial role in the advancement of computer architecture. This document provides an in-depth understanding of these classifications, their features, advantages, disadvantages, and real-world applications.

8-BIT MICROPROCESSORS

DEFINITION

An 8-bit microprocessor is a central processing unit (CPU) that processes 8 bits of data simultaneously. It means that the registers, buses, and ALU (Arithmetic Logic Unit) operate on 8-bit data at a time.

KEY FEATURES

- The word size (basic unit of data processing) is 8 bits.
- Can address up to 64 KB (2^{16} bytes) of memory.



- Supports simple instruction sets, making it easy to program.
- Uses an 8-bit data bus and typically a 16-bit address bus.
- Operates at relatively low clock speeds (typically 1 MHz to 8 MHz).

EXAMPLES

- Intel 8008 (1972): The first commercially available 8-bit processor.
- Intel 8080 (1974): Used in early personal computers.
- Zilog Z80 (1976): Popular in embedded systems and home computers.
- MOS 6502 (1975): Used in early gaming consoles like the Nintendo Entertainment System (NES).

16-BIT MICROPROCESSORS

DEFINITION

A 16-bit microprocessor processes 16 bits of data at a time. This improvement over 8-bit processors allows for better performance and efficiency in computing operations.

KEY FEATURES

- The word size is 16 bits.
- Can address up to 1 MB (2^{20} bytes) of memory.
- Uses a 16-bit data bus and typically a 20-bit address bus.
- Supports multiplication and division instructions, unlike most 8-bit processors.
- Enhanced instruction set for more efficient computing.

EXAMPLES



- Intel 8086 (1978): The foundation of modern x86 architecture.
- Intel 80286 (1982): Improved version of 8086 with protected mode capabilities.
- Motorola 68000 (1979): Used in early Apple Macintosh computers.
- Texas Instruments TMS9900: One of the earliest 16-bit microprocessors.

32-BIT MICROPROCESSORS

Definition

A 32-bit microprocessor processes 32 bits of data at a time. These processors became mainstream in the 1990s, providing a balance between performance and efficiency.

KEY FEATURES

- The word size is 32 bits.
- Can address up to 4 GB (2^{32} bytes) of memory.
- Uses a 32-bit data bus and typically a 32-bit address bus.
- Supports floating-point arithmetic for better mathematical computations.
- Capable of running advanced operating systems like Windows and Linux.

EXAMPLES

- Intel 80386 (1985): First widely used 32-bit processor.
- Intel 80486 (1989): Introduced integrated floating-point processing.
- Pentium series (1993-2000s): Dominated the consumer market.
- Motorola 68020: Used in early Mac computers.
- ARM Cortex-A series: Used in smartphones and tablets.

64-BIT MICROPROCESSORS

DEFINITION



- A 64-bit microprocessor processes 64 bits of data at a time, allowing for superior computational power, large memory addressing, and high-performance computing.

KEY FEATURES

- The word size is 64 bits.
- Can address up to 16 exabytes (2^{64} bytes) of memory.
- Uses a 64-bit data bus and a 64-bit address bus.
- Supports advanced parallel processing and virtualization.
- Optimized for multi-core architectures.

EXAMPLES

- AMD Opteron (2003): First commercially successful 64-bit x86 processor.
- Intel Core i series: Modern high-performance CPUs.
- Apple M1/M2 Chips: ARM-based 64-bit processors for MacBooks.
- ARM Cortex-A72/A76: Found in high-end smartphones.

KEY POINTS

The classification of microprocessors based on bit-width defines their processing power, memory capacity, and application scope.

- 8-bit processors are ideal for basic embedded systems.
- 16-bit processors allow for better performance and multitasking.
- 32-bit processors enable advanced computing and operating systems.
- 64-bit processors dominate modern high-performance computing.

As technology advances, we may see further developments in 128-bit and quantum processors, shaping the future of computing.

- Bus: Processes 16-bit data at a time.
- 20-bit Address Bus: Supports 1 MB (2^{20} bytes) memory addressing.
- Clock Speed: Ranges from 5 MHz to 10 MHz.



- Segmented Memory Architecture: Divides memory into segments (Code, Data, Stack, Extra) for efficient addressing.
- Instruction Set: Supports assembly-level programming with 117 basic instructions.
- General-Purpose Registers: Eight 16-bit registers for arithmetic and data storage.
- Pipelining: Uses a 6-byte instruction queue for improved efficiency.
- BIU and EU (Bus Interface Unit and Execution Unit): Separates instruction fetching and execution to increase speed.

PENTIUM MICROPROCESSOR

INTRODUCTION

The Intel Pentium series, introduced in 1993, revolutionized computing with its super-scalar architecture and high-speed processing. It was the first microprocessor in the fifth-generation x86 family.

ARCHITECTURE AND FEATURES

- 32-bit Architecture: Supports 32-bit data processing.
- 64-bit Data Bus: Allows faster data transfers.
- Addressing Capacity: Can address up to 4 GB (2^{32} bytes) of memory.
- Clock Speed: Ranged from 60 MHz (Pentium 1) to 3 GHz+ (Pentium 4).
- Superscalar Architecture: Allows multiple instructions to be executed simultaneously.
- Branch Prediction: Reduces execution delays by predicting the flow of a program.
- Dual Instruction Pipeline: Improves performance by executing two instructions at once.
- Floating-Point Unit (FPU): Integrated into the CPU for faster arithmetic calculations.

GENERATIONS OF PENTIUM PROCESSORS

- Pentium (1993-1996): Introduced dual pipelines and FPU improvements.
- Pentium Pro (1995): Enhanced performance with dynamic execution.
- Pentium II (1997): Included MMX technology for multimedia processing.
- Pentium III (1999): Improved 3D graphics and Internet capabilities.



- Pentium 4 (2000-2008): Featured Hyper-Threading for better multitasking.

ARM MICROPROCESSORS

INTRODUCTION

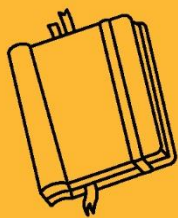
ARM (Advanced RISC Machine) processors are based on the Reduced Instruction Set Computing (RISC) architecture. Unlike Intel's x86 processors, ARM CPUs focus on power efficiency and performance per watt, making them ideal for mobile and embedded systems.

ARCHITECTURE AND FEATURES

- 32-bit and 64-bit Architecture: Modern ARM processors support both 32-bit (ARMv7) and 64-bit (ARMv8 and ARMv9) architectures.
- RISC-based Design: Executes simpler instructions with fewer clock cycles, improving efficiency.
- Low Power Consumption: Designed for battery-powered devices.
- Scalability: ARM cores can be found in smartphones, tablets, IoT devices, and even servers.
- Integrated GPUs and AI Acceleration: Many ARM processors include dedicated graphics and AI processing units.
- Multiple Cores: Supports multi-core processing for better performance.

KEY TAKEAWAYS

- They are classified based on architecture (CISC vs RISC) and bit-width (8-bit, 16-bit, 32-bit, 64-bit), which influences their performance, complexity, and application areas.
- A microprocessor's instruction set, which includes data transfer, arithmetic, logical, and control instructions, is fundamental to its operation and determines how software interacts with the hardware.



INTRODUCTION TO MICROPROCESSORS





SUB LESSON 2.3

POPULAR MICROPROCESSOR FAMILIES (8085, 8086, PENTIUM, ARM)

POPULAR MICROPROCESSOR FAMILIES (8085, 8086, PENTIUM, ARM)

Microprocessors are the backbone of modern computing, powering everything from basic embedded systems to high-performance computers and mobile devices. Over the years, several microprocessor families have played a crucial role in technological advancement. Among them, the 8085, 8086, Pentium, and ARM processor families have significantly shaped computing history.

8085 MICROPROCESSOR

INTRODUCTION

The Intel 8085 microprocessor was introduced in 1976 as an improvement over the Intel 8080. It is an 8-bit microprocessor designed for embedded systems and low-power applications.

ARCHITECTURE AND FEATURES

- 8-bit Data Bus: Processes 8-bit data at a time.
- 16-bit Address Bus: Can access up to 64 KB of memory.
- Clock Speed: Typically 3 MHz.
- Single Power Supply: Operates on +5V, unlike its predecessor (8080), which required multiple voltages.
- Fewer Support Chips Required: Integrated features reduce the need for external ICs.
- Instruction Set: Supports 246 instructions, including arithmetic, logical, branching, and control operations.
- Registers: Contains six general-purpose registers (B, C, D, E, H, L) and special-purpose registers like Accumulator (A) and Flag Register.



8086 MICROPROCESSOR

INTRODUCTION

The Intel 8086, launched in 1978, was a 16-bit microprocessor that laid the foundation for modern x86 architecture. It introduced powerful features, making it a popular choice for early personal computers.

ARCHITECTURE AND FEATURES

- 16-bit Data Bus: Processes 16-bit data at a time.
- 20-bit Address Bus: Supports 1 MB (2^{20} bytes) memory addressing.
- Clock Speed: Ranges from 5 MHz to 10 MHz.
- Segmented Memory Architecture: Divides memory into segments (Code, Data, Stack, Extra) for efficient addressing.
- Instruction Set: Supports assembly-level programming with 117 basic instructions.
- General-Purpose Registers: Eight 16-bit registers for arithmetic and data storage.
- Pipelining: Uses a 6-byte instruction queue for improved efficiency.
- BIU and EU (Bus Interface Unit and Execution Unit): Separates instruction fetching and execution to increase speed.

PENTIUM MICROPROCESSOR

INTRODUCTION

The Intel Pentium series, introduced in 1993, revolutionized computing with its super-scalar architecture and high-speed processing. It was the first microprocessor in the fifth-generation x86 family.

ARCHITECTURE AND FEATURES

- 32-bit Architecture: Supports 32-bit data processing.
- 64-bit Data Bus: Allows faster data transfers.
- Addressing Capacity: Can address up to 4 GB (2^{32} bytes) of memory.



- Clock Speed: Ranged from 60 MHz (Pentium 1) to 3 GHz+ (Pentium 4).
- Superscalar Architecture: Allows multiple instructions to be executed simultaneously.
- Branch Prediction: Reduces execution delays by predicting the flow of a program.
- Dual Instruction Pipeline: Improves performance by executing two instructions at once.
- Floating-Point Unit (FPU): Integrated into the CPU for faster arithmetic calculations.

GENERATIONS OF PENTIUM PROCESSORS

- Pentium (1993-1996): Introduced dual pipelines and FPU improvements.
- Pentium Pro (1995): Enhanced performance with dynamic execution.
- Pentium II (1997): Included MMX technology for multimedia processing.
- Pentium III (1999): Improved 3D graphics and Internet capabilities.
- Pentium 4 (2000-2008): Featured Hyper-Threading for better multitasking.

ARM MICROPROCESSORS

INTRODUCTION

ARM (Advanced RISC Machine) processors are based on the Reduced Instruction Set Computing (RISC) architecture. Unlike Intel's x86 processors, ARM CPUs focus on power efficiency and performance per watt, making them ideal for mobile and embedded systems.

ARCHITECTURE AND FEATURES

- 32-bit and 64-bit Architecture: Modern ARM processors support both 32-bit (ARMv7) and 64-bit (ARMv8 and ARMv9) architectures.
- RISC-based Design: Executes simpler instructions with fewer clock cycles, improving efficiency.
- Low Power Consumption: Designed for battery-powered devices.
- Scalability: ARM cores can be found in smartphones, tablets, IoT devices, and even servers.



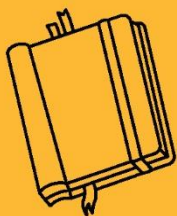
- Integrated GPUs and AI Acceleration: Many ARM processors include dedicated graphics and AI processing units.
- Multiple Cores: Supports multi-core processing for better performance.

KEY TAKEAWAYS

- 8085 and 8086 laid the foundation for early computing, offering basic and protected mode operations.
- Pentium introduced superscalar architecture, multitasking, and multimedia enhancements.
- ARM excels in mobile/embedded systems with power efficiency, scalability, and integrated AI/GPU features.

OUTCOMES OF CHAPTER 2 (CLASSIFICATION OF MICROPROCESSOR)

- They are classified based on architecture (CISC vs RISC) and bit-width (8-bit, 16-bit, 32-bit, 64-bit), which influences their performance, complexity, and application areas.
- A microprocessor's instruction set, which includes data transfer, arithmetic, logical, and control instructions, is fundamental to its operation and determines how software interacts with the hardware.



INTRODUCTION TO MICROPROCESSORS





SUB LESSON 3.1

DEFINITION AND IMPORTANCE OF INSTRUCTION SET

OBJECTIVES OF CHAPTER 3 (INSTRUCTION SET AND COMPUTER LANGUAGES)

To explain the importance of instruction set and to understand languages.

DEFINITION AND IMPORTANCE OF INSTRUCTION SET

Microprocessors follow different types of Instruction Set Architectures (ISA), each optimized for specific computing needs. The two primary categories are:

COMPLEX INSTRUCTION SET COMPUTING (CISC)

DEFINITION: CISC processors support complex, multi-step instructions that can execute in multiple clock cycles

FEATURES:

- Rich instruction set (hundreds of instructions).
- Microprogrammed control unit for decoding instructions.
- Fewer lines of assembly code, making programming easier.
- Examples: Intel x86 (8086, Pentium), Motorola 68000.

ADVANTAGES:

- Efficient use of memory.
- Easier to program due to high-level instructions.

DISADVANTAGES:

- Slower execution speed compared to RISC.
- More power consumption.

REDUCED INSTRUCTION SET COMPUTING (RISC)

Definition: RISC processors use a simplified instruction set with faster execution per instruction.

FEATURES:



- Fixed-length instructions, usually one cycle per instruction.
- Hardwired control unit, leading to faster decoding.
- More general-purpose registers for optimized performance.
- Examples: ARM, MIPS, PowerPC.

ADVANTAGES:

- Higher execution speed.
- Lower power consumption.

DISADVANTAGES:

- Requires more instructions for complex tasks.
- Larger code size compared to CISC.

VERY LONG INSTRUCTION WORD (VLIW)

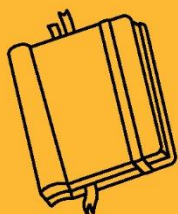
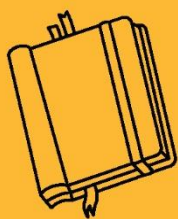
- Used in high-performance processors like Intel Itanium.
- Executes multiple operations in a single instruction.

SPECIAL-PURPOSE INSTRUCTION SETS

- Digital Signal Processors (DSPs): Used in audio and image processing.
- Microcontroller ISAs: Optimized for embedded systems (e.g., AVR, PIC).

KEY TAKEAWAYS

- The instruction set defines the functions a microprocessor can perform.
- CISC supports complex instructions (easier coding, slower execution), RISC uses simple, fast ones.



INTRODUCTION TO MICROPROCESSORS



SUB LESSON 3.2

TYPES OF INSTRUCTIONS (DATA TRANSFER, ARITHMETIC, LOGICAL, CONTROL)

TYPES OF INSTRUCTIONS (DATA TRANSFER, ARITHMETIC, LOGICAL, CONTROL)

DATA TRANSFER INSTRUCTIONS

Instruction	Description
MOV Rd, Rs	Transfer data from source register to destination register.
MVI R, data	Move immediate data to register.
LDA addr	Load accumulator from memory address.
STA addr	Store accumulator into memory address.
LDAX Rp	Load accumulator indirectly via register pair (BC/DE).
STAX Rp	Store accumulator indirectly via register pair (BC/DE).
XCHG	Exchange contents of DE and HL register pairs.

ARITHMETIC INSTRUCTIONS

Instruction	Description
ADD R	Add register to accumulator.
ADI data	Add immediate data to accumulator.
SUB R	Subtract register from accumulator.



SUI data	Subtract immediate data from accumulator.
INR R	Increment register by 1.
DCR R	Decrement register by 1.
INX Rp	Increment register pair.
DCX Rp	Decrement register pair.
DAD Rp	Add register pair to HL.
CMP R	Compare register with accumulator (sets flags).

LOGICAL INSTRUCTIONS

Instruction	Description
ANA R	Logical AND between register and accumulator.
ANI data	AND immediate data with accumulator.
ORA R	Logical OR between register and accumulator.
ORI data	OR immediate data with accumulator.
XRA R	Logical XOR between register and accumulator.
XRI data	XOR immediate data with accumulator.
CMA	Complement accumulator (invert all bits).
CMC	Complement carry flag.
STC	Set carry flag.
RLC, RRC, RAL, RAR	Rotate accumulator left/right (with/without carry).

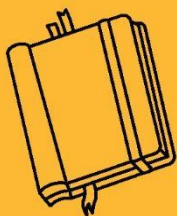


CONTROL INSTRUCTIONS

Instruction	Description
JMP addr	Unconditional jump to memory address.
JC addr	Jump if carry flag is set.
JZ addr	Jump if zero flag is set.
JNZ addr	Jump if zero flag is not set.
CALL addr	Call subroutine at given address.
RET	Return from subroutine.
PCHL	Load program counter with HL contents.
HLT	Halt the microprocessor (stop execution).
NOP	No operation – does nothing (used for delay or timing).

KEY TAKEAWAYS

- Data Transfer: Moves data between registers, memory, and accumulator (e.g., MOV, LDA).
- Arithmetic and Logical: Performs calculations and bitwise operations (e.g., ADD, CMP, ORA)



INTRODUCTION TO MICROPROCESSORS





SUB LESSON 3.3

ASSEMBLY LANGUAGE VS MACHINE LANGUAGE

ASSEMBLY LANGUAGE VS MACHINE LANGUAGE

MACHINE LANGUAGE

Machine Language is the most basic programming language understood directly by the computer's central processing unit (CPU). It consists entirely of binary code (0s and 1s). Each instruction in machine language is executed directly by the CPU without any translation.

For example, an instruction in machine language might look like:

```
10110000 01100001
```

This sequence has a specific meaning for the processor, such as loading a value into a register.

Characteristics:

- **Binary Format:** Instructions are written in binary digits.
- **Hardware-Specific:** The instruction set is tailored to a particular CPU architecture.
- **Fast Execution:** Since there is no need for translation, execution is immediate.
- **Difficult to Understand:** Human programmers find it extremely hard to read, debug, or modify.

ASSEMBLY LANGUAGE

Assembly Language is a low-level language that uses mnemonics and symbolic names instead of binary codes. It acts as a human-readable representation of machine instructions.

For example, instead of the binary code shown above, you would write:

```
MOV AL, 61H
```



This instruction means "move the hexadecimal value 61 into the AL register."

Characteristics:

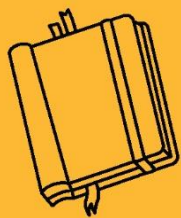
- Symbolic Representation: Uses short codes (like MOV, ADD, SUB) for operations.
- Requires Assembler: An assembler converts the assembly code into machine code.
- More Readable: Easier to understand and write compared to binary.
- Hardware-Specific: Still dependent on the architecture of the CPU.

KEY TAKEAWAYS

- Machine Language uses binary code, fast but hard for humans to understand and debug.
- Assembly Language uses mnemonics (e.g., MOV, ADD) and is easier to write, requiring an assembler.
- Both are hardware-specific and critical for low-level programming and control.

OUTCOMES OF CHAPTER 3 (INSTRUCTION SET AND COMPUTER LANGUAGES)

- A microprocessor's instruction set, which includes data transfer, arithmetic, logical, and control instructions, is fundamental to its operation and determines how software interacts with the hardware.
- The internal organization of microprocessors, such as that of the 8085, involves various registers (general-purpose and special-purpose), memory organization, and bus structures, all of which work together to ensure effective data flow and system integration.



INTRODUCTION TO MICROPROCESSORS



SUB LESSON 4.1

TYPES OF BUSES (DATA BUS, ADDRESS BUS, CONTROL BUS).

OBJECTIVES OF CHAPTER 4 (MICROPROCESSOR BUS ORGANIZATION)

To describe microprocessor bus organization and its role in data communication and interfacing.

TYPES OF BUSES (DATA BUS, ADDRESS BUS, CONTROL BUS).

For a microprocessor to function effectively, it requires auxiliary components to facilitate communication and data processing. A microprocessor-based system typically consists of the following:

1. **Microprocessor (CPU):** The central component responsible for executing instructions and managing system operations.
2. **Memory:** Used for storing both programs and data. It is categorized into:
 - **ROM (Read-Only Memory):** Non-volatile memory used for permanent data storage.
 - **RAM (Random Access Memory):** Volatile memory that temporarily stores data during program execution.
3. **Input/Output (I/O) Devices:** Components that enable interaction with the external environment. Examples include:
 - **Input Devices:** Keyboard, mouse, sensors.
 - **Output Devices:** Monitor, LED, LCD display.
4. **System Bus:** A communication link that interconnects the microprocessor, memory, and I/O devices, ensuring smooth data transmission.

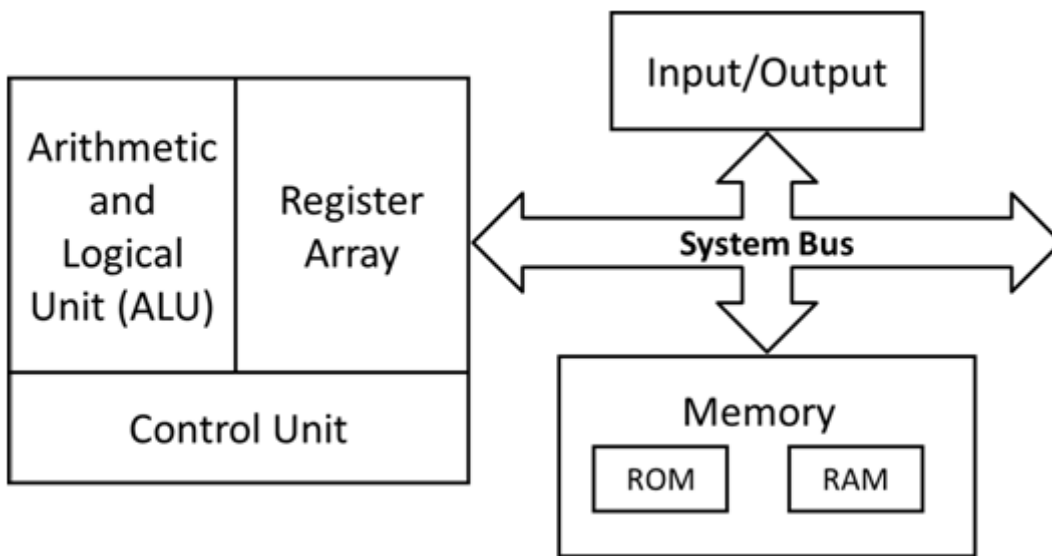


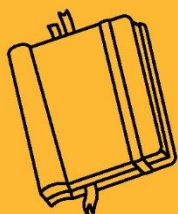
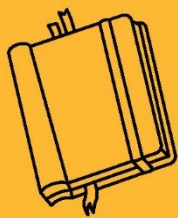
Figure 4.1 Microprocessor general block diagram

Figure 4.1 shows the general block diagram of Microcontroller. The microprocessor acts as the master controller of the system, executing stored instructions and coordinating with memory and I/O devices to complete tasks. Depending on the level of sophistication, microprocessor-based systems can range from simple devices like washing machines to complex computing systems such as personal computers and industrial automation units.

By integrating these components over a common communication framework, microprocessors form the backbone of modern computing technology, driving innovation across various industries.

KEY TAKEAWAYS

- Data Bus transfers actual data; it is bidirectional.
- Address Bus carries memory/I/O addresses; it is unidirectional.
- Control Bus manages operations like reading, writing, and instruction fetch.



INTRODUCTION TO MICROPROCESSORS



SUB LESSON 4.2

BUS STRUCTURE AND COMMUNICATION IN A MICROPROCESSOR.

BUS STRUCTURE AND COMMUNICATION IN A MICROPROCESSOR

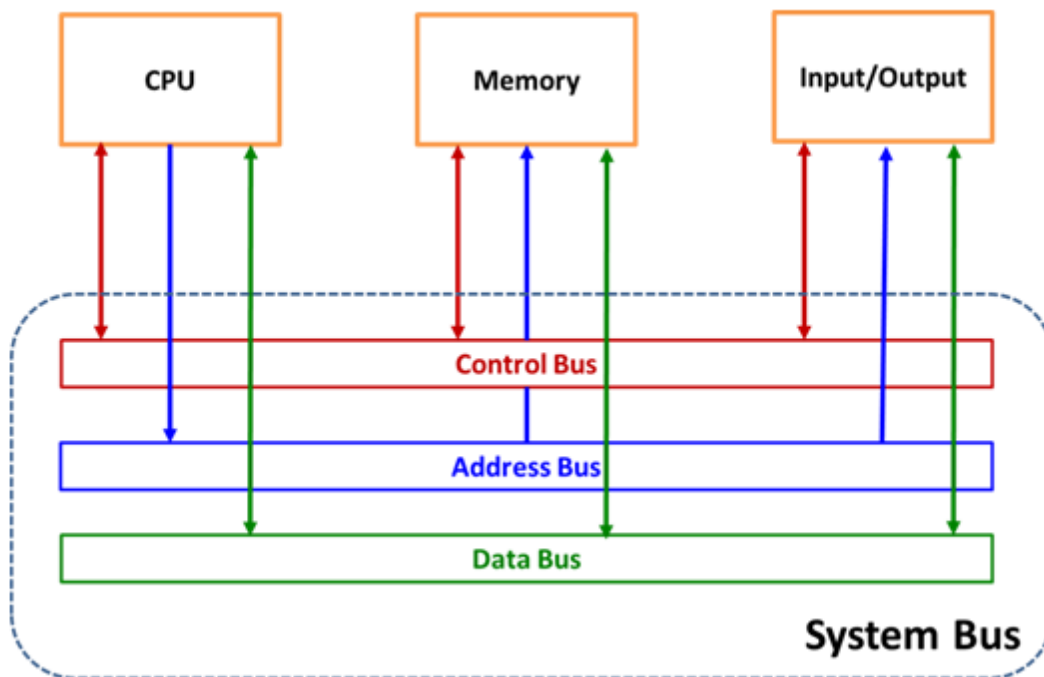


Figure 4.2 Bus system of Microprocessor

The system bus is a communication pathway that interconnects various components of a microprocessor-based system. It consists of three main types of buses:

ADDRESS BUS:

- A set of unidirectional lines responsible for carrying memory or I/O device addresses.
- Determines the destination for data storage or retrieval within the system.



- The width of the address bus dictates the maximum addressing capability of the system (e.g., a 16-bit address bus can address up to 2^{16} memory locations).

DATA BUS

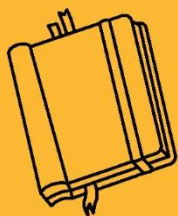
- A bidirectional bus used for data transfer between the microprocessor and memory or I/O devices.
- Each wire in the data bus carries a single bit of binary data.
- A wider data bus allows more data to be transferred simultaneously, enhancing processing speed.

CONTROL BUS

- A set of signals used to control and coordinate microprocessor operations.
- Includes essential control signals such as Read, Write, and Opcode Fetch.
- Determines the direction of data flow and synchronizes interactions between different components of the system.

KEY TAKEAWAYSS

- The system bus interconnects CPU, memory, and I/O devices for seamless communication.
- Address bus determines destination, data bus transfers data, and control bus directs operation flow.
- Efficient bus communication ensures smooth system functioning and faster data processing.



INTRODUCTION TO MICROPROCESSORS





SUB LESSON 4.3

ROLE OF SYSTEM BUSES IN MEMORY AND I/O INTERFACING

ROLE OF SYSTEM BUSES IN MEMORY AND I/O INTERFACING

These components communicate via a shared communication pathway called the system bus, which ensures seamless interaction between the microprocessor, memory, and peripherals. In this system architecture, the microprocessor acts as the master, responsible for executing stored programs and coordinating with connected peripherals. Depending on the complexity of the application, microprocessor-based systems can range from simple household appliances like washing machines to advanced computing devices such as personal computers and industrial automation systems.

KEY TAKEAWAYS

- System buses (data, address, control) are essential for communication between CPU, memory, and I/O devices.
- Address bus selects memory or I/O location; data bus transfers data; control bus coordinates operations.
- Efficient bus interfacing ensures smooth data exchange and device communication.

OUTCOMES OF CHAPTER 4 (MICROPROCESSOR BUS ORGANIZATION)

- Illustrate the structure and function of system buses and their significance in memory and I/O interfacing.
- The internal organization of microprocessors, such as that of the 8085, involves various registers (general-purpose and special-purpose), memory organization, and bus structures, all of which work together to ensure effective data flow and system integration.



INTRODUCTION TO MICROPROCESSORS



SUB LESSON 5.1

DEFINITION AND IMPORTANCE OF ADDRESSING MODES

OBJECTIVES OF CHAPTER 5 (ADDRESSING MODES IN MICROPROCESSORS)

To explore addressing modes and the programming model of the 8085 microprocessor in detail.

DEFINITION AND IMPORTANCE OF ADDRESSING MODES

In microprocessor programming, an addressing mode refers to the method used to specify operands (data) for operations. It defines how the CPU identifies and accesses the data required for an instruction. Each instruction in a microprocessor operates on some data, and addressing modes help specify where this data is located — whether in a register, memory location, or provided directly in the instruction itself.

Understanding addressing modes is essential in assembly language programming because they directly influence how instructions are written, how efficient they are, and how flexibly they can operate on data. Addressing modes enable programmers to write compact and efficient programs that make effective use of the system's resources.

The 8085 microprocessor supports several addressing modes, each tailored for specific scenarios. These modes determine whether the operand is located in memory, stored in a register, or included in the instruction itself. For instance, if a constant value is used in an operation, it can be provided directly in the instruction (immediate addressing). If data needs to be retrieved from a specific memory location, direct or indirect addressing modes are used. If the data already exists in registers, register addressing is more suitable.



KEY TAKEAWAYS

- Addressing modes determine how operands are accessed in instructions.
- They enhance programming flexibility and efficiency.
- Crucial for writing optimized and modular assembly programs.



INTRODUCTION TO MICROPROCESSORS





SUB LESSON 5.2

TYPES OF ADDRESSING MODES (IMMEDIATE, DIRECT, INDIRECT, REGISTER)

TYPES OF ADDRESSING MODES (IMMEDIATE, DIRECT, INDIRECT, REGISTER)

The 8085 microprocessor uses several addressing modes to locate operands efficiently. Each mode serves a specific purpose and is used depending on the location and nature of the data. The primary addressing modes supported by the 8085 are Immediate, Direct, Indirect, and Register Addressing.

IMMEDIATE ADDRESSING MODE

In this mode, the operand is explicitly specified in the instruction. That means the data to be operated on is provided immediately after the opcode. For example:

```
MVI A, 32H
```

This instruction moves the hexadecimal value 32H directly into the accumulator register A. Since the operand is part of the instruction, no memory fetch is needed after the instruction itself, making this mode fast and suitable for constants.

DIRECT ADDRESSING MODE

In direct addressing, the memory address of the operand is given explicitly in the instruction. The microprocessor reads the value from the specified memory location. Example:

```
LDA 2050H
```

This command loads the accumulator with the contents of memory location 2050H. It is useful when data is stored at a known or fixed address in memory.

INDIRECT ADDRESSING MODE



Here, the address of the operand is provided indirectly through a register pair, such as HL, BC, or DE. The instruction does not contain the actual address but refers to the memory location pointed to by the register pair. Example:

```
MOV A, M
```

In this instruction, M refers to the memory location pointed to by the HL register pair. If HL = 3050H, then the data at 3050H is moved into the accumulator. This mode is powerful when working with arrays or buffers.

REGISTER ADDRESSING MODE

In register addressing, the operand is stored in one of the CPU's internal registers. The instruction specifies the register name directly. For example:

```
MOV B, C
```

This moves the content of register C into register B. Since registers are accessed more quickly than memory, this mode is the fastest and is preferred for frequent or temporary data manipulation.

Each of these addressing modes provides a way to interact with data in different ways. Immediate mode is great for constants, direct mode is ideal for fixed memory locations, indirect is flexible for memory blocks or arrays, and register mode is excellent for high-speed data processing. Choosing the appropriate addressing mode helps in optimizing both memory usage and execution time.

KEY TAKEAWAYS

- Immediate: Operand is part of the instruction.
- Direct: Address of operand is explicitly given.
- Indirect/Register: Address is in a register or a register pair, enabling dynamic data access.



INTRODUCTION TO MICROPROCESSORS





SUB LESSON 5.3

EXAMPLES OF ADDRESSING MODES IN 8085

EXAMPLES OF ADDRESSING MODES IN 8085

Understanding the concept of addressing modes becomes clearer when we study real examples using 8085 instructions. Each instruction below demonstrates a specific addressing mode and how data is fetched and processed.

Addressing Mode	Instruction Example	Explanation
Immediate	MVI A, 3EH	Loads 3EH directly into accumulator A.
Immediate	ADI 05H	Adds immediate value 05H to accumulator A.
Direct	LDA 2050H	Loads A with the value at memory location 2050H.
Direct	STA 3050H	Stores the content of A into memory location 3050H.
Indirect	MOV A, M	Moves data from memory (pointed by HL) to A.
Indirect	LDAX B	Loads A with data from address pointed by BC.
Register	MOV B, C	Copies contents of register C to B.
Register	ADD D	Adds content of register D to accumulator A.



KEY TAKEAWAYS

- MOV M, A (Indirect mode via HL pair).
- MVI A, 05H (Immediate mode).
- LDA 2050H (Direct mode) – demonstrates real usage of each mode in 8085 programming.

OUTCOMES OF CHAPTER 5 (ADDRESSING MODES IN MICROPROCESSORS)

- Addressing modes (Immediate, Direct, Indirect, Register) play a crucial role in defining how operands are accessed during instruction execution, directly affecting the efficiency and flexibility of programming.
- The internal organization of microprocessors, such as that of the 8085, involves various registers (general-purpose and special-purpose), memory organization, and bus structures, all of which work together to ensure effective data flow and system integration.



INTRODUCTION TO MICROPROCESSORS





SUB LESSON 6.1

REGISTERS IN 8085 (ACCUMULATOR, GENERAL-PURPOSE REGISTERS)

OBJECTIVES OF CHAPTER 6 (8085 PROGRAMMING MODEL)

- To understand the working of Registers in 8085
- To understand the working of Special purpose Register
- To understand the Memory flow

REGISTERS IN 8085 (ACCUMULATOR, GENERAL-PURPOSE REGISTERS)

Microprocessors are designed to perform multiple computing tasks and make decisions that influence the execution flow of a program. A microprocessor can be divided into three key functional units:

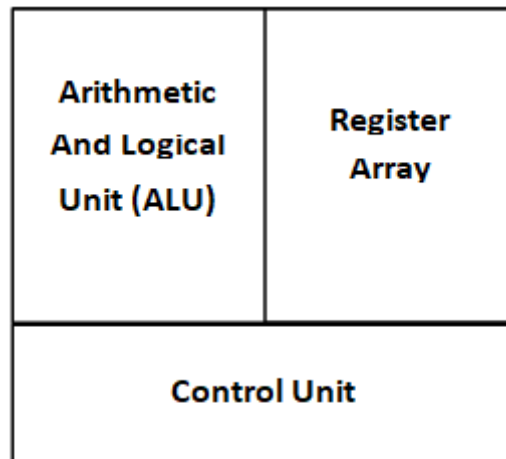


Figure 6.1 Microprocessor Unit

1. **Arithmetic and Logic Unit (ALU)**
2. **Register Array**



3. Control Unit

1. ARITHMETIC AND LOGIC UNIT (ALU):

The Arithmetic and Logic Unit (ALU) is the computational hub of the microprocessor. It executes various arithmetic operations, such as addition, subtraction, multiplication, and division, along with logical functions like AND, OR, and XOR (Exclusive-OR). Since the ALU is responsible for all mathematical and logical processing, it is often considered the "brain" of the microprocessor.

2. REGISTER ARRAY:

The Register Array comprises a set of small, high-speed storage locations, referred to as registers. These registers play a crucial role in temporarily storing data, instructions, and addresses while a program is being executed. Registers are commonly identified using letters such as B, C, D, E, H, and L. Their efficient data-handling capability significantly enhances the speed of the microprocessor by reducing reliance on external memory.

3. CONTROL UNIT:

The Control Unit orchestrates the entire operation of the microprocessor by generating the necessary timing and control signals. It ensures a smooth flow of data between the microprocessor, memory, and peripheral devices. The control unit manages the execution of instructions by decoding them and directing the appropriate system components. It utilizes the control bus to synchronize operations and regulate data exchange, thereby facilitating seamless program execution.

REGISTERS

8085 has several registers used for data storage and execution control:

Accumulator (A): A special 8-bit register used for arithmetic and logical operations.

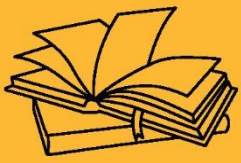
General-Purpose Registers (B, C, D, E, H, L): These six 8-bit registers can be used individually or in pairs (BC, DE, HL) for 16-bit operations.



Instruction Register (IR): Temporarily holds the instruction fetched from memory before execution.

KEY TAKEAWAYS

- Microprocessors are the core processing units that execute instructions and control system operations, evolving significantly from early 8-bit designs (like the 8085) to modern 64-bit architectures used in advanced computing systems.
- They are classified based on architecture (CISC vs RISC) and bit-width (8-bit, 16-bit, 32-bit, 64-bit), which influences their performance, complexity, and application areas.
- A microprocessor's instruction set, which includes data transfer, arithmetic, logical, and control instructions, is fundamental to its operation and determines how software interacts with the hardware.
- Addressing modes (Immediate, Direct, Indirect, Register) play a crucial role in defining how operands are accessed during instruction execution, directly affecting the efficiency and flexibility of programming.
- The internal organization of microprocessors, such as that of the 8085, involves various registers (general-purpose and special-purpose), memory organization, and bus structures, all of which work together to ensure effective data flow and system integration.



INTRODUCTION TO MICROPROCESSORS





SUB LESSON 6.2

SPECIAL PURPOSE REGISTERS (PROGRAM COUNTER, STACK POINTER, FLAGS)

REGISTERS IN 8085 (ACCUMULATOR, GENERAL-PURPOSE REGISTERS)

Microprocessors are designed to perform multiple computing tasks and make decisions that influence the execution flow of a program. A microprocessor can be divided into three key functional units:

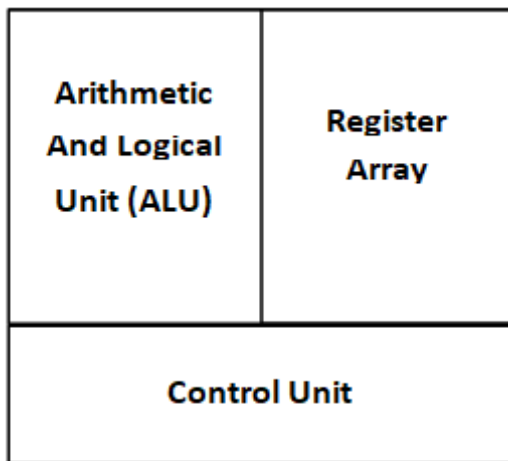


Figure 6.1 Microprocessor Unit

4. **Arithmetic and Logic Unit (ALU)**
5. **Register Array**
6. **Control Unit**

1. Arithmetic and Logic Unit (ALU):

The Arithmetic and Logic Unit (ALU) is the computational hub of the microprocessor. It executes various arithmetic operations, such as addition, subtraction, multiplication, and division, along



with logical functions like AND, OR, and XOR (Exclusive-OR). Since the ALU is responsible for all mathematical and logical processing, it is often considered the "brain" of the microprocessor.

2. Register Array:

The Register Array comprises a set of small, high-speed storage locations, referred to as registers. These registers play a crucial role in temporarily storing data, instructions, and addresses while a program is being executed. Registers are commonly identified using letters such as B, C, D, E, H, and L. Their efficient data-handling capability significantly enhances the speed of the microprocessor by reducing reliance on external memory.

3. Control Unit:

The Control Unit orchestrates the entire operation of the microprocessor by generating the necessary timing and control signals. It ensures a smooth flow of data between the microprocessor, memory, and peripheral devices. The control unit manages the execution of instructions by decoding them and directing the appropriate system components. It utilizes the control bus to synchronize operations and regulate data exchange, thereby facilitating seamless program execution.

Registers

8085 has several registers used for data storage and execution control:

Accumulator (A): A special 8-bit register used for arithmetic and logical operations.

General-Purpose Registers (B, C, D, E, H, L): These six 8-bit registers can be used individually or in pairs (BC, DE, HL) for 16-bit operations.

Instruction Register (IR): Temporarily holds the instruction fetched from memory before execution.

SPECIAL PURPOSE REGISTERS (PROGRAM COUNTER, STACK POINTER, FLAGS)

Program Counter (PC)

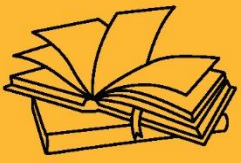


- The Program Counter (PC) is a 16-bit register responsible for managing the sequence of instruction execution.
- Since memory locations have 16-bit addresses, the PC is also 16-bit.
- It holds the address of the next instruction to be fetched and executed.
- After fetching an instruction, the PC is automatically incremented to point to the next memory location.
- Stack Pointer (SP)
- The Stack Pointer (SP) is a 16-bit register that acts as a memory pointer.
- It holds the memory address of the top of the stack, which is an area of memory used for storing temporary data, function calls, and interrupt handling.
- The stack's starting address is initialized by loading a 16-bit value into the SP register.
- Flag Register: A special 8-bit register that stores condition flags (Zero, Sign, Carry, Parity, and Auxiliary Carry) based on the result of operations.



KEY TAKEAWAYS

- **Special Purpose Registers in 8085** are essential for managing instruction flow and system control, distinct from general-purpose registers.
- The **Program Counter (PC)** is a 16-bit register that keeps track of the address of the next instruction to be executed. It auto-increments after each instruction fetch.
- The **Stack Pointer (SP)** is another 16-bit register that points to the top of the stack, a dedicated memory area for temporary data storage, function return addresses, and interrupt handling.
- The **Flag Register** is an 8-bit register that holds status flags—Zero (Z), Sign (S), Carry (CY), Parity (P), and Auxiliary Carry (AC)—which indicate the outcomes of arithmetic and logical operations.
- These registers are crucial for **program control, decision-making, and efficient memory and data management** in the microprocessor.
- Together, these registers help **coordinate instruction sequencing, data storage, and conditional operations**, enabling structured and controlled program execution.



INTRODUCTION TO MICROPROCESSORS





SUB LESSON 6.3

MEMORY ORGANIZATION AND DATA FLOW IN 8085)

MEMORY ORGANIZATION AND DATA FLOW IN 8085

The 8085 microprocessor has a 16-bit address bus, enabling it to access 64KB of memory (from 0000H to FFFFH). Memory in 8085 is organized in a linear fashion, and it can be divided into two main segments: program memory (used to store instructions) and data memory (used to store variables and data).

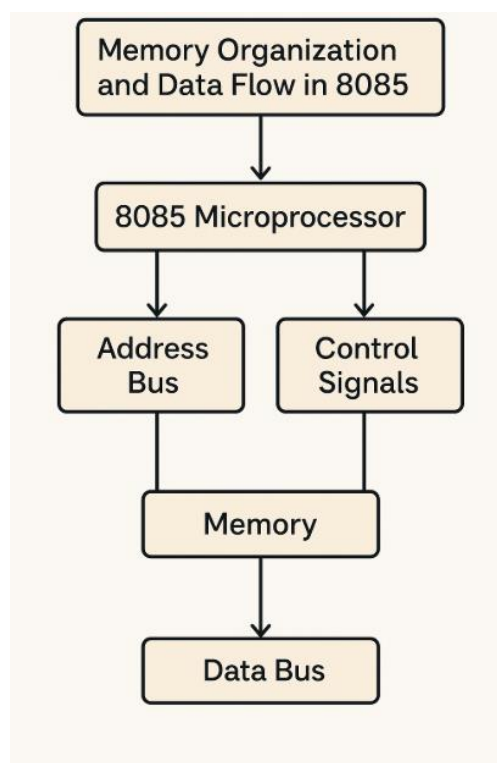


Figure 6.2 Data flow diagram of Microprocessor

As shown in figure 6.2 the data flow in 8085 begins when the microprocessor fetches an instruction from memory. The address of the instruction is placed on the address bus, and the corresponding data is fetched through the 8-bit data bus. Control signals such as RD (Read) and



WR (Write) manage the direction of data transfer. When RD is activated, data is read from memory or I/O. When WR is activated, data is written to memory or an output device.

Internally, the 8085 uses the Program Counter (PC) to keep track of the next instruction, and the Accumulator (A) and other general-purpose registers (B, C, D, E, H, L) are used for temporary storage and processing. The HL register pair is often used to point to memory locations during data manipulation.

Efficient memory organization and clear data flow ensure seamless execution of instructions and enable precise control over hardware operations.

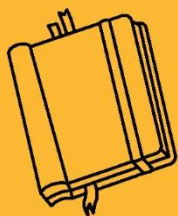
KEY TAKEAWAYS

- 8085 uses a 16-bit address bus to access 64KB memory.
- Memory is segmented and interfaced using control signals.
- Data flows between CPU, memory, and I/O via system buses.



OUTCOMES OF CHAPTER 6 (8085 PROGRAMMING MODEL)

- **Understand the internal architecture of the 8085 microprocessor**, including the roles of the Arithmetic and Logic Unit (ALU), Control Unit, and Register Array.
- **Explain the function and use of various registers in 8085**, including general-purpose registers (B, C, D, E, H, L), the Accumulator, and register pairs for 16-bit operations.
- **Describe the purpose and operation of special purpose registers** such as the Program Counter (PC), Stack Pointer (SP), and Flag Register, and how they control instruction flow and decision-making.
- **Illustrate how memory is organized in 8085**, including segmentation of program and data memory, and how the address and data buses facilitate access to memory.
- **Analyze the data flow between CPU, memory, and I/O devices**, and understand the role of control signals (RD, WR) in read/write operations.
- **Apply knowledge of registers and memory flow to understand instruction execution**, program sequencing, stack operations, and conditional branching in microprocessor-based systems.



MICROPROCESSOR ARCHITECTURE AND INTERFACING



SUB LESSON 7.1

MICROPROCESSOR ARCHITECTURE AND IT'S OPERATION

OBJECTIVES OF CHAPTER 7 (MICROPROCESSOR ARCHITECTURE)

- Understand the basic structure of a microprocessor and identify its main components such as the ALU, control unit, registers, and system buses.
- Explain the function of each major component within a microprocessor and how they interact during instruction execution.
- Describe the instruction cycle (Fetch-Decode-Execute-Write-back) and understand how instructions are processed within the microprocessor.
- Illustrate the role of buses (data, address, and control buses) in facilitating communication between the microprocessor, memory, and I/O devices.
- Explain the PIN diagram of 8085.

MICROPROCESSOR ARCHITECTURE AND ITS OPERATIONS

A microprocessor is an integrated circuit that performs the essential functions of a computer's central processing unit (CPU). It is the brain of any computing system, designed to execute instructions, carry out logical and arithmetic operations, and control other parts of the system. The architecture of a microprocessor refers to the arrangement and interaction of its internal components, which determine its efficiency, speed, and overall functionality.

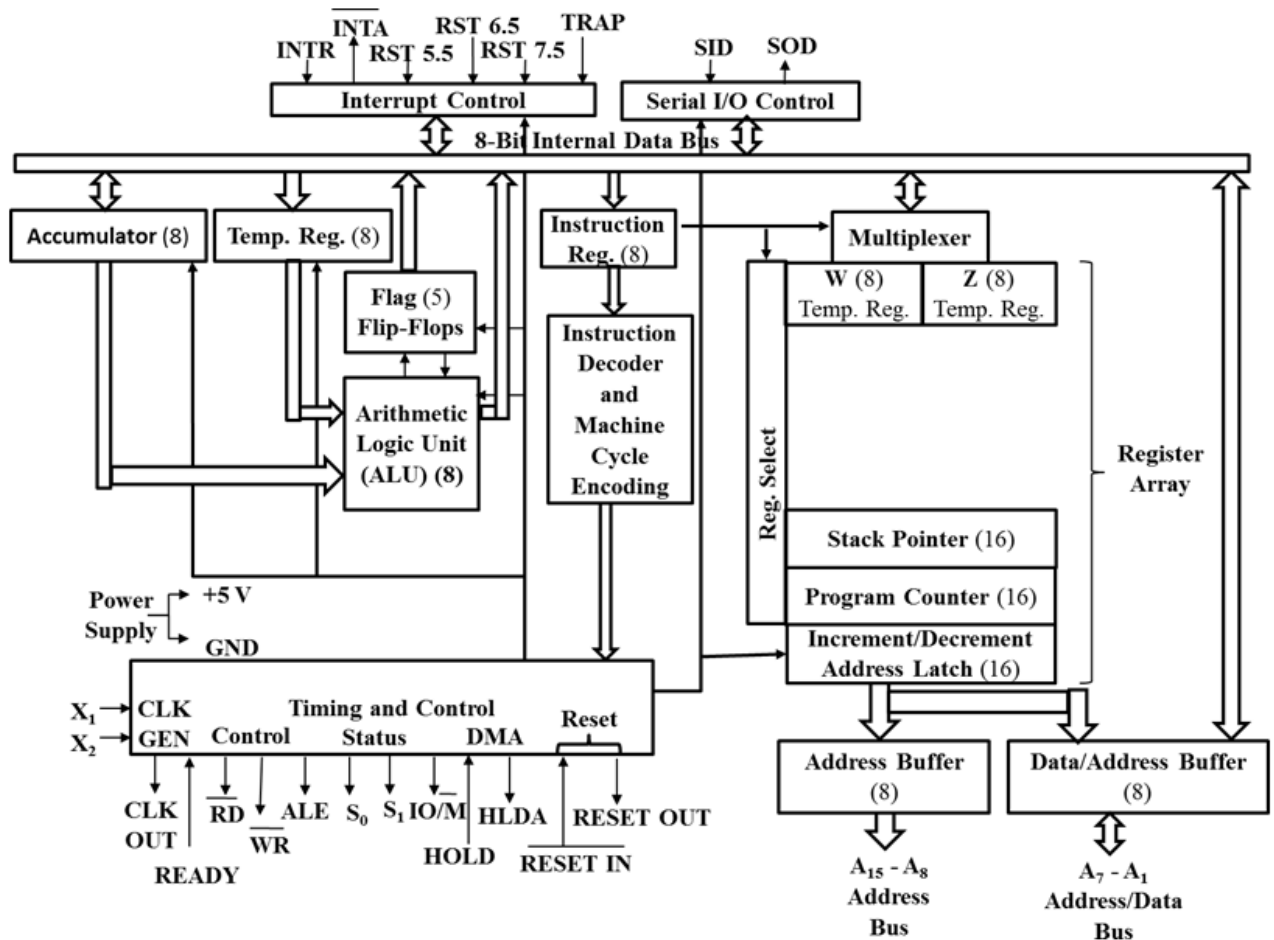
OVERVIEW OF MICROPROCESSOR ARCHITECTURE

The architecture of a microprocessor involves several key components working together to process instructions and manage data flow. These components include:

Arithmetic Logic Unit (ALU):

This is a fundamental part of the processor responsible for performing arithmetic operations

(such as addition, subtraction, multiplication, and division) as well as logical operations (such as AND, OR, NOT, and XOR). The ALU takes input from registers and performs computations as directed by the control unit.



Registers:

Registers are small, high-speed storage locations within the microprocessor. They temporarily hold data, instructions, and intermediate results that are used during processing. Some common types of registers include the Accumulator, Program Counter, Stack Pointer, and Status Register. Registers facilitate quick data access, reducing the time taken to fetch and store data from memory.

Control Unit (CU):

The control unit acts as the coordinator within the microprocessor. It interprets instructions



fetches from memory and generates control signals to direct the operation of other units like the ALU and registers. The CU ensures that instructions are executed in the correct sequence and oversees data transfers between components.

- **SystemBuses:**

Buses are communication pathways that connect different components of a computer system. The microprocessor uses three primary types of buses:

- **Data Bus:** Transfers actual data between the processor, memory, and I/O devices.
- **Address Bus:** Carries the address of the memory location to or from which data is being transferred.
- **Control Bus:** Transmits control signals from the control unit to other parts of the system, such as memory and I/O devices.

KEY OPERATIONS OF A MICROPROCESSOR

The microprocessor performs a sequence of operations to execute instructions and process data. These operations can be grouped into four main stages, forming what is known as the Instruction Cycle or Fetch-Decode-Execute Cycle:

Fetch:

The first step involves retrieving the instruction from memory. The program counter (PC) holds the address of the instruction to be fetched. This address is placed on the address bus, and the instruction is retrieved from the corresponding memory location via the data bus. Once the instruction is fetched, the PC is incremented to point to the next instruction.

Decode:

After fetching, the instruction is sent to the instruction decoder, which interprets the binary code to determine what operation needs to be performed. The control unit breaks down the instruction into signals that activate various parts of the processor to carry out the task.



Execute:

During the execution phase, the decoded instruction is carried out. This may involve performing an arithmetic or logic operation in the ALU, transferring data from one register to another, or interacting with memory or I/O devices.

Write-back:

After execution, the result of the operation is written back to a register or a specific memory location. This final step ensures that the outcomes of the instruction is stored and ready for future use or further processing.

FUNCTIONAL FLOW OF INSTRUCTION PROCESSING

The entire operation of the microprocessor can be visualized as a continuous loop of fetching, decoding, executing, and writing back results. This cycle is repeated millions of times per second in modern microprocessors. The ability of a microprocessor to perform this cycle quickly and efficiently determines its performance.

Each instruction processed may affect the values stored in registers or the contents of memory. The microprocessor must also manage program control instructions, such as jumps or branches, which alter the normal sequence of instruction execution.

ROLE OF CLOCK AND TIMING

A crucial component that regulates microprocessor operations is the clock. The clock generates a sequence of regular pulses that synchronize all the internal activities of the processor. Each stage of the instruction cycle is timed according to these clock pulses, ensuring proper sequencing and coordination among components.

Faster clock speeds generally result in quicker instruction processing, although other architectural factors like pipelining, caching, and instruction set efficiency also play important roles.



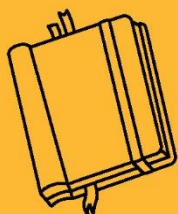
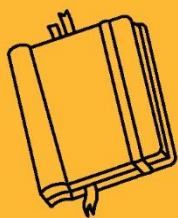
MICROPROCESSOR DESIGN CONSIDERATIONS

Designing a microprocessor involves balancing multiple factors:

- **Instruction Set Architecture (ISA):** Defines the set of instructions that the microprocessor can execute. It includes types of operations, addressing modes, and data types supported.
- **Data Width:** Microprocessors can be 8-bit, 16-bit, 32-bit, or 64-bit, which indicates the size of data units it can process at a time. A wider data path allows for more data to be processed simultaneously.
- **Performance Features:** Modern microprocessors incorporate features like pipelining, parallel execution units, and multi-core designs to enhance performance.
- **Power Efficiency:** Especially in embedded systems and mobile devices, power consumption is a critical concern. Efficient microarchitectures help prolong battery life.

KEY TAKEAWAYS

- 8085 architecture includes ALU, registers, control unit, and buses.
- It executes fetch-decode-execute cycle.
- Operations are synchronized using a clock and control signals.



MICROPROCESSOR ARCHITECTURE AND INTERFACING



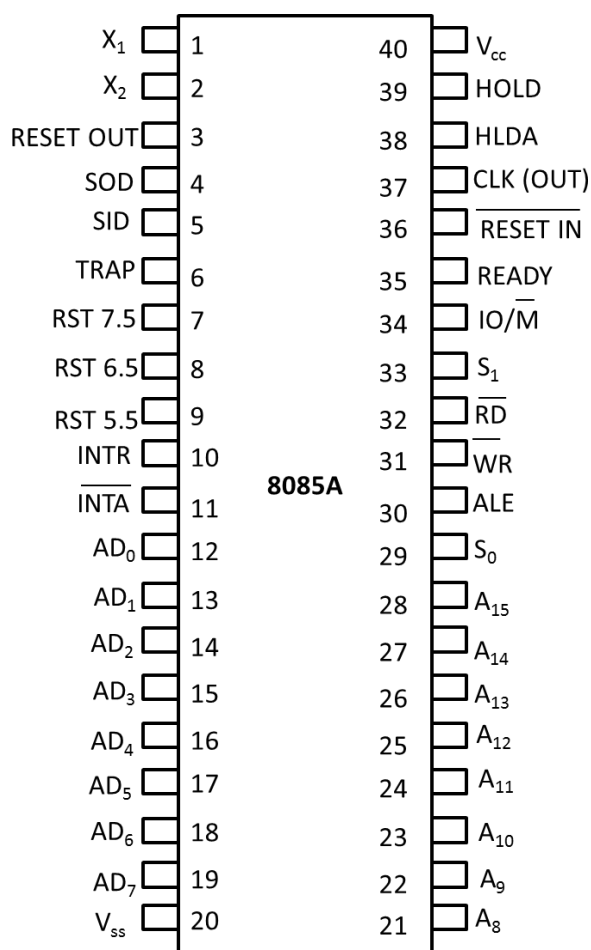


SUB LESSON 7.2

PIN DIAGRAM OF 8085

8085 MICROPROCESSOR PIN DIAGRAM

The Intel 8085 is an 8-bit microprocessor introduced by Intel in the mid-1970s. It features a simple and efficient architecture, widely used in educational, industrial, and embedded system applications. Understanding its pin diagram is crucial for grasping how it communicates with external devices and performs various operations. The 8085 microprocessor comes in a 40-pin Dual-Inline Package (DIP). Each pin has a specific function, supporting data transfer, control, address decoding, and power management.





OVERVIEW OF 8085 PIN CONFIGURATION

The 8085 microprocessor requires external support to function properly. The 40 pins are classified based on their roles into the following groups:

POWER SUPPLY AND CLOCK SIGNALS

- Address and data buses
- Control and status signals
- Interrupts and externally initiated signals
- Serial input/output signals
- Other functional pins

Each of these plays a distinct role in enabling the microprocessor to interface with memory, I/O devices, and other subsystems effectively.

POWER SUPPLY AND CLOCK SIGNALS

These are essential for the operation and synchronization of the microprocessor.

Vcc (Pin 40):

This pin is connected to a +5V power supply, providing the required operating voltage for the chip.

Vss (Pin 20):

It is the ground pin and should be connected to 0V reference potential.

X1 and X2 (Pins 1 and 2):

These are connected to an external crystal oscillator (typically 6 MHz) to provide the clock frequency. The clock frequency generated internally is half of the external crystal frequency.

CLK (OUT) (Pin 37):

This output pin provides the system clock for synchronization with peripheral devices.

ADDRESS AND DATA BUSES

The 8085 uses address and data lines to interact with memory and I/O devices.

AD0–AD7 (Pins 12–19):



These lines serve as a multiplexed address/data bus. During the first part of a machine cycle, they carry the lower-order address (A0–A7), and during the second part, they carry data (D0–D7). This multiplexing helps reduce pin count.

A8–A15 (Pins 21–28):

These are the higher-order address lines. They carry the upper byte (most significant 8 bits) of the 16-bit memory address, used to access memory locations.

CONTROL AND STATUS SIGNALS

These pins coordinate data flow between the processor and external devices.

ALE (Address Latch Enable) – Pin 30:

This signal is used to demultiplex the A0–A7 bus. It indicates when the address is available on the multiplexed lines, allowing an external latch (like 74LS373) to separate the address from data.

RD (Read) – Pin 32:

This active-low signal tells memory or I/O devices that the processor is performing a read operation.

WR (Write) – Pin 31:

This active-low signal indicates that the processor is performing a write operation to memory or an I/O port.

IO/M – Pin 34:

This signal distinguishes between I/O and memory operations. When IO/M = 0, the operation is related to memory; when IO/M = 1, it refers to I/O.

S0 and S1 (Pins 29 and 33):

These status signals indicate the current machine cycle being executed, such as opcode fetch, memory read, memory write, etc. They help in decoding the state of the processor during execution.

S0 and

INTERRUPT AND EXTERNALLY INITIATED SIGNALS

Interrupts are crucial for handling asynchronous events and allowing peripheral devices to access the processor.

**INTR (Pin 10):**

This is a general-purpose interrupt request input. When enabled, it allows external devices to request interrupt service.

INTA (Interrupt Acknowledge) – Pin 11:

This is an output signal given by the processor in response to an INTR request. It informs the requesting device that its interrupt has been recognized.

RST7.5, RST6.5, RST5.5 (Pins 7, 8, and 9):

These are vectored interrupts. Each of these is assigned a fixed memory location for interrupt service routines, allowing fast handling of priority-based interrupts.

TRAP (Pin 6):

The TRAP is the highest priority and non-maskable interrupt. It is edge and level triggered, used for urgent tasks such as power failure or emergency shutdown.

HOLD (Pin 39):

This signal indicates that an external device is requesting control of the buses for Direct Memory Access (DMA).

HLDA (Hold Acknowledge) – Pin 38:

This is a response signal from the microprocessor to the HOLD request. It indicates that the processor has released the buses.

RESET IN (Pin 36):

This active-low input signal resets the microprocessor, clearing the program counter and other registers.

RESET OUT (Pin 3):

It provides a reset signal to other devices in the system when the processor is reset.

SERIAL I/O CONTROL SIGNALS

The 8085 supports basic serial communication using two dedicated lines.

SID (Serial Input Data) – Pin 5:

This pin is used to receive serial data input to the processor.

SOD (Serial Output Data) – Pin 4:

This is used to send serial data output from the processor.



FUNCTION OF KEY CONTROL PINS IN OPERATION

To understand the function of the 8085's pin configuration, it's helpful to analyze how these pins operate during a typical instruction execution:

When an instruction is being fetched, ALE goes high to indicate the availability of address bits on AD0–AD7.

S0 and S1 define the type of operation being executed.

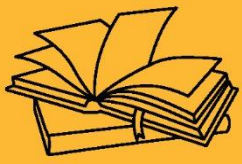
During a read or write operation, either RD or WR goes low.

In case of an interrupt (like INTR or TRAP), the processor halts the current execution, saves the program counter, and jumps to a predefined interrupt address.

HOLD and HLDA are especially useful in systems using DMA for high-speed data transfer.

KEY TAKEAWAYS

- 8085 has 40 pins including address/data lines and control signals.
- Key pins: ALE, RD, WR, IO/M differentiate operations.
- Pin functions support memory and I/O interfacing



MICROPROCESSOR ARCHITECTURE AND INTERFACING



SUB LESSON 7.3

BLOCK DIAGRAM OF THE 8085 MICROPROCESSOR

BLOCK DIAGRAM OF THE 8085 MICROPROCESSOR

The **Intel 8085 microprocessor** is a classic 8-bit processor, extensively studied in microprocessor courses due to its simple and clear architecture. A crucial step in understanding how the 8085 works is by analyzing its **block diagram**, which illustrates the internal structure and how its various units collaborate to execute instructions.

The block diagram of the 8085 is a conceptual representation showing the interaction between functional units. These include the Arithmetic and Logic Unit (ALU), registers, timing and control unit, interrupt system, buses, and other critical components.

OVERVIEW OF THE 8085 MICROPROCESSOR ARCHITECTURE

The 8085 is an **8-bit microprocessor** with a **16-bit address bus** and an **8-bit data bus**. It operates on a **+5V power supply** and has a **single-phase clock**. The processor is designed to process instructions sequentially using the **Fetch-Decode-Execute** cycle. Its internal structure consists of several functional blocks that enable instruction execution, data manipulation, and peripheral communication.

ARITHMETIC AND LOGIC UNIT (ALU)

The ALU is the computational heart of the microprocessor. It performs all **arithmetic operations** such as addition, subtraction, increment, and decrement, as well as **logical operations** like AND, OR, XOR, and comparison.

The ALU takes inputs from the **accumulator** and one of the **general-purpose registers**.

The **results** of operations are usually stored back in the accumulator.

The **status flags** (carry, zero, sign, parity, and auxiliary carry) are updated based on the result of operations.



ACCUMULATOR

The **accumulator** is a special-purpose 8-bit register that plays a central role in data operations.

It stores one of the operands during ALU operations and holds the final result.

The accumulator is **connected directly to the ALU**, enabling fast execution of operations.

It is also used in data transfer between the microprocessor and peripheral devices.

GENERAL-PURPOSE REGISTERS

The 8085 includes six 8-bit general-purpose registers labeled **B, C, D, E, H, and L**. These are used for temporary data storage during program execution.

These registers can also be **paired** to function as 16-bit registers: BC, DE, and HL.

The HL pair is especially significant as it can serve as a **memory pointer**.

PROGRAM COUNTER (PC)

The **program counter** is a 16-bit register that holds the **address of the next instruction** to be fetched from memory.

It is automatically incremented after each instruction fetch.

In case of jump or branch instructions, the PC is updated to point to a different memory address, ensuring non-sequential instruction execution when needed.

STACK POINTER (SP)

The **stack pointer** is another 16-bit register that always points to the **top of the stack** in memory.



The stack is a portion of RAM used to **store return addresses, subroutine values, or processor state**.

The SP is **automatically incremented or decremented** as data is pushed to or popped from the stack.

INSTRUCTION REGISTER AND DECODER

Once an instruction is fetched from memory, it is stored in the **instruction register**.

The **decoder** interprets the opcode and signals the control unit to perform the appropriate operation.

This is a crucial step in converting binary instructions into actual operations within the processor.

TIMING AND CONTROL UNIT

The **control unit** is responsible for coordinating all activities within the microprocessor.

- It generates **timing signals** that manage data flow and execution sequence.
- It produces **control signals** like RD, WR, IO/M, and ALE to synchronize operations with memory and I/O devices.
- The control unit also ensures that instructions are executed in the correct order.

INTERRUPT CONTROL

The 8085 supports **hardware interrupts** to allow the processor to respond to external events.

- The interrupt control unit manages five interrupts: **TRAP, RST7.5, RST6.5, RST5.5, and INTR**.
- **TRAP** is the highest priority, non-maskable interrupt.



- These interrupts allow real-time applications by letting the microprocessor pause the current task, service the interrupt, and resume.

SERIAL INPUT/OUTPUT CONTROL

The 8085 includes basic facilities for **serial communication** using:

- **SID (Serial Input Data)** and
- **SOD (Serial Output Data)** pins.

These enable simple data transfer over a single line, typically used in embedded systems and small-scale communication.

TEMPORARY REGISTER AND FLAGS

- The **temporary register** is used internally for processing intermediate results.
- The **flag register** is an 8-bit register where **five specific bits** reflect the outcomes of operations:

BUSES (DATA, ADDRESS, AND CONTROL)

The microprocessor communicates with memory and I/O devices using three types of buses:

- **Data Bus (8-bit):** Carries data to and from the processor.
- **Address Bus (16-bit):** Carries addresses of memory locations.
- **Control Bus:** Carries control signals to coordinate operations.

These buses enable external communication and internal coordination of instruction execution.



KEY TAKEAWAYS

- Shows interaction between ALU, registers, buses, and control unit.
- BIU and EU handle instruction fetch and execution.
- Clear visualization of internal data flow and control.

OUTCOMES OF CHAPTER 7 (MICROPROCESSOR ARCHITECTURE)

By the end of this chapter, learners will be able to:

- **Understand the internal structure** of the 8085 microprocessor and PIN diagram of it.
- **Identify and describe the role** each major functional unit including ALU, registers, program counter, stack pointer, and control unit.
- **Explain the data flow and execution process** through the Fetch-Decode-Execute cycle.
- **Recognize the importance of buses, control signals, and flags** in the operation of the 8085.
- **Comprehend how the interrupt control and serial I/O** blocks contribute to external device interfacing.



MICROPROCESSOR ARCHITECTURE AND INTERFACING



SUB LESSON 8.1

INTERNAL REGISTERS ORGANIZATION OF 8085 MICROPROCESSOR

OBJECTIVE OF CHAPTER 8 (REGISTERS ORGANIZATION)

- **Understand the internal register structure of the 8085 microprocessor**, including general-purpose and special-purpose registers.
- **Explore the types and functions of memory (RAM, ROM)** and the interaction of memory with the 8085 microprocessor.
- **Learn the concept and necessity of memory interfacing**, including decoding techniques and memory selection.
- **Examine I/O interfacing methods**, including memory-mapped and isolated I/O with relevant control signals.
- **Develop practical understanding of peripheral interfacing**, especially with devices like LEDs and seven-segment displays using I/O ports.

INTERNAL REGISTERS ORGANIZATION OF 8085 MICROPROCESSOR

The 8085 microprocessor, like other CPUs, consists of a number of internal registers that are essential for performing arithmetic and logic operations, handling data, managing program flow, and interacting with memory. These registers act as temporary storage elements and play a crucial role in the microprocessor's performance and functionality.

Understanding the **organization of internal registers** is fundamental in grasping how the 8085 executes instructions and processes data efficiently.

OVERVIEW OF INTERNAL REGISTERS

The 8085 microprocessor contains a variety of **internal registers**, categorized into two broad types:

General-purpose registers (used for temporary data storage)



Special-purpose registers (used for instruction control, flag handling, and stack management)

Together, these registers support smooth and fast data manipulation, enhance the control flow of the program, and enable decision-making operations.

GENERAL-PURPOSE REGISTERS (B, C, D, E, H, L)

The 8085 microprocessor includes **six 8-bit general-purpose registers** labeled as **B, C, D, E, H, and L**. These registers are versatile and primarily used for storing temporary data during instruction execution.

Each register can hold **8 bits** of data.

These registers can be used **individually** or in **register pairs** to perform **16-bit operations**.

BC (B and C)

DE (D and E)

HL (H and L)

The **HL register pair** is especially important because it can act as a **memory pointer**, meaning it can hold the address of a memory location where data is to be read or written. This enables **indirect addressing**, where the operand is accessed indirectly through a register.

ACCUMULATOR (A REGISTER)

The **accumulator** is an 8-bit special-purpose register and a core component of the Arithmetic and Logic Unit (ALU).

It temporarily stores data during **arithmetic and logical operations**.

Most operations are carried out between the accumulator and another register or memory location.

The **result of the operation** is stored back in the accumulator.

For example, in the instruction **ADD B**, the content of register B is added to the accumulator, and the result is stored in the accumulator.

FLAG REGISTER



The **flag register** is an 8-bit register that contains **five status flags**, each represented by a single bit. These flags reflect the result of arithmetic or logical operations and help the microprocessor make **decisions** based on certain conditions.

PROGRAM COUNTER (PC)

The **Program Counter** is a 16-bit special-purpose register that holds the **address of the next instruction** to be executed.

It **automatically increments** after each instruction fetch.

When executing **jump, call, or branch instructions**, the PC is updated with a new address.

It plays a critical role in maintaining the **sequential flow** of program execution.

The PC helps the microprocessor know **what to do next** in the program.

STACK POINTER (SP)

The **Stack Pointer** is also a 16-bit register that points to the **top of the stack** in memory.

The **stack** is a portion of memory used to **store temporary data**, return addresses, and processor states.

When data is **pushed** to the stack, the SP is **decremented**.

When data is **popped**, the SP is **incremented**.

The SP is particularly important in **function calls** and **interrupt handling**, where preserving and restoring data is necessary.

TEMPORARY REGISTER

Though not directly accessible to the programmer, the 8085 has **temporary internal registers** that are used by the ALU to hold intermediate values during complex operations.

These internal registers ensure **faster and smoother execution** by eliminating the need to store temporary results in main memory.

INSTRUCTION REGISTER

When an instruction is fetched from memory, it is stored in the **Instruction Register** temporarily before being decoded.



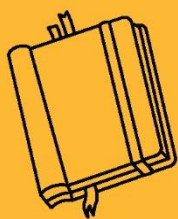
The **decoder** then interprets the instruction and activates the required micro-operations. This step is part of the **instruction cycle**, enabling the CPU to understand and carry out commands.

SUMMARY OF REGISTER USAGE

Register	Size	Usage
A (Accumulator)	8-bit	Arithmetic/logic operations
B, C, D, E, H, L	8-bit	General-purpose storage
PC	16-bit	Tracks instruction address
SP	16-bit	Points to stack top
Flag Register	8-bit	Stores condition flags
HL	16-bit	Acts as a memory pointer (indirect access)

KEY TAKEAWAYS

- Includes 8-bit and 16-bit registers for processing and addressing.
- Special-purpose registers control execution flow.
- Register pairs allow 16-bit operations.



MICROPROCESSOR ARCHITECTURE AND INTERFACING



SUB LESSON 8.2

MEMORY AND I/O DEVICES

MEMORY AND I/O DEVICES IN 8085 MICROPROCESSOR

In a microprocessor-based system, memory and input/output (I/O) devices are essential components that work alongside the microprocessor to form a complete computing unit. The microprocessor itself is just a data processing unit—it requires **memory to store data and instructions**, and **I/O devices** to interact with the external environment. The **8085 microprocessor**, being an 8-bit processor, supports both memory and I/O interfacing effectively, with a well-defined set of instructions and addressing techniques.

MEMORY IN MICROPROCESSOR SYSTEMS

Memory refers to the storage units that temporarily or permanently hold data and program instructions for the microprocessor. The **8085 microprocessor** has a **16-bit address bus**, which means it can address $2^{16} = 65,536$ memory locations, i.e., a total of **64 KB** of memory.

TYPES OF MEMORY

The memory used in microprocessor systems is broadly categorized into two types:

ROM (READ-ONLY MEMORY)

- **Non-volatile** memory: retains data even when power is off.
- Stores **firmware or boot code** required to start and initialize the system.
- Cannot be modified during normal execution.

Essential for embedded systems or microcontroller environments where fixed code is required.



RAM (RANDOM ACCESS MEMORY)

- **Volatile** memory: data is lost when power is turned off.
- Used for **temporary storage** of instructions, variables, and intermediate results.
- Readable and writable by the microprocessor.
- Acts as working memory during program execution.

In the 8085 system, both ROM and RAM are mapped within the 64 KB addressable memory space.

BYTE-ADDRESSABLE MEMORY

In the **8085 architecture**, memory is **byte-addressable**, which means each memory address refers to a single byte. Therefore, each address location holds exactly **8 bits** or **1 byte** of data. When multi-byte data is stored or accessed (e.g., 16-bit numbers), it occupies **two consecutive memory locations**.

I/O DEVICES IN 8085 MICROPROCESSOR

Input/Output (I/O) devices are the interface between the microprocessor and the external environment. These devices allow the system to **receive inputs** from the user or sensors and **produce outputs** such as lighting LEDs, displaying text, or activating motors.

EXAMPLES OF I/O DEVICES

- **Input Devices:** Keyboard, switches, sensors.
- **Output Devices:** LEDs, seven-segment displays, LCD screens, buzzers, actuators.

I/O devices are crucial for real-time interaction and control in digital systems and are connected through appropriate ports or interfaces.



IN AND OUT INSTRUCTIONS IN 8085

Mnemonics, Operand	Opcode (in HEX)	Bytes
IN Port-address	DB	2
OUT Port-Address	D3	2

The **8085 instruction set** provides two specific instructions for I/O communication:

****IN port-address****: Used to **read data** from an input device.

****OUT port-address****: Used to **send data** to an output device.

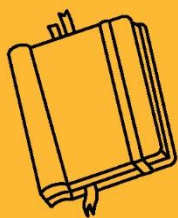
KEY POINTS

- Both instructions are **2 bytes long**: the first byte is the opcode, and the second byte is the 8-bit I/O port address.
- The 8085 can address $2^8 = 256$ **input ports** and **256 output ports**, because the I/O address is 8 bits wide.
- The microprocessor uses these instructions along with control signals to communicate with I/O devices.

These instructions are used in **isolated I/O mapping**, where a separate address space is reserved for I/O, independent of memory addressing.

KEY TAKEAWAYS

- Memory holds programs/data; I/O enables external communication.
- Interfaced via address decoding and control signals.
- Divided into mapped and separate I/O systems.



MICROPROCESSOR ARCHITECTURE AND INTERFACING





SUB LESSON 8.3

MEMORY INTERFACING IN 8085 MICROPROCESSOR

MEMORY INTERFACING IN 8085 MICROPROCESSOR

In a microprocessor-based system, **memory interfacing** refers to the process of connecting external memory chips (RAM or ROM) to the microprocessor so it can read instructions and store or retrieve data. The **Intel 8085 microprocessor**, being an 8-bit processor with a 16-bit address bus, can access up to **64 KB of memory ($2^{16} = 65,536$ locations)**. Efficient memory interfacing ensures that the right memory chip responds to the right address and timing signals, enabling the microprocessor to perform tasks smoothly and accurately.

PURPOSE OF MEMORY INTERFACING

Memory interfacing is a crucial part of system design for the following reasons:

- To **connect external memory devices** like ROM and RAM with the microprocessor.
- To ensure that only the **correct memory chip** is selected for any operation.
- To enable the processor to **access memory locations** in a structured, collision-free manner.
- To implement **address decoding**, which matches the address provided by the microprocessor to the address assigned to a specific memory chip.

Without proper interfacing, the processor may fail to read or write data accurately, leading to system errors or unexpected behavior.

MEMORY ADDRESSING IN 8085

The **8085 microprocessor** has:



- **16-bit address bus**, allowing it to access **64 KB of memory**, with each location storing **1 byte**.
- **8-bit data bus**, enabling it to read or write **8-bit data at a time**.

Memory locations are addressed from 0000H to FFFFH. The system designer must decide how much memory is to be allocated for **ROM** and **RAM**, and which address range each memory chip will occupy.

TYPES OF MEMORY INTERFACING

There are mainly two types of memory interfacing techniques used in 8085 systems:

A) INTERFACING ROM

ROM is typically used to store **firmware**, such as the system boot program or monitor code.

- It is generally connected at the **beginning of the address space**, starting at 0000H.
- During system reset, the microprocessor starts fetching instructions from this location.

B) INTERFACING RAM

RAM is used for **temporary data storage** during program execution.

- It can be placed anywhere in the address space after the ROM region.
- The range should be carefully planned to **avoid address overlap** with ROM or I/O mapped areas.

ADDRESS DECODING

Address decoding is the most important aspect of memory interfacing. It helps the system determine which memory chip is being referred to by a particular address.



There are two main decoding techniques:

FULL ADDRESS DECODING

- All **16 address lines** are used in the decoding process.
- Ensures **unique and non-overlapping selection** of memory chips.
- Requires a more complex decoder but offers accurate memory chip selection.
- Prevents multiple devices from responding to the same address.

Example: Using a **3-to-8 decoder (74LS138)** with inputs connected to higher address lines (A13, A14, A15), a memory range can be uniquely selected.

MEMORY MAPPING

The process of **assigning specific address ranges** to each memory chip in the system is known as memory mapping.

CONTROL SIGNALS FOR MEMORY INTERFACING

The 8085 microprocessor uses **specific control signals** to perform read and write operations:

- **$\overline{RD}$ (Read)**: Active-low signal to read from memory.
- **$\overline{WR}$ (Write)**: Active-low signal to write to memory.
- **$IO/\overline{M}$** : Determines whether the operation is on memory ($IO/\overline{M} = 0$) or I/O ($IO/\overline{M} = 1$).

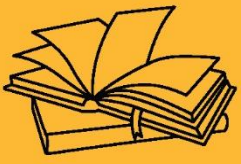
MEMORY MAPPED VS I/O MAPPED INTERFACING



Characteristics	Memory mapped I/O	I/O mapped I/O
Device Address	16 bit	8 Bit
Control signals for inputs	MEMR & MEMW	IOR & IOW
Instruction Available	All memory related instruction : LDA; STA; LDAX; STAX; MOV M,R; ADD M; SUB M	IN and OUT instructions only
Data Transfer	Between any register and I/O devices.	Between I/O device and Accumulator only.
Maximum Numbers of I/Os Possible	Memory Map (64K) is shared between I/Os and System memory.	I/O Mapped is independent of memory map; 256 Input and 256 output devices can be connected.
Execution Speed	13 T-State (LDA, STA, ..) 7 T-State (MOV M,R)	10 T-State
Hardware Requirement	More hardware is needed to decode 16 bit address	Less hardware is needed to decode 8 bit address
Other Feature	Arithmetic and logical operations are directly performed with I/O devices.	Not available

KEY TAKEAWAYS

- Requires address decoding logic and control signal configuration.
- Memory chips are selected using chip select (CS) logic.
- Ensures correct data access by CPU.



MICROPROCESSOR ARCHITECTURE AND INTERFACING





SUB LESSON 8.4

INTERFACING I/O DEVICES

INTERFACING I/O DEVICES WITH 8085 MICROPROCESSOR

In a microprocessor-based system, interfacing Input/Output (I/O) devices is essential for communication between the processor and the outside world. The 8085 microprocessor itself cannot interact with real-world devices directly—it requires a bridge in the form of input and output ports to send or receive data. I/O interfacing enables the system to receive data from input devices (like keyboards or sensors) and send control signals or display outputs to devices (like LEDs, displays, and motors).

INTRODUCTION TO I/O INTERFACING

I/O interfacing is the method by which peripheral devices are connected to the microprocessor. These devices allow the user or external systems to interact with the processor. The 8085 provides efficient I/O interfacing through its instruction set and bus system.

Examples of I/O devices include:

Input devices: Switches, keyboards, sensors.

Output devices: LEDs, buzzers, LCDs, seven-segment displays.

Efficient I/O interfacing ensures fast, reliable communication between the microprocessor and these external device.

TYPES OF I/O INTERFACING TECHNIQUES

There are two primary techniques to interface I/O devices with the 8085 microprocessor:

a) Memory-Mapped I/O



In this technique, I/O devices are treated as memory locations. This means:

I/O devices share the same address space as memory.

The microprocessor uses memory-related instructions (like LDA, STA, MOV M, etc.) to communicate with I/O.

Each I/O device is assigned a 16-bit address.

The same control signals ($R\bar{D}$ and $W\bar{R}$) are used for both memory and I/O operations.

ADVANTAGES:

More instructions available for I/O.

Simple integration with memory-mapped peripherals.

DISADVANTAGES:

Reduces available memory address space.

Requires 16-bit decoding logic.

CONTROL SIGNALS AND DEVICE SELECTION

The 8085 microprocessor uses control signals and address decoding logic to select the correct I/O device for communication.

Important Control Signals:

- $IO/\bar{M}$: Indicates whether the operation is related to memory ($IO/\bar{M} = 0$) or I/O ($IO/\bar{M} = 1$).
- $R\bar{D}$: Active-low signal for reading data.
- $W\bar{R}$: Active-low signal for writing data.
- $IO\bar{R}$: I/O Read signal generated during IN instruction.



- $\text{IO}\bar{\text{W}}$: I/O Write signal generated during OUT instruction.

These signals are combined with address lines to generate chip select (CS) signals for the correct I/O device.

ADDRESS DECODING FOR I/O DEVICES

To select a particular I/O device, the microprocessor places its port address on the address bus. This address must be decoded using logic circuits to enable the chip select signal for that specific port.

PRACTICAL I/O INTERFACING EXAMPLE: LEDS

LEDs are common output devices used to indicate binary or status outputs. To interface LEDs with the 8085 microprocessor:

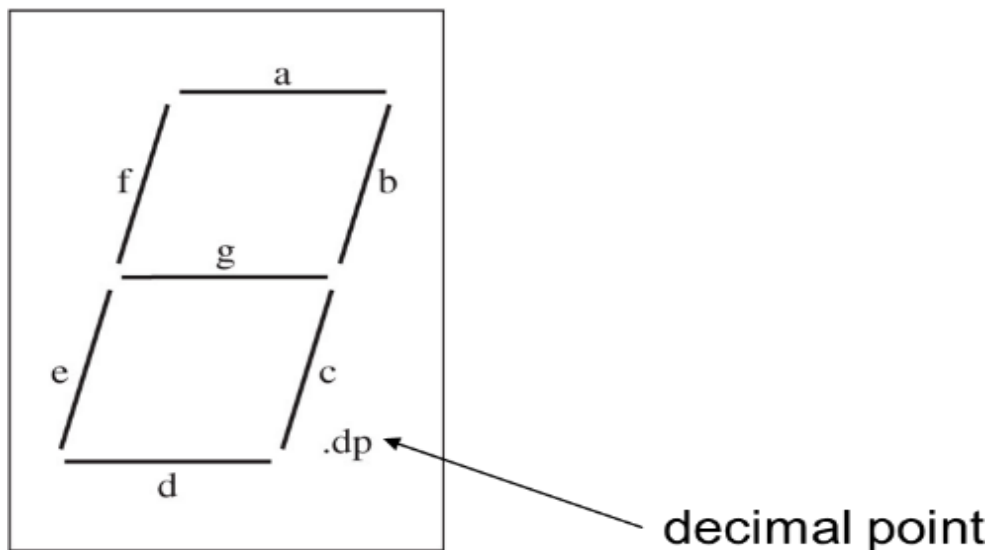
Connect the LEDs to the output port (like through 8255 Programmable Peripheral Interface).

Write data to the port using the OUT instruction.

Each bit of the data byte can turn ON or OFF a corresponding LED.

INTERFACING SEVEN-SEGMENT DISPLAY

A seven-segment display is used to show numeric data (and some characters). It contains seven LEDs (a to g) arranged to display numbers from 0 to 9.



Steps to Interface with 8085:

Use a port (like 8255) to send a 7-bit or 8-bit code to the display.

Generate BCD to seven-segment code using a lookup table in memory.

Activate corresponding segments by writing to the port using the OUT instruction.

Optionally use multiplexing for multiple-digit displays.

Seven-segment displays are widely used in counters, timers, digital clocks, and meters.

KEY TAKEAWAYS

- Uses IN and OUT instructions for I/O communication.
- Requires decoding of I/O addresses.
- Can be mapped via memory-mapped or I/O-mapped methods.



OUTCOMES OF CHAPTER 8 (REGISTERS ORGANIZATION)

After studying this chapter, learners will be able to:

- Explain the types of I/O interfacing (memory-mapped and isolated I/O) and understand their differences.
- Describe the function of control signals like $\text{IO}\overline{\text{R}}$, $\text{IO}\overline{\text{W}}$, $\text{IO}/\overline{\text{M}}$, and how they are used in I/O operations.
- Design address decoding logic to enable specific I/O devices based on port addresses.
- Interface LEDs and seven-segment displays with the 8085 microprocessor using I/O ports.
- Use programmable peripheral devices like 8255 to manage multiple I/O devices efficiently.



PROGRAMMING METHODS WITH INSTRUCTIONS





SUB LESSON 9.1

FLAG REGISTERS OF 8085

OBJECTIVES OF CHAPTER 9 (PROGRAMMING METHODS WITH INSTRUCTIONS)

- To explain the Flag Register of microprocessors.
- To provide an understanding of looping and Counting.
- To describe microprocessor Rotate and Compare Instruction.

FLAG REGISTERS OF 8085

The **flag register** is an **8-bit register** containing **five status flags** that indicate the outcomes of an operation. These flags help in decision-making during program execution.

Flags and Their Functions:

- **Zero Flag (Z):** Set to **1** if the result of an operation is **zero**, otherwise **0**.
- **Carry Flag (CY):** Set to **1** if an arithmetic operation generates a **carry or borrow**, otherwise **0**.
- **Sign Flag (S):** Set to **1** if the result is **negative** (MSB = 1), otherwise **0**.
- **Parity Flag (P):** Set to **1** if the result contains an **even number of 1s**, otherwise **0**.
- **Auxiliary Carry Flag (AC):** Set to **1** if a **carry is generated from bit 3 to bit 4** during an arithmetic operation.

The microprocessor uses these flags to **evaluate conditions** and control the flow of execution through **conditional jump, call, and return** instructions.

FLAGS IN 8085 MICROPROCESSOR



D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀
S	Z		AC		P		CY

- The **flags** are stored in an **8-bit register**, allowing the **programmer** to access and examine their status using specific instructions.
- These **flags play a crucial role in decision-making** within the microprocessor.
- Their status (**set or reset**) can be checked using **software instructions** to control program flow.
- For example, the **JC (Jump on Carry) instruction** alters the execution sequence if the **Carry (CY) flag** is set.

ZERO (Z) FLAG

- This flag **indicates whether the result of an arithmetic or logical operation is zero**.
- If the operation results in **zero**, the flag is **set (1)**; otherwise, it is **reset (0)**.

CARRY (CY) FLAG

- The **Carry Flag** is used to detect **carry (in addition) or borrow (in subtraction)** during arithmetic operations.
- If a **carry or borrow** is generated, the flag is **set (1)**; otherwise, it remains **reset (0)**.

AUXILIARY CARRY (AC) FLAG

- This flag is used to indicate **carry propagation** between **bit 3 (D3) and bit 4 (D4)** in an arithmetic operation.
- It is primarily used for **BCD (Binary-Coded Decimal) arithmetic**.



AUXILIARY CARRY (AC) FLAG

- The **Auxiliary Carry Flag** is set when a **carry is generated from bit D3 to bit D4** during an arithmetic operation.
- This carry propagation is essential in **BCD (Binary-Coded Decimal) arithmetic** and certain special operations.

SIGN (S) FLAG

- The **Sign Flag** determines whether the result of an arithmetic operation is **positive or negative**.
- If the result is **positive**, the flag is **reset (0)**, and if the result is **negative**, the flag is **set (1)**.
- This flag is essentially a **replica of the most significant bit (D7)** of the result, indicating the **sign in two's complement representation**.

PARITY (P) FLAG

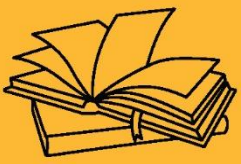
- The **Parity Flag** represents the **number of 1s in the binary representation** of a result.
- If the number of **1s is even**, the flag is **set (1)**, indicating **even parity**.
- If the number of **1s is odd**, the flag is **reset (0)**, indicating **odd parity**.



- This flag helps in **error detection and checking** during data transmission and computation.

KEY TAKEAWAYS

- Contains 5 flags: Zero, Sign, Carry, Parity, Auxiliary Carry.
- Reflects result of ALU operations.
- Flags influence program decisions via conditional jumps.



PROGRAMMING METHODS WITH INSTRUCTIONS





SUB LESSON 9.2

LOOPING, COUNTING AND INDEXING

LOOPING, COUNTING AND INDEXING

The 8085 microprocessor is a versatile device used in numerous digital systems. One of its most powerful capabilities is handling repetitive tasks, data management, and efficient memory access, all of which are made possible through looping, counting, and indexing techniques. This section explores these programming techniques and explains their usage with examples and benefits.

LOOPING IN 8085

Looping is a fundamental programming concept where a set of instructions is repeated multiple times until a specific condition is met. In the context of the 8085 microprocessor, looping is extensively used to perform tasks like generating delays, iterating over data structures, or performing repetitive arithmetic or logical operations.

In 8085, looping is implemented using jump instructions. These instructions allow the program control to move back to a previous instruction, forming a loop. A typical loop structure includes a counter, which keeps track of the number of times the loop has been executed, and a condition, which decides whether the loop should continue or terminate.

For example, a simple loop can use the JNZ (Jump if Not Zero) instruction to repeat a block of code as long as a counter register is not zero.

b. Example: Delay Loop

A common application of looping is generating time delays. Consider the following code:

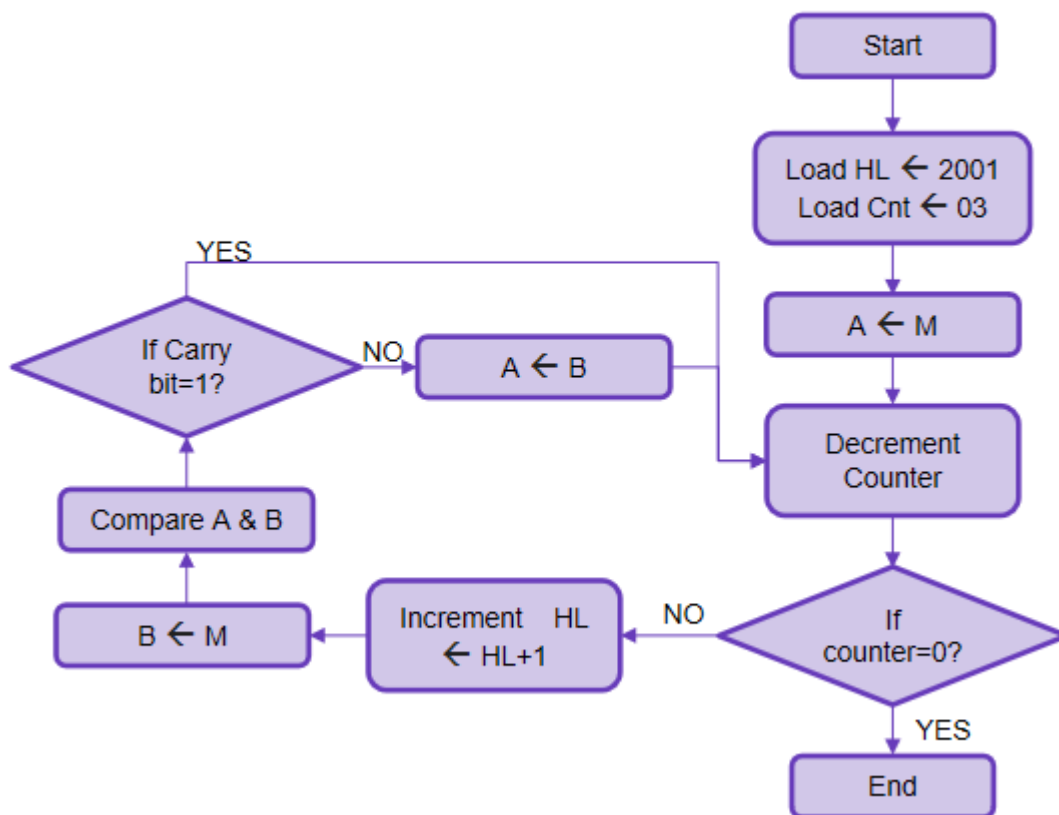
```
MVI B, 05H  
LOOP: DCR B  
      JNZ LOOP
```




This loop loads the register B with a value (in this case, 05H), then continuously decrements B until it reaches zero. The JNZ LOOP instruction keeps the control in the loop until B becomes zero, effectively creating a small time delay.

This technique is widely used to generate time delays in embedded systems when timers are not used or unavailable.

Write an 8085 assembly language program to find the minimum from the block of N 8-bit numbers





COUNTING IN LOOPS

Counting is an integral part of loop execution in the 8085 microprocessor. The loops generally use a register to serve as a counter, typically one of the general-purpose registers such as B, C, or D. The counting mechanism is used to determine how many times the loop should execute.

A. USING REGISTERS AS COUNTERS

To perform counting in loops, a value is first loaded into a register using instructions like MVI. This register is then decremented with the DCR instruction after each loop iteration. The JNZ instruction is used to test whether the count has reached zero.

B. STRUCTURE OF A COUNTING LOOP

Here is a breakdown of the typical structure of a counting loop in 8085:

Load the counter:

```
MVI C, 0AH ; Load 10 into register C
```

Loop start label:

```
LOOP: ...
```

Perform loop operations.

Decrement counter:

```
DCR C
```

Jump if not zero:

```
JNZ LOOP
```

This structure repeats the loop 10 times before exiting, making it ideal for applications where tasks need to be repeated a specific number of times.

INDEXING IN MEMORY

Indexing is a technique used to access sequential elements stored in memory, such as arrays or tables. It allows a program to manipulate or retrieve data from memory in a structured and efficient way.

A. INDEXING MECHANISM



In 8085, indexing is typically done using the HL register pair, which serves as a pointer to a memory location. The H and L registers together hold the 16-bit address of the memory location to be accessed. Instructions like MOV A, M are used to transfer the content of memory (pointed by HL) into the accumulator.

To move through memory locations sequentially, the INX H instruction is used. This increments the HL register pair so the pointer moves to the next memory location.

B. INDEXING EXAMPLE

Suppose we have an array of 8-bit numbers stored in memory starting from location 2000H. To process each element, the HL register can be set to 2000H and incremented after each operation:

Copy

Edit

LXI H, 2000H ; Load HL with starting address

MOV A, M ; Move first value to accumulator

...

INX H ; Point to next memory location

MOV A, M ; Move next value to accumulator

...

This sequence helps in scanning and processing an array efficiently using indexing.

COMBINING LOOPING, COUNTING, AND INDEXING

Often, these three techniques are used together for powerful results. A common use case is searching for a particular value in an array or computing the maximum or minimum value from a set of numbers.

Example: Find Minimum from N Numbers

The following steps illustrate how looping, counting, and indexing can be combined to find the minimum number from a block of N numbers stored in memory:

Initialize the HL register to the base address of the array.

Load the first number into a register (e.g., accumulator).

Set up a loop with a counter to process all N numbers.



In each iteration:

Compare the current number with the current minimum.

Update the minimum if needed.

Increment the HL pointer to move to the next number.

Decrement the loop counter and continue until all numbers are processed.

This method uses:

Looping to repeat the comparison for each element,

Counting to limit the number of iterations, and

Indexing to traverse through the memory block.

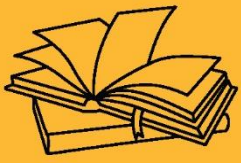
ADVANTAGES OF LOOPING, COUNTING, AND INDEXING

- These programming methods offer several advantages in microprocessor programming:
- Reduces Code Redundancy: Using loops eliminates the need for writing repeated code manually. This simplifies the code structure and enhances readability.
- Efficient Memory Access: Indexing enables efficient navigation and manipulation of data stored in memory, especially useful for arrays and data tables.
- Improves Program Control: Counting allows precise control over how many times a task is performed, enabling better management of processor time and resources.
- Essential in Embedded Applications: These techniques are fundamental in embedded systems for tasks such as delays, I/O operations, data handling, and decision-making logic.
- Supports Modular Programming: Looping and indexing promote structured and modular programming, which is easier to debug, maintain, and optimize.



KEY TAKEAWAYS

- The 8085 microprocessor supports structured programming techniques like looping, counting, and indexing, making it suitable for a wide range of applications.
- By mastering these concepts, one can design efficient assembly language programs that are scalable and easy to maintain.
- These techniques are not only essential for basic tasks like delays or arithmetic but also form the backbone of more complex operations such as searching, sorting, and data processing in embedded systems.



PROGRAMMING METHODS WITH INSTRUCTIONS





SUB LESSON 9.3

LOGIC OPERATIONS ROTATE AND COMPARE

LOGIC OPERATIONS ROTATE AND COMPARE

In assembly language programming for microprocessors like the 8085, logical operations are critical for controlling data flow, manipulating bit-level information, and implementing decision-making logic. Two major logical operations in the 8085 instruction set are rotate operations and compare operations. These functions are not just theoretical—they play an essential role in areas such as encryption, decoding, decision control structures, and sorting algorithms.

LOGICAL OPERATIONS OVERVIEW

Logical operations allow a processor to manipulate individual bits of data rather than treating data as whole units. These operations affect specific status flags in the flag register—such as Carry (CY), Zero (Z), Sign (S), Parity (P), and Auxiliary Carry (AC)—to indicate the result of the operation. Logical instructions do not modify the content of operands in compare operations but may modify the accumulator in rotate operations.

Among logical operations in 8085, the rotate instructions manipulate data at the bit level, rotating bits either to the left or right. On the other hand, compare instructions allow conditional branching based on the comparison of values.

ROTATE INSTRUCTIONS

Rotate instructions in 8085 are used to cyclically shift the bits of the accumulator either left or right. These operations may or may not involve the carry flag, depending on the specific instruction used. The accumulator (register A) is the only operand involved in these operations.

There are four types of rotate instructions:

- RLC (Rotate Left)
- RRC (Rotate Right)
- RAL (Rotate Left through Carry)



- RAR (Rotate Right through Carry)

Each of these instructions shifts the bits of the accumulator and sometimes incorporates the carry flag as part of the operation.

RLC – ROTATE ACCUMULATOR LEFT

The RLC instruction rotates all the bits of the accumulator to the left by one position. The most significant bit (D7) is moved to the least significant bit (D0) position and also copied to the Carry flag (CY).

Bit movement: $D7 \rightarrow D6 \rightarrow \dots \rightarrow D0 \rightarrow CY$, and $D7 \rightarrow D0$

The status flags S, Z, P, and AC remain unaffected.

Usage Example:

MVI A, 85H ; A = 10000101

RLC ; A becomes 00001011, CY = 1

RRC – ROTATE ACCUMULATOR RIGHT

The RRC instruction rotates the bits of the accumulator to the right. The least significant bit (D0) moves to the most significant position (D7) and is also copied to the Carry flag.

Bit movement: $D0 \rightarrow D1 \rightarrow \dots \rightarrow D7 \rightarrow CY$, and $D0 \rightarrow D7$

Other flags (S, Z, P, AC) remain unchanged.

Usage Example:

MVI A, 85H ; A = 10000101

RRC ; A becomes 11000010, CY = 1

RAL – ROTATE ACCUMULATOR LEFT THROUGH CARRY

This operation rotates all bits of the accumulator to the left by one bit, using the Carry flag as part of the operation. The D7 bit is moved to the CY flag, and the previous value of the Carry flag is inserted into D0.

Bit movement: $D7 \rightarrow CY$, $CY (old) \rightarrow D0$

Flags S, Z, P, and AC remain unaffected.

Usage Example:



```
MVI A, 85H ; A = 10000101
```

```
STC      ; Set CY = 1
```

```
RAL      ; A becomes 00001011, CY = 1
```

RAR – ROTATE ACCUMULATOR RIGHT THROUGH CARRY

This is the opposite of RAL. All bits of the accumulator are rotated right through the Carry flag.

The bit D0 is transferred to CY, and the previous CY value is moved into D7.

Bit movement: D0 → CY, CY (old) → D7

Flags other than CY remain unchanged.

Usage Example:

```
MVI A, 85H ; A = 10000101
```

```
STC      ; CY = 1
```

```
RAR      ; A becomes 11000010, CY = 1
```

These rotate instructions are particularly useful in bitwise algorithms, such as shifting data left or right during multiplication/division by powers of two or rotating data in encoding schemes.

COMPARE INSTRUCTIONS

The compare instructions in 8085 are used to check the relationship between the accumulator and another operand (either a register, memory location, or immediate data). These instructions do not alter the contents of any registers but do modify the flag register to indicate whether the accumulator is greater than, equal to, or less than the operand.

There are two main compare instructions:

- CMP (Compare Register or Memory with Accumulator)
- CPI (Compare Immediate Data with Accumulator)

CMP INSTRUCTION

The CMP instruction compares the contents of the accumulator with another register or a memory location pointed to by the HL register pair. Internally, the comparison is carried out by subtracting the operand from the accumulator (A - Operand), but the result is not stored—only the flags are affected.



If $A < \text{Operand}$: $CY = 1, Z = 0$

If $A = \text{Operand}$: $Z = 1, CY = 0$

If $A > \text{Operand}$: $CY = 0, Z = 0$

Example:

`MVI A, 32H`

`MVI B, 45H`

`CMP B ; A = 32H < B = 45H → CY = 1, Z = 0`

This instruction is particularly useful in conditional branching using instructions like JC (Jump if Carry), JNC (Jump if No Carry), JZ (Jump if Zero), and JNZ (Jump if Not Zero).

CPI INSTRUCTION

The CPI instruction compares an 8-bit immediate value with the contents of the accumulator. Similar to CMP, the values are not modified, and only the flags reflect the outcomes of the comparison.

Syntax: `CPI <8-bit value>`

The comparison logic is the same as in CMP.

Example:

`MVI A, 32H`

`CPI 32H ; A == 32H → Z = 1, CY = 0`

APPLICATIONS OF ROTATE AND COMPARE INSTRUCTIONS

Data Encryption and Decryption: Rotate instructions help in creating basic encryption schemes where bit positions are changed to scramble data.

Bit Manipulation Algorithms: Rotate instructions are used to implement bitwise shifts and rotations, useful in checksum calculations or encoding.

Sorting Algorithms: Compare instructions enable comparison-based sorting, such as bubble sort, selection sort, etc.

Search Routines: Compare operations are essential when scanning a memory block or data array to find matching values.



Decision Making in Programs: Using the flags set by CMP/CPI, conditional branches determine the logical flow of the program, making these operations crucial for implementing if-else structures.

KEY TAKEAWAYS

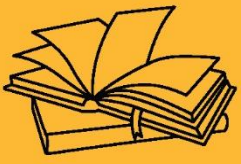
Rotate instructions (RLC, RRC, etc.) shift accumulator bits.

CMP compares values and updates flags without changing content.

Useful in decision-making logic in programs.

OUTCOMES OF CHAPTER 9 (PROGRAMMING METHODS WITH INSTRUCTIONS)

- Understand the concept of looping in 8085 microprocessor and how it enables repeated execution of instruction blocks using jump instructions like JNZ.
- Explain the role of counters in loops, using registers and decrement instructions (DCR) to control the number of iterations.
- Describe the functionality of rotate instructions (RLC, RRC, RAL, RAR) and how they shift bits within the accumulator with or without the carry flag.
- Analyze the effect of rotate instructions on flags, especially the Carry (CY) flag, while observing that other flags remain unaffected.
- Explain the working of compare instructions (CMP, CPI) to evaluate relationships between accumulator and other data, influencing program control flow.



ADDITIONAL PROGRAMMING TECHNIQUES





SUB LESSON 10.1

ADDITION AND SUBTRACTION OF TWO 8-BIT NUMBERS

OBJECTIVES OF CHAPTER 10 (SIMPLE PROGRAMS USING 8085 INSTRUCTION SET, DELAY)

1. To explain the Addition and subtraction programs in microprocessor.
2. To explain multiplication and division programming.
3. To describe the program of finding largest and smallest number.
4. To explore programming using delay loop.

ADDITION AND SUBTRACTION OF TWO 8-BIT NUMBERS USING THE 8085 MICROPROCESSOR

The 8085 microprocessor, an 8-bit general-purpose microprocessor, allows various arithmetic and logical operations. In this section, we focus on two fundamental arithmetic operations: **addition** and **subtraction** of two 8-bit numbers. These operations are the backbone of any computational task and understanding them is essential for mastering microprocessor programming.

ADDITION OF TWO 8-BIT NUMBERS

To add two 8-bit numbers using the 8085 microprocessor, the numbers are typically stored in memory. The first step involves loading one of the numbers into the accumulator, the central register used for arithmetic operations. The other number is either directly added from another register or from a memory location.

Let us consider a simple scenario where two numbers are added, and the result is stored in memory

STEPS INVOLVED:



- Load the first 8-bit number into the accumulator (register A).
- Add the second number to the accumulator.
- Store the result from the accumulator into a memory location.

EXAMPLE PROGRAM:

`MVI A, 25H` ; Move the hexadecimal value 25H into register A

`ADD B` ; Add the contents of register B to A

`STA 2050H` ; Store the result from accumulator into memory location 2050H

`HLT` ; Terminate the program

In this example, the `MVI` instruction initializes the accumulator with the value 25H. The `ADD` instruction adds the content of register B to A. Finally, the result is saved at address 2050H using the `STA` (Store Accumulator) instruction.

EXPLANATION OF INSTRUCTIONS:

- `MVI A, data`: Loads immediate 8-bit data into register A.
- `ADD B`: Adds the contents of register B to accumulator A.
- `STA addr`: Stores the result of A into the specified memory address.
- `HLT`: Ends program execution.

In practical scenarios, the numbers might be stored in memory, and addressing modes will be used accordingly.

ADDITION USING DAA INSTRUCTION

If the numbers are in **BCD (Binary Coded Decimal)** format, the `DAA` (Decimal Adjust Accumulator) instruction must be used after the `ADD` operation to ensure the result is a valid BCD number.



DAA corrects the result of BCD addition by checking and adjusting the lower and upper nibble of the accumulator if required. This is crucial when handling arithmetic operations in applications that involve decimal digit manipulation.

SUBTRACTION OF TWO 8-BIT NUMBERS

Subtraction is another essential operation supported by the 8085 instruction set. The process involves subtracting one number from another using the `SUB` instruction. For multi-byte subtraction, the `SBB` (Subtract with Borrow) instruction is used to account for borrows from the previous operation.

Let's examine a program that subtracts two 8-bit numbers stored in memory.

SUBTRACTION PROCESS STEPS:

1. Load the first 8-bit number from memory address 9000H into register pair HL.
2. Exchange HL with DE register pair so that DE holds the first number.
3. Load the second number from memory address 9002H into HL.
4. Move the content of E (from DE) into accumulator A.
5. Subtract the content of L from A using `SUB` instruction.
6. Store the result back into register L.
7. Move the content of D (from DE) into A.
8. Use `SBB H` to subtract the contents of H with borrow.
9. Store this result into register H.
10. Save the final 16-bit result from HL to memory address 9004H.
11. Terminate the program.

`LHLD 9000H` ; Load first 8-bit number into HL

`XCHG` ; Exchange HL with DE to preserve first number

`LHLD 9002H` ; Load second number into HL



MOV A, E ; Move lower byte of first number to accumulator

SUB L ; Subtract lower byte of second number

MOV L, A ; Store result into register L

MOV A, D ; Move higher byte of first number

SBB H ; Subtract higher byte with borrow

MOV H, A ; Store result in H

SHLD 9004H ; Store final 16-bit result at address 9004H

HLT ; Terminate the program

UNDERSTANDING FLAGS AND RESULT ANALYSIS

During arithmetic operations, various flags are affected. In subtraction, if the first number is smaller than the second, the **carry flag** (also referred to as **borrow** in subtraction) is set. Other flags like **Zero**, **Sign**, **Parity**, and **Auxiliary Carry** also change depending on the result.

For example:

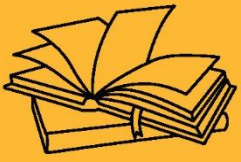
- **Zero Flag (Z)**: Set if the result is zero.
- **Sign Flag (S)**: Indicates the sign of the result (for signed operations).
- **Carry Flag (CY)**: Set during borrow in subtraction.

This status helps in decision-making instructions and further computations.



KEY TAKEAWAYS

- The addition and subtraction of two 8-bit numbers using the 8085 microprocessor require a good understanding of data movement and arithmetic instructions.
- Mastery of these basic programs lays the groundwork for handling more complex operations involving larger data sizes or decimal formats.
- Through practical implementation and analysis of flags, one gains deeper insight into microprocessor programming and computational logic.



ADDITIONAL PROGRAMMING TECHNIQUES





SUB LESSON 10.2

MULTIPLICATION AND DIVISION OF TWO NUMBERS USING REPEATED ADDITION/ SUBTRACTION

MULTIPLICATION AND DIVISION USING REPEATED ADDITION AND SUBTRACTION IN 8085 MICROPROCESSOR

The 8085 microprocessor is an 8-bit microprocessor developed by Intel, known for its simplicity and ease of use in learning assembly language programming. However, one notable limitation of the 8085 is the absence of direct multiplication (MUL) and division (DIV) instructions. As a result, arithmetic operations like multiplication and division must be implemented using basic instructions such as addition, subtraction, increment, and decrement. This document explains how multiplication and division of two numbers can be implemented using repeated addition and subtraction, respectively, in the 8085 assembly language.

MULTIPLICATION USING REPEATED ADDITION

Since the 8085 does not have a dedicated instruction for multiplication, it can be simulated through repeated addition. For instance, multiplying 5 by 3 is equivalent to adding 5 three times or adding 3 five times. This concept is implemented through a loop that performs the addition operation multiple times.

CONCEPT

Multiplication is essentially a process of adding a number (the multiplicand) repeatedly, based on the number of times specified by another number (the multiplier). Mathematically:

8085 PROGRAM FOR MULTIPLICATION

The following example illustrates the multiplication of two numbers using repeated addition.



EXAMPLE:

To multiply 5 and 3, where 5 is the multiplier and 3 is the multiplicand:

MVI C, 05H ; Load multiplier (5) into register C

MVI D, 03H ; Load multiplicand (3) into register D

MVI A, 00H ; Clear accumulator (A) to hold the result

LOOP: ADD D ; Add the value in D to A

DCR C ; Decrement the multiplier

JNZ LOOP ; If C is not zero, repeat the loop

STA 2052H ; Store the result at memory location 2052H

HLT ; Halt the program

EXPLANATION OF THE CODE

- **MVI Instructions** load constants into the registers.
- The **ADD D** instruction adds the contents of register D (multiplicand) to the accumulator A.
- **DCR C** decrements the count in register C (multiplier) by one after each addition.
- **JNZ LOOP** ensures that the addition is repeated until the multiplier becomes zero.
- Finally, **STA 2052H** stores the result from the accumulator to memory location 2052H.

FLOWCHART DESCRIPTION

1. Initialize registers with multiplier and multiplicand.
2. Clear the accumulator.



3. Enter loop:
 - Add multiplicand to the accumulator.
 - Decrement multiplier.
 - Continue if multiplier is not zero.
4. Store result and halt.

This loop effectively adds the multiplicand to the accumulator the number of times specified by the multiplier, resulting in the desired product.

DIVISION USING REPEATED SUBTRACTION

Just like multiplication, the 8085 does not have a direct instruction for division. However, the division operation can be carried out using repeated subtraction, which is a basic and logical approach. The idea is to subtract the divisor from the dividend repeatedly until the dividend becomes less than the divisor. The number of times the subtraction is performed gives the quotient, and the final value left in the dividend is the remainder.

CONCEPT

Division is the process of determining how many times one number (divisor) can be subtracted from another (dividend) before the result becomes zero or negative.

8085 PROGRAM FOR DIVISION

The following program divides 10 by 2 using repeated subtraction:

MVI A, 0AH ; Load dividend (10) into accumulator

MVI B, 02H ; Load divisor (2) into register B

MVI C, 00H ; Initialize quotient counter in register C

DIV_LOOP: SUB B ; Subtract B (divisor) from A (accumulator)



JZ END ; If A is zero after subtraction, end loop

INR C ; Increment quotient

JMP DIV_LOOP ; Repeat subtraction

END: STA 2053H ; Store the remainder (in A) at 2053H

HLT ; Halt the program

EXPLANATION OF THE CODE

1. The **MVI** instructions load the dividend, divisor, and a zero-initialized quotient register.
2. **SUB B** subtracts the divisor from the current value in the accumulator (initially the dividend).
3. **JZ END** checks if the result of the subtraction is zero; if so, it ends the loop.
4. **INR C** increments the quotient counter each time a subtraction is performed.
5. The loop continues until the accumulator is reduced to zero or less.
6. The **STA** instruction stores the final remainder, which is the value left in the accumulator after the last subtraction.

FLOWCHART DESCRIPTION

1. Load dividend into A and divisor into B.
2. Initialize quotient (C) to zero.
3. Begin loop:
 - Subtract divisor from dividend.
 - Check if result is zero.
 - If not, increment quotient and repeat.
4. Store remainder and halt.



The quotient is available in register C, and the remainder remains in register A at the end of the loop.

When working with the 8085 microprocessor, all operations are limited to 8-bit data, meaning the maximum value any register can hold is 255 (FFH). This limits the range of input values for multiplication and division. For results exceeding 8 bits, extended logic or a 16-bit approach must be used.

EFFICIENCY

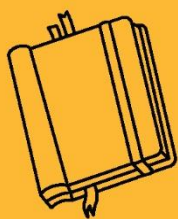
While repeated addition and subtraction are logically sound, they are not computationally efficient. For small numbers, they work reasonably well, but for larger numbers, the loop takes more time, resulting in performance overhead. Nonetheless, this approach is essential for educational purposes and understanding the fundamentals of assembly language logic.

REAL-WORLD APPLICATIONS

In embedded systems and educational microcontroller kits where instruction sets are minimal, such techniques are vital. They demonstrate how low-level computations are achieved in environments with limited hardware support.

KEY TAKEAWAYS

- These methods provide foundational insights into how complex arithmetic operations are built from simple instruction sets. Understanding and applying these techniques is essential for developing a strong grasp of low-level programming and microprocessor architecture.
- By mastering these approaches, learners gain deeper insight into logic construction, algorithm design, and processor functionality—skills that are crucial in the field of embedded systems and computer engineering.



ADDITIONAL PROGRAMMING TECHNIQUES





SUB LESSON 10.3

FINDING THE LARGEST AND SMALLEST NUMBER IN AN ARRAY

FINDING THE LARGEST AND SMALLEST NUMBER IN AN ARRAY USING 8085 ASSEMBLY LANGUAGE

Working with arrays is a fundamental concept in programming, and operations like finding the largest or smallest value are commonly performed. In high-level languages, such tasks are handled with simple loops and comparison operators. However, in assembly language, particularly in the 8085 microprocessor, such tasks require a more hands-on approach using basic instructions like comparison, jumps, and register operations. This write-up explores how to find the largest and smallest numbers in an array using 8085 assembly language, reinforcing the logic-building skills necessary for low-level programming.

UNDERSTANDING THE PROBLEM

The goal is to find the maximum and minimum values in an array of 8-bit numbers stored sequentially in memory. In both cases, the procedure involves:

1. Loading the base address of the array.
2. Loading the count of the elements in the array.
3. Comparing each number with the current maximum or minimum.
4. Replacing the current value if a larger or smaller number is found.
5. Storing the result in a specified memory location.

FINDING THE LARGEST NUMBER IN AN ARRAY

CONCEPT



The largest number in an array can be found by sequentially comparing each element to a running maximum. If a current element is greater than the running maximum, it replaces it. This process continues until all elements are compared.

FLOW OF EXECUTION

1. Initialize a pointer to the base address of the array.
2. Load the number of elements in the array into a counter.
3. Load the first element of the array into the accumulator as the initial maximum.
4. Use a loop to compare this value with each subsequent array element.
5. If a larger value is found, store it in the accumulator.
6. After all elements are compared, store the result (maximum value) in a memory location.

8085 ASSEMBLY PROGRAM FOR FINDING THE LARGEST NUMBER

LXI H, 2050H ; Load base address of array

MVI C, 05H ; Load number of elements (5)

MOV A, M ; Load first element into accumulator

INX H ; Point to the next element

DCR C ; Decrease count by 1 (first element already considered)

LOOP: CMP M ; Compare current memory content with accumulator

JC NEXT ; If A < M, jump to NEXT (keep current max)

MOV A, M ; Else, move current memory content to accumulator

NEXT: INX H ; Move to the next memory location



DCR C ; Decrement counter

JNZ LOOP ; Repeat until all elements are compared

STA 2060H ; Store the largest number at memory location 2060H

HLT ; Halt program

EXPLANATION

- **LXI H, 2050H** sets the HL pair to the start of the array.
- **MVI C, 05H** loads the number of array elements (5).
- **MOV A, M** initializes the accumulator with the first array value.
- The loop then compares the accumulator with each element.
- If the new element is larger, it replaces the current value in the accumulator.
- After completing the loop, **STA 2060H** saves the maximum value.

INSTRUCTION NOTES

- **CMP M** compares the accumulator (A) with the content of memory addressed by HL. The result is reflected in the flags.
- **JC** (Jump if Carry) transfers control if $A < M$.
- **MOV A, M** updates the accumulator when a higher number is found.

FINDING THE SMALLEST NUMBER IN AN ARRAY

CONCEPT

Finding the smallest number in an array follows a similar approach but involves keeping track of the minimum value. Each element is compared, and if a smaller number is found, it replaces the current value in the accumulator.

FLOW OF EXECUTION



1. Initialize the HL pair to the starting address of the array.
2. Load the count of elements into register C.
3. Load the first element into the accumulator.
4. Loop through the rest of the elements:
 - Compare the current element with the accumulator.
 - Replace if the current element is smaller.
5. Store the result in memory after completion.

8085 ASSEMBLY PROGRAM FOR FINDING THE SMALLEST NUMBER

LXI H, 2050H ; Load base address of array

MVI C, 05H ; Number of elements

MOV A, M ; Load first element

INX H ; Move to second element

DCR C ; One element already considered

LOOP: CMP M ; Compare memory content with accumulator

JNC NEXT ; If A <= M, jump to NEXT

MOV A, M ; Else, update accumulator with smaller value

NEXT: INX H ; Move to next array element

DCR C ; Decrement counter

JNZ LOOP ; Repeat until counter is zero

STA 2061H ; Store the smallest number at memory location 2061H



HLT ; Halt the program

EXPLANATION

- **JNC** (Jump if No Carry) checks if $A \geq M$; if so, no need to update.
- **MOV A, M** updates the minimum value if a smaller element is found.
- **STA 2061H** stores the final smallest value.

INSTRUCTION NOTES

- **JNC** is used here because we're only updating the accumulator if the current memory value is less than the accumulator.
- Using the same loop structure, the comparison is reversed for finding the smallest value.

RESULT STORAGE AND MEMORY LAYOUT

The programs store:

- **Maximum value at 2060H**
- **Minimum value at 2061H**

These memory locations can be accessed later for further processing or display.

USE CASES IN REAL-TIME SYSTEMS

Though modern systems use high-level programming, these techniques are still relevant:

- In embedded systems with low-level hardware programming.
- In scenarios with memory constraints where high-level abstractions are not feasible.
- In academic settings for understanding low-level data handling and algorithm development.

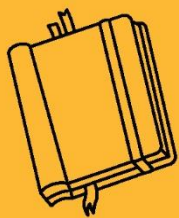


COMPARISON BETWEEN FINDING MAXIMUM AND MINIMUM

Task	Jump Condition Used	Logic Flow
Find Maximum	JC (Jump if $A < M$)	Replace A if new value is greater
Find Minimum	JNC (Jump if $A \geq M$)	Replace A if new value is smaller
Result Storage	STA 2060H or 2061H	Store result after loop finishes

KEY TAKEAWAYS

- Both tasks require comparing elements in a loop.
- The accumulator serves as a temporary register for storing the result during iteration.
- Memory pointers and counters help navigate the array efficiently.
- These programs help learners understand logic development and control flow in low-level programming.



ADDITIONAL PROGRAMMING TECHNIQUES



SUB LESSON 10.4

DELAY GENERATION USING LOOPING TECHNIQUES

DELAY GENERATION USING LOOPING TECHNIQUES IN THE 8085 MICROPROCESSOR

In microprocessor-based systems, timing and synchronization are crucial. Many real-world applications require time delays to manage the sequence of operations or to interface with slower external peripherals. Since the 8085 microprocessor does not have a built-in delay instruction, delays must be generated using software loops or hardware timers. One of the most common software-based approaches involves creating loops that execute specific instructions repeatedly to occupy time and thus produce a delay.

This document provides an in-depth explanation of delay generation using looping techniques in the 8085 microprocessor, including code examples, explanations, applications, and limitations.

PURPOSE OF DELAY IN MICROPROCESSOR SYSTEMS

Delays in microprocessor programs serve several important purposes:

1. **Synchronization:** To synchronize internal microprocessor operations with external devices.
2. **Timing Control:** To allow hardware signals to stabilize or complete before the next operation begins.
3. **User Interface:** To control response timing, such as debouncing a switch or blinking an LED.
4. **Communication:** To wait for signals or data in serial/parallel communication protocols.
5. **Sequencing Tasks:** To regulate execution time between tasks.

These delays are critical when working with electromechanical systems, communication interfaces, and any application where real-time control is necessary.



BASIC CONCEPT OF DELAY LOOPS

The basic idea behind generating a delay using software is to execute instructions that do not alter the program output but consume clock cycles. When a particular instruction or set of instructions is executed repeatedly in a loop, the microprocessor remains busy and hence delays further operations.

The 8085 instruction set includes commands such as MVI, DCR, and JNZ that can be used to build such loops. The overall delay depends on:

- The number of iterations
- The number of clock cycles taken by each instruction
- The microprocessor's clock frequency

SIMPLE DELAY LOOP EXAMPLE

Let us consider a simple example to create a basic delay using a loop:

PROGRAM

```
MVI B, FFH    ; Load register B with FFH (255 decimal)
```

```
DELAY: DCR B   ; Decrement register B
```

```
    JNZ DELAY ; Jump to DELAY if B is not zero
```

```
    HLT      ; Halt the program
```

EXPLANATION

1. **MVI B, FFH** initializes register B with the maximum 8-bit value (255).
2. The **DCR B** instruction decrements B by 1.
3. **JNZ DELAY** checks whether B has reached zero. If not, the loop continues.



4. Once B becomes zero, the loop ends and the **HLT** instruction halts the program.

This simple loop will create a short time delay depending on the clock speed of the system and the time taken by each instruction.

FLOWCHART

- Start
- Load count in register
- Decrement register
- Check if zero
 - If not, repeat
 - If yes, halt

NESTED DELAY LOOPS FOR LONGER DELAYS

A single-level loop can provide only a limited delay, especially in fast clock systems. To generate longer delays, nested loops are used. This involves placing one loop inside another. Each time the outer loop iterates, the inner loop runs completely, multiplying the total number of instructions executed.

PROGRAM EXAMPLE

```
MVI C, FFH    ; Outer loop counter
```

```
OUTER: MVI B, FFH ; Inner loop counter
```

```
INNER: DCR B
```

```
    JNZ INNER ; Loop until B = 0
```

```
    DCR C
```



```
JNZ OUTER ; Loop until C = 0
```

```
HLT
```

EXPLANATION

- The inner loop (register B) runs 255 times for each value of C.
- The outer loop (register C) also runs 255 times.
- Thus, the total number of iterations becomes $255 \times 255 = 65025$.

This significantly increases the delay time, allowing the program to introduce more noticeable pauses suitable for real-time applications such as blinking LEDs or polling inputs.

APPLICATION OF DELAY LOOPS

Delay loops are useful in many real-time and embedded systems. Some practical applications include:

1. LED BLINKING

Delays allow enough time between turning LEDs on and off to make blinking visible to the human eye.

2. SWITCH DEBOUNCING

Mechanical switches may generate noisy signals when pressed. A short delay after a press can ensure the switch stabilizes before its state is read.

3. DISPLAY CONTROL

Some displays (e.g., 7-segment or LCD) require specific delays for refresh cycles or between data commands.

4. COMMUNICATION PROTOCOLS



In serial communication, proper delays ensure that the data is sent or received at the correct baud rate.

5. SIGNAL GENERATION

Delay loops can help generate time-controlled waveforms or pulses.

LIMITATIONS OF SOFTWARE DELAY LOOPS

Although delay loops are simple and useful, they come with some drawbacks:

1. INACCURACY

The delay generated is dependent on the clock frequency of the microprocessor. Any variation in the clock speed affects the accuracy of the delay.

2. PROCESSOR OCCUPANCY

The microprocessor remains engaged in executing the delay loop and cannot perform any other task during that time. This leads to inefficient CPU usage.

3. SCALABILITY

If longer delays are required, nested or extended loops must be used, which consume more memory and instructions.

4. HARD TO CALIBRATE

Precise delay timing requires careful calculation of the number of machine cycles each instruction consumes. Any mistake can lead to incorrect timing.

5. NOT IDEAL FOR MULTITASKING



In systems requiring multitasking or real-time interrupts, software delay loops block the processor, making them unsuitable unless carefully controlled.

ALTERNATIVE: HARDWARE DELAY USING TIMERS

In real-world applications, especially in more complex systems, hardware timers are preferred over software delay loops. These can be configured to generate precise delays and free up the microprocessor to perform other tasks.

ADVANTAGES OF HARDWARE TIMERS

- **Precision:** Accurate and independent of instruction cycle timing.
- **Multitasking:** Allows the microprocessor to continue other tasks while the timer runs.
- **Interrupt Driven:** Can trigger events after the delay period without polling.

However, hardware timers come with their own disadvantages such as:

- Additional cost
- Requirement of extra ICs (Integrated Circuits)
- Complexity in interfacing

SUMMARY

Aspect	Software Delay	Hardware Timer
Accuracy	Less accurate	Highly accurate
CPU Usage	Fully occupied	Frees CPU
Complexity	Simple	Complex
Cost	No extra cost	Additional hardware required
Scalability	Limited	High

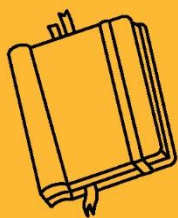


KEY TAKEAWAYS

- Delays are created using software loops.
- NOP or repeated instructions generate time delays.
- Useful for timing control in interfacing.

OUTCOMES OF CHAPTER 10 (SIMPLE PROGRAMS USING 8085 INSTRUCTION SET, DELAY)

- **Understand the use of basic 8085 instructions** to perform arithmetic operations like addition, subtraction, multiplication (using repeated addition), and division (using repeated subtraction) without dedicated MUL/DIV instructions.
- **Develop and analyze 8085 assembly programs** to identify the largest and smallest numbers in an array using comparison logic and register operations.
- **Implement delay generation techniques** using single and nested software loops, and explain their role in synchronization and timing control in microprocessor applications.
- **Evaluate the limitations of software-based delays** and understand the advantages and trade-offs of using hardware timers in real-time embedded systems.



ADDITIONAL PROGRAMMING TECHNIQUES





SUB LESSON 11.1

CONCEPT OF STACK IN 8085 (PUSH AND POP INSTRUCTIONS)

OBJECTIVES OF CHAPTER 11 (STACK & SUBROUTINES, RESTART, CALL AND RETURN INSTRUCTION, CODE CONVERSION)

1. To explain the Stack and Subroutine in microprocessor.
2. To explain Restart and call instruction in programming.
3. To describe the code conversion.

WHAT IS A STACK?

A **stack** is a special kind of memory structure used to store data in a **Last In, First Out (LIFO)** manner. This means that the last data item pushed (added) onto the stack will be the first one to be popped (removed). Think of it as a stack of plates — you add new plates on top and remove them from the top.

In programming and microprocessor terms, a stack is mainly used to:

- Store return addresses when subroutines are called.
- Save register contents temporarily.
- Assist in interrupt handling and context switching.

STACK IN THE 8085 MICROPROCESSOR

The 8085 microprocessor supports stack operations and provides instructions to use the stack effectively. Key points about the stack in 8085:

- The stack grows **downward in memory** — from higher memory addresses to lower addresses.



- It is accessed via a special **16-bit register called the Stack Pointer (SP)**.
- The **SP** always points to the **top of the stack (TOS)**, which is the memory address of the last item stored.
- The stack is not a separate hardware component but a **reserved area in main memory**.

STACK POINTER (SP) IN 8085

The **Stack Pointer (SP)** is a **16-bit register** that holds the memory address of the top of the stack. It is initialized by the programmer to point to a suitable memory location in RAM.

- When data is pushed onto the stack, the SP is **decremented** first and then the data is stored.
- When data is popped from the stack, the data is read from the address pointed by SP, and then the SP is **incremented**.

Thus, the SP acts as a pointer and moves up and down the memory as data is pushed or popped.

PUSH INSTRUCTION

DEFINITION

The **PUSH** instruction stores the contents of a register pair (e.g., BC, DE, HL, or PSW) onto the stack.

OPERATION

1. The stack pointer (SP) is decremented by 1.
2. The higher-order byte of the register pair is copied to the memory location pointed to by SP.
3. The SP is decremented again by 1.
4. The lower-order byte of the register pair is copied to the new location pointed by SP.



This means both bytes of the register pair are saved on the stack, with the lower byte being pushed **after** the higher byte.

SYNTAX

PUSH rp

Where rp can be:

- B (BC register pair)
- D (DE register pair)
- H (HL register pair)
- PSW (Accumulator and Flags)

EXAMPLE

PUSH B

EXPLANATION:

Assume:

- BC = 1234H
- SP = 3000H

After execution:

- SP becomes 2FFEh.
- Memory location 2FFFh contains 12H (high byte).
- Memory location 2FFEh contains 34H (low byte).

So, the stack now holds the BC register pair values.



POP INSTRUCTION

DEFINITION

The **POP** instruction retrieves data from the stack and loads it into the specified register pair.

OPERATION

1. The contents of the memory location pointed by SP are moved to the **low-order register**.
2. The SP is incremented by 1.
3. The next memory content is moved to the **high-order register**.
4. The SP is incremented again.

Thus, the register pair gets restored from the stack, and the stack pointer returns to its previous position.

SYNTAX

POP rp

Where rp is one of:

- B (BC)
- D (DE)
- H (HL)
- PSW (Accumulator and Flags)

EXAMPLE

POP B

EXPLANATION:

Assume:



- $SP = 2FFE_{H}$
- Memory at $2FFE_{H} = 34_{H}$
- Memory at $2FFF_{H} = 12_{H}$

After execution:

- $C = 34_{H}$ (low byte)
- $B = 12_{H}$ (high byte)
- SP becomes 3000_{H}

Thus, the contents of the BC pair are restored as 1234_{H} .

STACK OPERATIONS VISUALIZATION

PUSH OPERATION STEPS:

Before PUSH:

$SP \rightarrow 3000_{H}$

After PUSH B ($BC = 1234_{H}$):

Memory[$2FFF_{H}$] = 12_{H} ; High byte (B)

Memory[$2FFE_{H}$] = 34_{H} ; Low byte (C)

$SP = 2FFE_{H}$

POP OPERATION STEPS:

Before POP:

$SP \rightarrow 2FFE_{H}$

Memory[$2FFE_{H}$] = 34_{H} ; Low byte



Memory[2FFFH] = 12H ; High byte

After POP B:

BC = 1234H

SP = 3000H

These operations show how the stack grows downward (decrementing SP) and how values are restored by incrementing SP.

USES OF STACK IN 8085

The stack is used extensively in many situations, including:

SUBROUTINE HANDLING

When a subroutine is called using the CALL instruction, the return address is pushed onto the stack. When the subroutine ends, the RET instruction pops this address and resumes execution from that point.

INTERRUPT SERVICE ROUTINES

Interrupts often cause the program to jump to a different memory location. The current program counter is stored on the stack so that the program can resume after servicing the interrupt.

TEMPORARY DATA STORAGE

Register contents can be temporarily saved onto the stack using PUSH before entering a block of code and then restored using POP afterward. This is especially useful in nested subroutines.

CONTEXT SWITCHING



In multitasking systems, the stack helps save the state (context) of a task so another task can run.

Later, the saved state is restored from the stack.

ADVANTAGES OF USING STACK IN 8085

- **Efficient Register Management:** Temporary data can be stored without disturbing general-purpose registers permanently.
- **Nested Subroutine Support:** Enables the use of multiple nested procedures.
- **Program Flow Control:** Supports complex control structures through return address storage.
- **Memory Optimization:** Uses main memory for temporary data without allocating fixed memory locations.

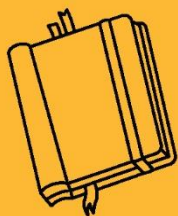
LIMITATIONS AND CONSIDERATIONS

- The programmer must ensure that the **stack pointer is initialized** properly before using PUSH/POP.
- **Stack overflow** may occur if the stack grows into memory used by the program or other data.
- Similarly, **stack underflow** can happen if a POP is executed when the stack is empty or incorrectly initialized.



KEY TAKEAWAYS

- The stack is a powerful and essential feature of the 8085 microprocessor. It supports a wide range of operations related to temporary data storage, subroutine and interrupt management, and execution control.
- Through the **PUSH** and **POP** instructions, data can be safely stored and retrieved from the stack in a structured LIFO manner.
- Understanding stack operations allows programmers to write efficient assembly programs, particularly when dealing with modular code and multitasking.



ADDITIONAL PROGRAMMING TECHNIQUES





SUB LESSON 11.2

ROLE OF STACK POINTER AND STACK MEMORY MANAGEMENT

ROLE OF STACK POINTER AND STACK MEMORY MANAGEMENT IN 8085 MICROPROCESSOR

In microprocessor-based systems, data storage, function calling, and return mechanisms require a structured and temporary memory area known as the **stack**. The 8085 microprocessor uses the stack as a temporary memory structure to hold data such as return addresses, register contents, and intermediate values. The **Stack Pointer (SP)** plays a key role in managing this stack by pointing to the current top of the stack. Stack memory management ensures orderly access to this area, allowing efficient use of system resources and supporting modular and interrupt-driven programming.

This document provides a clear understanding of the stack, the function of the stack pointer, and the importance of stack memory management in the context of the 8085 microprocessor.

WHAT IS A STACK?

A **stack** is a region in the microprocessor's read/write memory (RAM) used for **temporary storage** of binary data during the execution of a program. The stack works based on the **Last-In, First-Out (LIFO)** principle. This means the last data item stored in the stack is the first one to be retrieved.

Stacks are typically used for:

- Storing return addresses during subroutine and interrupt handling.
- Temporarily saving register contents.
- Managing local variables in advanced systems.



In the 8085 microprocessor, the stack is not a dedicated hardware component but is implemented using a portion of RAM.

STACK POINTER (SP): DEFINITION AND ROLE

The **Stack Pointer (SP)** is a **16-bit special-purpose register** in the 8085 microprocessor. It always points to the **top of the stack**, i.e., the memory address where the most recent data is stored.

KEY CHARACTERISTICS:

- The SP is initialized by the programmer using the instruction `LXI SP, address`, where address is the starting location of the stack.
- The stack in 8085 **grows downward**. That is, new data is stored at progressively **lower** memory addresses.
- The SP is automatically **decremented** when data is pushed onto the stack and **incremented** when data is popped from the stack.

For example, if we initialize the stack pointer as `LXI SP, 2099H`, the first data pushed will be stored at **2098H**, then 2097H, and so on.

STACK MEMORY MANAGEMENT

Proper stack memory management is essential to avoid **memory conflicts**, **data loss**, or **program crashes**. The programmer must decide the memory region to be reserved for the stack and ensure it doesn't overlap with the main program or data.

SETTING UP THE STACK

1. Choose a safe area in RAM, preferably towards the higher end of the available memory.
2. Initialize the stack pointer using the `LXI SP` instruction.

`LXI SP, 2099H` ; Example: Load SP with memory address 2099H



3. Ensure that PUSH and POP operations only occur within the designated stack area.

If stack operations go beyond allocated space (e.g., due to too many nested calls), a **stack overflow** occurs. Similarly, accessing the stack without proper initialization leads to **stack underflow** or invalid data reads.

STACK OPERATIONS

PUSH OPERATION

The PUSH instruction is used to store the contents of a register pair (like BC, DE, HL, or PSW) onto the stack.

SEQUENCE:

- SP is **decremented by 1**.
- The high-order byte is stored at the new SP address.
- SP is **decremented again by 1**.
- The low-order byte is stored at the new SP address.

This sequence ensures that both bytes of the register pair are safely saved.

EXAMPLE:

PUSH B ; Stores BC pair on stack

If SP = 2099H, then:

- SP becomes 2098H → store B
- SP becomes 2097H → store C

Now SP = 2097H, pointing to the top of the stack.



POP OPERATION

The POP instruction is used to retrieve data from the stack and restore it into the register pair.

SEQUENCE:

- The low-order byte is read from the address pointed to by SP.
- SP is **incremented by 1**.
- The high-order byte is read.
- SP is **incremented by 1** again.

The original register values are thus restored, and the stack pointer returns to its previous state.

EXAMPLE:

POP B ; Restores BC pair from stack

If SP = 2097H (after PUSH), then:

- Read C from 2097H
- SP → 2098H
- Read B from 2098H
- SP → 2099H

PRACTICAL USE OF STACK AND SP

The stack is essential for many important functions in a microprocessor program:

SUBROUTINE CALLS

When a subroutine is called using the CALL instruction, the return address is pushed onto the stack. When the subroutine completes, the RET instruction retrieves the return address from the stack and resumes program execution.



INTERRUPT HANDLING

Interrupts temporarily pause normal program execution. The microprocessor saves the current state (especially the program counter) on the stack before handling the interrupt. Once the interrupt is serviced, the state is restored using RET.

TEMPORARY DATA STORAGE

Registers can be saved using PUSH before modifying them inside a subroutine and restored using POP after returning.

IMPORTANCE OF PROPER STACK MANAGEMENT

Failing to manage the stack carefully can lead to:

- **Stack Overflow:** Occurs when the stack overwrites other parts of the memory.
- **Stack Underflow:** Happens if a POP is issued without corresponding PUSH or without initialization.
- **Data Corruption:** If the SP is not correctly initialized or goes out of range.

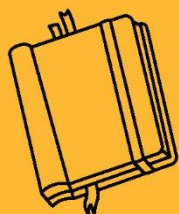
TIPS FOR SAFE STACK USE:

- Always initialize the SP at the beginning of the program.
- Keep track of the maximum expected depth of the stack.
- Avoid excessive nesting of subroutines unless stack size is assured.
- Use stack operations only when needed to avoid unnecessary memory access.



KEY TAKEAWAYS

- The stack and stack pointer are fundamental components in the 8085 microprocessor, enabling efficient memory management, modular programming, and reliable subroutine and interrupt handling.
- The Stack Pointer ensures that stack operations occur at the correct memory locations, while stack memory management ensures that the system remains stable and free from memory conflicts.
- A solid understanding of these concepts is vital for anyone working with assembly language or embedded systems based on microprocessors like the 8085.



ADDITIONAL PROGRAMMING TECHNIQUES



SUB LESSON 11.3

SUBROUTINES AND THEIR IMPLEMENTATION USING CALL AND RET INSTRUCTIONS

INTRODUCTION

A key aspect of structured and modular programming is the use of **subroutines**. Subroutines are blocks of code designed to perform specific tasks and can be invoked from different parts of a program. In the **8085 microprocessor**, subroutines are implemented using the CALL and RET instructions, which allow the main program to temporarily transfer control to a different location and resume execution after the subtask is completed.

Using subroutines not only improves code organization but also helps avoid redundancy, making programs shorter, easier to read, and maintainable. This document explains what subroutines are, how they work in the 8085 microprocessor, and the mechanism of CALL and RET instructions for subroutine implementation.

WHAT IS A SUBROUTINE?

A **subroutine** is a set of instructions that performs a specific task, which may be required multiple times within a program. Instead of rewriting the same instructions every time, they can be grouped as a subroutine and **called** whenever needed. Once the subroutine completes its job, it **returns** control back to the main program.

In assembly language, this concept is particularly important because code space is limited and repeating the same instructions leads to inefficient use of memory.

KEY BENEFITS OF SUBROUTINES

- Reduces code duplication



- Promotes modular programming
- Simplifies debugging and testing
- Enhances program readability

SUBROUTINE IMPLEMENTATION IN 8085

The 8085 microprocessor handles subroutines using two main instructions:

- CALL: To transfer control to the subroutine
- RET: To return control back to the main program

Both instructions rely heavily on the **stack** and the **stack pointer (SP)** to store and retrieve the return address.

THE CALL INSTRUCTION

DEFINITION

The CALL instruction is used to invoke a subroutine. It causes the processor to jump to the memory location specified in the instruction operand.

SYNTAX:

CALL 16-bit address

OPERATION STEPS:

1. The CALL instruction is fetched, and the next two bytes in memory contain the **address of the subroutine**.
2. The microprocessor reads the 16-bit address and stores it in a temporary register pair, typically referred to as **WZ**.
3. The **current value of the program counter (PC)**, which points to the instruction after CALL, is **pushed onto the stack**. This serves as the **return address**.



4. The program counter is then updated with the subroutine address from the WZ register.
5. Control is transferred to the subroutine.

EXAMPLE:

CALL 000CH

This instruction will:

- Push the address of the next instruction onto the stack.
- Jump to memory location 000CH where the subroutine begins.

THE RET INSTRUCTION

DEFINITION

The RET instruction is used at the end of a subroutine to return control to the calling location in the main program.

SYNTAX:

RET

OPERATION STEPS:

1. The microprocessor **pops** the return address from the stack.
2. This address is loaded back into the **program counter (PC)**.
3. Execution resumes from the instruction **immediately following the CALL** instruction.

This mechanism ensures that the subroutine does not interfere with the flow of the main program once its job is done.

EXAMPLE:



RET

Placed at the end of the subroutine, this will return control to the main program.

SUMMARY OF CALL AND RET OPERATIONS

INSTRUCTION	FUNCTION
CALL	Saves the return address on stack and jumps to subroutine
RET	Retrieves the saved return address from stack and resumes execution

EXAMPLE SCENARIO

MAIN PROGRAM:

CALL 2000H ; Call subroutine

; other instructions

HLT ; End of main program

SUBROUTINE AT 2000H:

MVI A, 05H ; Load value into accumulator

RET ; Return to calling location

In this case, control jumps to address 2000H, executes the instruction to load 5 into register A, and then returns back to the main program after RET.

CONCLUSION

Subroutines are fundamental to building organized and reusable programs in assembly language. The 8085 microprocessor offers a straightforward way to implement subroutines using the CALL



and RET instructions. These instructions allow the program to jump to a task-specific routine and return gracefully, preserving the execution flow.

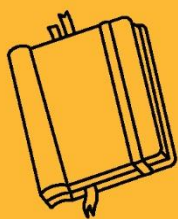
By using subroutines:

- The same block of instructions can be reused multiple times.
- Programs become shorter and more manageable.
- Stack-based execution ensures that the microprocessor remembers where to return after completing the subroutine.

Understanding the working of CALL and RET, and how the stack is used in this process, is essential for any programmer aiming to develop robust applications on the 8085 microprocessor.

KEY TAKEAWAYS

- CALL transfers control to a subroutine.
- RET returns control back to main program.
- Efficient for modular programming.



ADDITIONAL PROGRAMMING TECHNIQUES



SUB LESSON 11.4

RST INSTRUCTIONS

RST INSTRUCTIONS

UNDERSTANDING RST INSTRUCTIONS IN THE 8085 MICROPROCESSOR

In microprocessor programming, handling interrupts and transferring control to specific locations in memory quickly and efficiently is essential. The **8085 microprocessor** supports a unique set of instructions known as **Restart (RST) instructions**, which serve this very purpose. These are single-byte instructions that act as software interrupts or function calls, redirecting program execution to predefined memory locations. RST instructions are particularly useful in time-sensitive operations and interrupt service routines (ISRs) where speed and code compactness matter.

This write-up explores the function, structure, and applications of RST instructions in the 8085 microprocessor, highlighting their role in efficient subroutine handling and interrupt processing.

WHAT ARE RST INSTRUCTIONS?

RST stands for **Restart**. RST instructions are a group of eight software interrupt instructions available in the 8085 instruction set. These instructions are used to transfer program control to fixed memory locations, typically used to handle predefined subroutines or interrupt routines.

There are **eight RST instructions**, numbered from RST 0 to RST 7. Each of them points to a specific, hardcoded memory address.

THE RST INSTRUCTIONS AND THEIR CORRESPONDING ADDRESSES

RST Instruction	Target Address (in Hex)
RST 0	0000H



RST 1	0008H
RST 2	0010H
RST 3	0018H
RST 4	0020H
RST 5	0028H
RST 6	0030H
RST 7	0038H

Each of these instructions is **one byte** in size and automatically pushes the address of the next instruction onto the **stack** before jumping to the restart address. This makes them functionally similar to a CALL instruction, but with the advantage of speed and size efficiency.

SYNTAX AND OPERATION

SYNTAX:

RST n ; where n = 0 to 7

HOW IT WORKS:

- When an RST instruction is executed (e.g., RST 5), the microprocessor saves the **current program counter (PC)** on the stack.
- It then loads the program counter with a predefined address (in this case, 0028H for RST 5).
- Execution continues from that address.
- At the end of the routine, a RET instruction is typically used to return to the main program by popping the saved PC off the stack.

EXAMPLE:

RST 3



This instruction causes the program to jump to **0018H**. Before the jump, the return address (i.e., the address of the instruction following RST 3) is pushed onto the stack.

RST AS SOFTWARE INTERRUPTS

One of the primary uses of RST instructions is in implementing **software interrupts**. While hardware interrupts are triggered by external signals, software interrupts are triggered by instructions embedded in the program. RST instructions can simulate this behavior by immediately jumping to predefined memory locations.

For example, RST 7 is commonly used in DOS-based systems for interrupt 38H, which might be associated with keyboard input or screen display.

Using RST instructions as software interrupts is common in:

- Embedded systems
- Real-time control systems
- Small operating systems

RST IN HARDWARE INTERRUPTS

In addition to their use in software, RST instructions also serve an important function in **hardware interrupt handling**. In the 8085 microprocessor, certain hardware interrupts like TRAP, RST 7.5, RST 6.5, and RST 5.5 are internally mapped to specific RST instructions.

MAPPING EXAMPLE:

- **RST 7.5** corresponds to RST 6 → jumps to 0030H
- **RST 6.5** corresponds to RST 4 → jumps to 0020H
- **RST 5.5** corresponds to RST 2 → jumps to 0010H



These interrupts allow external hardware devices to signal the processor to execute a specific interrupt service routine (ISR) located at the restart address. This makes RST instructions highly efficient for managing multiple external devices and events.

ADVANTAGES OF USING RST INSTRUCTIONS

RST instructions offer several practical advantages in 8085 assembly programming:

1. COMPACTNESS

Being single-byte instructions, they save program memory space compared to multi-byte CALL instructions.

2. SPEED

RST instructions are faster to execute because they do not require fetching and decoding additional address bytes.

3. SIMPLICITY

Their fixed jump addresses simplify coding and debugging.

4. EFFICIENT ISR DESIGN

They allow immediate redirection to interrupt service routines, which is ideal in real-time applications.

LIMITATIONS OF RST INSTRUCTIONS

Despite their advantages, RST instructions have a few limitations:

- **Fixed Addresses:** All RST instructions transfer control to predefined locations, which limits flexibility in choosing where routines can be stored.



- **Limited Number:** Only eight RST vectors are available, which may not be enough for complex systems requiring many interrupts or subroutines.
- **Manual Return Handling:** The programmer must ensure the RET instruction is used to return control to the calling function, else the program flow will be disrupted.

USE CASES

RST instructions are ideal for:

- **Simple function calls** like clearing the screen, reading input, etc.
- **Time-critical interrupts** where response speed is crucial.
- **Embedded applications** that require minimal memory footprint and fast execution.

KEY TAKEAWAYS

- The RST instructions in the 8085 microprocessor provide a powerful tool for handling both software and hardware interrupts, as well as executing frequently used subroutines.
- Their simplicity, compactness, and speed make them highly useful in real-time systems and embedded applications.
- By redirecting execution to fixed memory addresses, RST instructions act like quick-access portals for subroutines and ISRs, ensuring efficient program flow.
- Understanding and correctly implementing RST instructions is essential for anyone working with the 8085 microprocessor or designing interrupt-driven systems



ADDITIONAL PROGRAMMING TECHNIQUES





SUB LESSON 11.5

RETURN INSTRUCTIONS AND STACK OPERATIONS DURING FUNCTION CALLS

INTRODUCTION

Function calls, also known as **subroutine calls**, are a fundamental part of structured programming in microprocessors. In the **8085 microprocessor**, these are implemented using the CALL and RET instructions. During a function call, the microprocessor must remember where to return after the subroutine finishes. This is done using a special memory area called the **stack**. The **Stack Pointer (SP)** and stack operations (PUSH, POP, etc.) play a vital role in managing return addresses and temporary data during these calls.

This write-up explains the **function of the RET (Return) instruction**, the **stack operations involved during function calls**, and how the stack enables smooth and reliable program control flow.

FUNCTION CALLS AND THE NEED FOR STACK

When a subroutine is called using the CALL instruction, the microprocessor must temporarily **leave the main program**, execute the subroutine, and then **return** to the correct point in the main program. To achieve this, it must:

1. **Save the return address** (the address of the instruction immediately after CALL).
2. **Jump to the subroutine** (a different memory location).
3. **Execute the subroutine instructions**.
4. **Return to the saved address** using the RET instruction.

Since the microprocessor can only execute one instruction at a time and has limited memory registers, it uses the **stack** to manage this process.



STACK BASICS

The **stack** is a section of the memory used for temporary storage during program execution. It works on the **Last In, First Out (LIFO)** principle—meaning the last item pushed onto the stack is the first one to be popped off.

The **Stack Pointer (SP)** is a special 16-bit register in 8085 that points to the current top of the stack. When data is **pushed**, the SP is **decremented**, and when data is **popped**, the SP is **incremented**. The stack grows downward in memory, from high addresses to low addresses.

CALL INSTRUCTION AND STACK OPERATIONS

When a CALL instruction is executed, the microprocessor performs the following operations:

1. **Stores the return address** (i.e., the address of the next instruction after CALL) on the stack.
2. **Decrements the SP** twice (since the address is 16-bit and stored in two bytes).
3. **Loads the Program Counter (PC)** with the subroutine address provided in the operand of CALL.

EXAMPLE:

CALL 2050H

If this instruction is at memory location 2000H, the return address (2003H) is pushed onto the stack, and the PC jumps to 2050H to execute the subroutine.

RET INSTRUCTION AND STACK OPERATIONS

The RET (Return) instruction is used at the **end of a subroutine** to return control to the calling point in the main program. The RET instruction:



1. **Reads the 16-bit return address** from the stack (starting from the memory location pointed by SP).
2. **Loads this address into the Program Counter (PC).**
3. **Increments the SP by 2**, effectively removing the return address from the stack.

This mechanism ensures the processor continues execution from where it left off before calling the subroutine.

SYNTAX:

RET

This instruction is essential to close a subroutine and return the flow of control correctly.

STACK EXAMPLE FOR FUNCTION CALL AND RETURN

Let's consider a short example:

MAIN PROGRAM:

LXI SP, 3000H ; Initialize Stack Pointer

CALL 2050H ; Call subroutine at 2050H

HLT ; Halt program

SUBROUTINE AT 2050H:

MVI A, 09H ; Load value into Accumulator

RET ; Return to calling function

STACK OPERATIONS:

- Before CALL, SP = 3000H.



- After CALL is executed:
 - SP becomes 2FFE_H.
 - Return address (2003_H) is saved at locations 2FFF_H and 2FFE_H.
- RET retrieves 2003_H from the stack and sets PC to it.

IMPORTANCE OF STACK DURING NESTED FUNCTION CALLS

When **functions are called within other functions**, the stack is essential for handling multiple return addresses. Each time a function is called, the return address is pushed onto the stack. When a RET is executed, the correct return address is popped.

In this case:

- When FUNC2 is called, its return address is pushed onto the stack.
- After FUNC2 finishes, RET pops that address, and control returns to FUNC1.
- Then FUNC1 finishes, and RET returns control to the main program.

This nested behavior is possible only because the stack maintains a **history of return addresses** in the correct order.

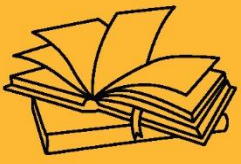
SUMMARY OF STACK USE IN FUNCTION CALLS

Operation	Stack Activity
CALL	Return address pushed onto stack
RET	Return address popped from stack
Nested Calls	Multiple return addresses handled via stack
Stack Pointer	Automatically adjusted (SP−2 for CALL, SP+2 for RET)

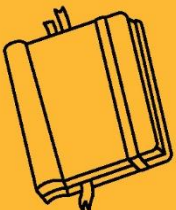


KEY TAKEAWAYS

- RET restores program counter from stack.
- Ensures proper return after subroutine execution.
- Stack maintains execution context.



ADDITIONAL PROGRAMMING TECHNIQUES





SUB LESSON 11.6

BINARY TO BCD AND DECIMAL CONVERSION

BINARY TO BCD AND DECIMAL CONVERSION

In digital electronics and microprocessor programming, numbers are represented and manipulated in various formats. Two common number systems are **binary** and **decimal**. While binary is the natural language of digital computers, decimal is more intuitive and widely used by humans. Additionally, **Binary Coded Decimal (BCD)** is a form of decimal representation where each decimal digit is encoded separately in binary form. Understanding the conversion between these systems is essential for designing efficient digital circuits and for programming tasks involving human-readable data displays or calculations.

This write-up explains the concepts of binary, decimal, and BCD numbers, and presents methods for converting binary numbers to BCD and decimal formats.

NUMBER SYSTEMS OVERVIEW

BINARY NUMBER SYSTEM

- Binary numbers use base 2.
- Each digit (bit) can be either 0 or 1.
- Example: Binary 1011_2 equals decimal 11_{10} .
- Computers operate natively on binary data.

DECIMAL NUMBER SYSTEM

- Decimal numbers use base 10.
- Digits range from 0 to 9.
- Humans typically use decimal for counting and calculations.



- Example: Decimal 145 means $1 \times 10^2 + 4 \times 10^1 + 5 \times 10^0$.

BINARY CODED DECIMAL (BCD)

- BCD represents each decimal digit as its 4-bit binary equivalent.
- Each digit of a decimal number is stored separately.
- For example, decimal 59 in BCD is 0101 1001.
 - 5 → 0101
 - 9 → 1001

BCD simplifies interfacing with decimal displays and reduces conversion complexity in certain applications.

WHY CONVERT BINARY TO DECIMAL OR BCD?

Microprocessors work internally with binary numbers. However, applications like calculators, digital clocks, and display devices require numbers in decimal or BCD format for easy human interpretation. Direct binary-to-decimal conversion is sometimes complex and costly in terms of processor time, so BCD representation serves as an intermediate or alternative.

METHODS OF BINARY TO DECIMAL CONVERSION

Converting binary to decimal involves calculating the sum of powers of two corresponding to bits set to 1.

METHOD 1: POSITIONAL WEIGHT METHOD

Each bit is multiplied by 2 raised to its positional power, then all results are summed.

Example: Convert 101101_2 to decimal



Bit position (right to left) 5 4 3 2 1 0

Binary digit 1 0 1 1 0 1

Value (bit $\times 2^{\text{position}}$) 32 0 8 4 0 1

Sum = $32 + 0 + 8 + 4 + 0 + 1 = 45$ decimal.

This is straightforward but inefficient for large binary numbers or embedded systems with limited computation.

BINARY TO BCD CONVERSION

Since each BCD digit corresponds to 4 bits, the conversion from binary to BCD involves separating the binary number into groups representing decimal digits.

COMMON ALGORITHM: DOUBLE DABBLE

A widely used algorithm for binary to BCD conversion is the **Double Dabble** or **Shift and Add 3** method, often implemented in hardware or software.

STEPS:

1. Initialize BCD digits to zero.
2. Shift the binary number left by one bit.
3. After each shift, check each BCD digit:
 - If a BCD digit is ≥ 5 , add 3 to it.
4. Repeat until all bits of the binary number are shifted.

EXPLANATION

Adding 3 to BCD digits ≥ 5 adjusts the digits so they remain valid BCD digits during the shifting process. This ensures that each 4-bit BCD nibble correctly represents a decimal digit.

EXAMPLE OF DOUBLE DABBLE ALGORITHM



Convert binary number 1011 (decimal 11) to BCD:

- Start with BCD digits 00 00.
- Shift left, process each bit with add-3 rule.
- After all bits are shifted and adjusted, the BCD result is 0001 0001, which corresponds to decimal 11.

DIRECT BINARY TO BCD CONVERSION PROGRAM

In microprocessor programming, the double dabble algorithm can be implemented via assembly instructions. The process includes:

- Using shift instructions to shift the binary number bit by bit.
- Checking each nibble (4 bits) to decide whether to add 3.
- Storing the result as BCD digits in separate registers or memory locations.

This method allows precise conversion without floating-point calculations, ideal for 8-bit microprocessors like 8085.

ADVANTAGES OF BCD REPRESENTATION

- **Ease of display:** Each BCD digit can be easily mapped to seven-segment displays.
- **Simplifies decimal arithmetic:** Addition and subtraction are easier compared to pure binary for decimal digits.
- **No floating point needed:** Useful in systems without floating-point hardware.

CHALLENGES WITH BCD

- **Storage inefficiency:** BCD uses 4 bits to store a decimal digit that could be stored more compactly in pure binary.
- **Slower arithmetic:** BCD arithmetic often requires correction steps, making it slower than binary arithmetic.



SUMMARY OF CONVERSION PROCESSES

Conversion Type	Description	Usage
Binary to Decimal	Calculate sum of bit $\times$ powers of 2	Display or human-readable output
Binary to BCD	Use Double Dabble algorithm	Display devices, calculators
Decimal to Binary	Divide decimal by 2 repeatedly	Machine processing

KEY TAKEAWAYS

- Binary is the native language of digital systems, while decimal is preferred for human interfaces.
- BCD provides a middle ground, allowing each decimal digit to be individually encoded using 4-bit binary.
- Binary to Decimal conversion uses positional weight methods, suitable for small-scale data.
- Binary to BCD conversion relies on the Double Dabble algorithm, which is efficient and hardware-friendly.
- BCD simplifies interfacing with display hardware and avoids complex binary-to-decimal conversion steps.
- Despite its advantages, BCD is less storage-efficient and has slower arithmetic performance.
- Practical applications, such as digital clocks and calculators, rely heavily on BCD conversion routines.



OUTCOMES OF CHAPTER 11 (STACK & SUBROUTINES, RESTART, CALL AND RETURN INSTRUCTION, CODE CONVERSION)

- Understand the concept of the **stack** in the 8085 microprocessor, including its Last-In-First-Out (LIFO) behavior and the role of the Stack Pointer (SP).
- Learn how to perform **stack operations** such as PUSH and POP to save and restore register pairs during program execution.
- Comprehend the implementation and importance of **subroutines** using CALL and RET instructions, enabling modular and reusable code.
- Analyze how the stack supports **return address storage** during function calls, including nested subroutine handling.
- Gain knowledge of **RST (Restart) instructions**, their fixed vector addresses, and their use in software interrupts and hardware interrupt handling..Develop the ability to write assembly language programs incorporating stack management, subroutine calls, and restart instructions to create efficient and structured 8085 microprocessor



8085 INTERRUPTS





SUB LESSON 12.1

INTRODUCTION

OBJECTIVES OF CHAPTER 12 (8085 INTERRUPTS)

1. To explain the 8085 interrupts.
2. To provide an understanding of Interrupt service routine.
3. To describe microprocessor interrupt classification.
4. To explore the processing of interrupt.

INTRODUCTION

In digital systems and embedded computing, real-time responsiveness is crucial. A microprocessor must often react immediately to external or internal events while executing its main program. To facilitate this responsiveness, the concept of interrupts is introduced. Interrupts are mechanisms that allow a microprocessor, such as the Intel 8085, to temporarily suspend its ongoing operations and respond to high-priority events. This chapter explores the definition, functionality, types, and application of interrupts in the 8085 microprocessor.

WHAT IS AN INTERRUPT?

An interrupt is a signal sent to the processor by hardware or software to request its attention. This signal indicates that an event requiring immediate processing has occurred. Upon receiving this signal, the microprocessor temporarily pauses its current program execution and transfers control to a specific routine known as an Interrupt Service Routine (ISR). After servicing the interrupt, the processor resumes execution of the main program from where it was interrupted.

In essence, an interrupt acts as an emergency mechanism that informs the CPU of an urgent task. For instance, if an input/output device is ready to send or receive data, it can notify the microprocessor through an interrupt.



APPLICATIONS OF INTERRUPTS

Interrupts are widely used in real-time systems to enhance efficiency and responsiveness. Key application areas include:

Input/Output Operations: Interrupts are heavily employed to manage I/O operations. For example, rather than continuously polling a keyboard for input, the system can respond to a keypress using an interrupt.

Timers and Clocks: Periodic interrupts can be generated using timers to maintain system clocks, perform time-sensitive tasks, or generate delays.

Communication Interfaces: In serial and parallel communication, interrupts are essential for handling data transmission and reception without constantly checking ports.

Multitasking Systems: Interrupts allow the CPU to switch between tasks efficiently, supporting multitasking and context switching.

How the Microprocessor Responds to Interrupts

When an interrupt is detected, the 8085 microprocessor performs the following steps:

Suspends the current instruction execution.

- Saves the address of the next instruction (called the program counter) onto the stack.
- Transfers control to the ISR associated with the interrupt.
- Executes the ISR instructions.
- Retrieves the stored return address from the stack using the RET instruction.
- Resumes execution of the interrupted program.
- This mechanism ensures that the system can address critical events without losing the context of the ongoing process.



INSTRUCTION LENGTH AND INTERRUPTS

Interrupt instructions, like other instructions in 8085, can be classified by their byte length:

- One-byte Instructions: E.g., RST n
- Two-byte Instructions: E.g., MVI A, data
- Three-byte Instructions: E.g., LXI H, address

When an interrupt occurs, regardless of instruction length, the processor ensures that the correct return point is saved, and the original program can resume seamlessly after the ISR is completed.

ADVANTAGES OF USING INTERRUPTS

- Efficient Resource Utilization: The processor can perform other tasks instead of continuously checking for an event (polling).
- Improved System Performance: The CPU can respond to high-priority tasks without delay.
- Supports Real-Time Processing: Ensures timely handling of critical events like alarms or sensor triggers.
- Scalability and Modularity: Easy to integrate additional interrupt-driven devices.

KEY TAKEAWAYS

- Interrupts allow peripheral devices to request CPU attention.
- Enhance system responsiveness.
- Can be maskable or non-maskable.



8085 INTERRUPTS



SUB LESSON 12.2

TYPES OF INTERRUPTS

TYPES OF INTERRUPTS

In the architecture of a microprocessor-based system, the ability to handle external and internal events effectively is vital. The 8085 microprocessor supports this capability through the use of interrupts—signals that temporarily divert the CPU from its current task to perform a more immediate one. This mechanism is essential in real-time and embedded systems where certain operations must be executed as soon as a specific event occurs.

This chapter explores the different types of interrupts in the 8085 microprocessor, explaining how they are classified, their characteristics, and their roles in improving system efficiency and responsiveness.

DEFINITION OF AN INTERRUPT

An interrupt is a mechanism by which the normal sequence of execution of instructions is halted so that the processor can respond to a more urgent task. Once the interrupt is serviced, the CPU resumes its previous activity from where it left off.

The interrupt mechanism is essential in modern computing systems to manage tasks like input/output (I/O) communication, timers, and error handling. In the 8085 microprocessor, interrupts can be initiated both internally by software and externally by hardware.

CLASSIFICATION OF INTERRUPTS IN 8085

Interrupts in the 8085 microprocessor are broadly classified into:

Hardware Interrupts

Software Interrupts

Further, based on other properties, they can be classified as:

Maskable and Non-maskable

Vectored and Non-vectored

Each of these classifications has a unique role in managing CPU behavior and efficiency.



HARDWARE INTERRUPTS

Hardware interrupts are those initiated by external hardware devices via physical pins on the microprocessor. These are asynchronous events that may occur at any time during the execution of a program.

The 8085 microprocessor includes the following five hardware interrupts:

TRAP

RST7.5

RST6.5

RST5.5

INTR (Interrupt Request)

TRAP

- This is the highest-priority and non-maskable hardware interrupt.
- It is both edge- and level-triggered, which means it must be recognized immediately and cannot be disabled.
- Typically used for urgent or critical events like power failure or emergency shutdown.
- RST7.5
- A maskable and edge-triggered interrupt.
- Can be enabled or disabled through software.
- Has a higher priority than RST6.5 and RST5.5.
- RST6.5 and RST5.5
- Both are maskable and level-triggered interrupts.
- Their execution priority is lower than RST7.5 and higher than INTR.
- INTR (Interrupt Request)
- A general-purpose, maskable, non-vectored interrupt.
- Its servicing requires the external device to supply the address of the ISR (Interrupt Service Routine).



SOFTWARE INTERRUPTS

Software interrupts are initiated within the program using specific instructions. These are predefined and synchronous, meaning they are executed in sequence as part of the code. The instruction used for software interrupts is RST n (where n = 0 to 7), making a total of eight software interrupts:

RST 0, RST 1, RST 2, RST 3, RST 4, RST 5, RST 6, RST 7

Each RST instruction corresponds to a fixed memory location (a vector address) where the interrupt service routine is located.

For example:

RST 0 → 0000H

RST 1 → 0008H

RST 2 → 0010H

.

.

RST 7 → 0038H

Software interrupts are useful for function calls, error handling, and debugging, allowing the programmer to execute code at specific memory locations.

MASK-ABLE VS. NON-MASKABLE INTERRUPTS

Interrupts can be enabled or disabled based on their criticality. This property divides them into:

MASKABLE INTERRUPTS

These interrupts can be disabled (masked) by the programmer using software instructions (DI, SIM).

They allow the CPU to temporarily ignore certain interrupts while executing critical sections of code.

Examples: RST7.5, RST6.5, RST5.5, INTR

NON-MASKABLE INTERRUPTS

These interrupts cannot be disabled, ensuring that the CPU always responds to them.



They are typically used for catastrophic events that require immediate action.

Example: TRAP

VECTORED VS. NON-VECTORED INTERRUPTS

Interrupts also differ in the way the address of the ISR is handled:

VECTORED INTERRUPTS

In these, the ISR address is predefined and known to the microprocessor.

The CPU automatically transfers control to this address upon receiving the interrupt.

Examples: TRAP (0024H), RST7.5 (003CH), RST6.5 (0034H), RST5.5 (002CH), RST0–RST7 (fixed addresses)

NON-VECTORED INTERRUPTS

These interrupts do not have predefined addresses.

The address must be provided externally by the interrupting device.

Example: INTR

This distinction determines how fast and efficiently the CPU can handle an interrupt, as vectored interrupts do not require extra address-fetching operations.

PRIORITY OF HARDWARE INTERRUPTS IN 8085

In 8085, hardware interrupts are handled based on a fixed priority order when multiple interrupts occur simultaneously:

TRAP – Highest Priority

RST7.5

RST6.5

RST5.5

INTR – Lowest Priority

This priority system ensures that the most critical tasks are serviced before less urgent ones.

INSTRUCTIONS RELATED TO INTERRUPT CONTROL

To manage interrupt execution, 8085 provides the following instructions:



EI (Enable Interrupts): Enables all maskable interrupts.

DI (Disable Interrupts): Disables all maskable interrupts.

SIM (Set Interrupt Mask): Used to mask or unmask specific interrupts.

RIM (Read Interrupt Mask): Used to check the status of interrupts and serial input data.

These instructions give the programmer flexibility in designing interrupt-driven programs.

ADVANTAGES OF INTERRUPTS IN 8085

Efficient CPU Utilization: Instead of polling devices continuously, the CPU can execute other instructions and respond to interrupts only when required.

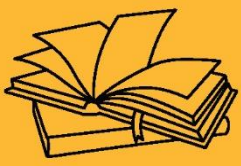
Real-Time Processing: Interrupts enable quick responses to real-world signals like sensor input or device errors.

Modular Programming: Interrupts separate critical processes into dedicated routines, enhancing program structure and readability.

Multitasking Support: Interrupts are foundational for systems that require simultaneous handling of multiple tasks or peripherals.

KEY TAKEAWAYS

- Hardware (TRAP, RST 7.5, 6.5, 5.5, INTR) and software interrupts.
- TRAP is non-maskable; others are maskable.
- Each has a priority and vector address.



8085 INTERRUPTS





SUB LESSON 12.3

PROCESSING OF INTERRUPTS

INTRODUCTION

In microprocessor systems, especially in real-time and embedded applications, it is essential to handle external and internal events promptly. The 8085 microprocessor achieves this efficiency using interrupts, which are signals that request the processor to pause its current activity and handle an urgent task. While previous chapters introduced the concept and types of interrupts, this section explores how interrupts are processed step-by-step. Understanding this process is vital for anyone looking to design responsive microprocessor-based systems.

WHAT IS INTERRUPT PROCESSING?

Interrupt processing refers to the systematic response of the microprocessor when it receives an interrupt signal. This mechanism allows the microprocessor to pause the currently executing program, save its context, respond to an interrupt through a dedicated Interrupt Service Routine (ISR), and then resume the previously suspended program.

The purpose of this process is to allow asynchronous events—events that occur independently of the current program execution—to be handled quickly and efficiently without losing the state of the system.

INTERRUPT FLOW IN 8085 MICROPROCESSOR

The interrupt processing cycle in the 8085 microprocessor follows a well-defined flow:

Main Program Execution: The processor executes its regular program instructions.

Interrupt Signal Received: A hardware device (or software command) sends an interrupt signal.

Interrupt Acknowledgment: If the interrupt is valid and enabled, the processor acknowledges it.

Context Saving: The current state of the processor, particularly the Program Counter (PC), is saved so execution can resume after servicing the interrupt.

ISR Execution: Control is transferred to a specific memory location called the Interrupt Service Routine (ISR).



RETI Instruction: At the end of the ISR, the RETI (Return from Interrupt) instruction signals that interrupt handling is complete.

Program Resumption: The processor retrieves the saved PC value and resumes the main program from where it was interrupted.

This flow ensures that the processor can handle external tasks without losing track of its original operation.

STEP-BY-STEP INTERRUPT PROCESSING IN 8085

Let's break down the interrupt processing cycle in more detail:

Interrupt Request Detection: A hardware device sends a signal to the microprocessor through one of its interrupt pins (e.g., INTR, RST7.5).

Instruction Completion: The processor allows the current instruction to finish executing before acting on the interrupt.

Interrupt Acknowledgment:

The processor checks if interrupts are enabled (using the EI instruction).

It also verifies whether the interrupt is unmasked (not blocked by SIM).

If both conditions are met, the CPU acknowledges the interrupt.

SAVING CONTEXT:

The value in the Program Counter (PC) is pushed onto the stack.

This is essential for returning to the exact point where the program was interrupted.

JUMP TO ISR:

For vectored interrupts, the address of the ISR is predefined.

For non-vectored interrupts, the external device must provide the ISR address.

The PC is then loaded with the ISR address, and control transfers to the ISR.

ISR EXECUTION:

The ISR is a small program written to handle the specific interrupt.

It may involve reading input, clearing buffers, sending acknowledgments, or initiating new operations.



RETURN TO MAIN PROGRAM:

The RETI instruction is executed at the end of the ISR.

It signals the end of interrupt servicing.

The processor pops the return address from the stack into the PC and resumes execution of the main program.

INTERRUPT CONTROL INSTRUCTIONS IN 8085

To effectively manage the execution of interrupts, the 8085 provides a set of interrupt control instructions:

1. EI (Enable Interrupts)

Enables all maskable interrupts.

Must be used to allow the processor to respond to interrupts after power-on or after a DI instruction.

2. DI (Disable Interrupts)

Disables all maskable interrupts.

Useful when executing critical code that must not be interrupted.

3. SIM (Set Interrupt Mask)

Used to set or reset interrupt masks and control RST7.5 latching.

Allows selective enabling/disabling of RST7.5, RST6.5, and RST5.5.

4. RIM (Read Interrupt Mask)

Checks the status of interrupt masks and serial input data.

Used to determine which interrupts are enabled or pending.

Example Code:

```
MVI A, 08H ; Mask RST6.5 and RST5.5, enable RST7.5
```

```
SIM
```

```
EI ; Enable interrupts
```

REAL-LIFE EXAMPLE: KEYBOARD INPUT VIA INTR

To illustrate interrupt processing in a practical context, consider a scenario where a keyboard sends a signal to the microprocessor through the INTR pin:



A key is pressed, and the keyboard sends an interrupt signal.

The 8085 finishes its current instruction.

If interrupts are enabled, it acknowledges the INTR.

The PC is pushed onto the stack.

The ISR associated with INTR reads the key from the input port.

After the ISR executes, the RETI instruction is used to return to the main program.

The main program resumes from the point it was paused.

This real-life example demonstrates how interrupts facilitate real-time interaction between external devices and the CPU.

APPLICATIONS OF INTERRUPT PROCESSING

Interrupt processing is a powerful feature that is extensively used in various applications, including:

Real-Time Systems: Immediate response to critical events like temperature thresholds, voltage limits, or emergency shutdowns.

Embedded Control Systems: Managing multiple peripherals and sensors in robotics, home automation, or industrial machinery.

Sensor Data Acquisition: Automatically triggering data collection when a sensor signal is received.

Communication Interfaces: Handling asynchronous data transmission and reception in serial and parallel communication.

ADVANTAGES OF INTERRUPT PROCESSING

Asynchronous Event Handling: The processor can respond to events as they occur, without waiting or continuous polling.

Efficient CPU Utilization: Interrupts allow the processor to execute other instructions while waiting for an event, thus reducing idle time.

Simplified Program Flow: Interrupts help break complex systems into manageable routines, improving modularity and clarity.

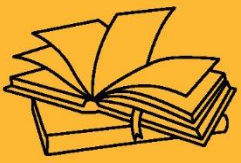
Enhanced Responsiveness: Time-critical applications benefit from faster reaction times due to interrupt-driven control.



Resource Optimization: Hardware resources like timers and I/O devices are better managed using interrupts than through constant monitoring.

KEY TAKEAWAYS

- CPU suspends current task and jumps to ISR.
- ISR executes and returns using RET.
- Flags and registers may be saved/restored.



8085 INTERRUPTS





SUB LESSON 12.4

INTERRUPT SERVICE ROUTINE

INTERRUPT SERVICE ROUTINE

In microprocessor-based systems, interrupts are vital for managing real-time events and asynchronous hardware or software requests. When an interrupt occurs, the processor halts the current program, performs a specific task to respond to the interrupt, and resumes the original program afterward. The dedicated set of instructions executed in response to the interrupt is called an Interrupt Service Routine (ISR).

The 8085 microprocessor uses ISRs to address interrupt signals systematically and ensure that each request is serviced correctly and efficiently. This chapter provides a deep dive into what an ISR is, how it functions, its structure, and its role in enhancing processor performance and responsiveness.

WHAT IS AN INTERRUPT SERVICE ROUTINE (ISR)?

An Interrupt Service Routine (or interrupt handler) is a small program designed to handle the specific task related to an interrupt. When a device triggers an interrupt, it effectively asks the microprocessor to perform a task—just like calling a subroutine. The CPU then pauses the ongoing process and jumps to the ISR, which addresses the interrupt's requirements. Once the ISR is completed, control is returned to the main program.

The concept is comparable to an emergency service—when an urgent situation arises (an interrupt), the service (ISR) is called to resolve it while normal operations are temporarily suspended.

FUNCTION OF ISR IN 8085

When an interrupt occurs in the 8085 system, the microprocessor undertakes the following steps involving the ISR

- **Pause Current Program Execution:** The ongoing program is temporarily halted after the current instruction completes.



- **Save the Context:** The address of the next instruction (Program Counter) is saved on the stack so the program can resume after servicing the interrupt.
- **Transfer Control to ISR:** The microprocessor jumps to the specific memory location of the ISR. For vectored interrupts like RST 7.5 or TRAP, these addresses are predefined.
- **Execute ISR:** The interrupt-specific routine runs. For instance, if the interrupt is due to a keyboard input, the ISR reads the keystroke.
- **Return to Main Program:** The ISR ends with the RETI instruction, allowing the processor to return to the main program using the saved address.
- This ensures seamless and systematic handling of real-time events without disrupting the overall program logic.

STRUCTURE AND CHARACTERISTICS OF ISRS

ISRs in the 8085 have the following key characteristics:

Short and Efficient: Since ISRs interrupt the main program flow, they should be concise to minimize execution delay.

Non-returnable: Unlike normal subroutines that may return multiple times or conditionally, ISRs have a fixed return path using the RETI instruction.

Independent: ISRs should not depend on the main program's temporary data or register states unless properly saved and restored.

Safe: They must not interfere with the main program's flow or corrupt shared data.

KEY INSTRUCTIONS FOR INTERRUPT CONTROL

The 8085 microprocessor provides special instructions to manage interrupt execution and handling:

1. EI (Enable Interrupts)

A single-byte instruction that enables all maskable interrupts (RST7.5, RST6.5, RST5.5, INTR).

Must be executed to allow the processor to respond to these interrupts.

2. DI (Disable Interrupts)

Disables all maskable interrupts.



Used to prevent interruptions during critical code sections or ISRs.

These instructions are critical in defining when and how the CPU should respond to incoming interrupt signals.

EXAMPLE SCENARIO OF ISR IN ACTION

Let's consider an example of a temperature monitoring system:

- A sensor continuously checks the room temperature.
- If the temperature exceeds a set limit, it triggers an interrupt signal via RST6.5.
- The 8085 processor pauses its ongoing operation and jumps to the ISR at 0034H (address for RST6.5).
- The ISR may activate a cooling fan, log the event, and display an alert.
- Once the ISR completes these actions, it executes the RETI instruction.
- The processor returns to the original task it was performing before the interrupt occurred.
- This flow demonstrates how ISRs allow real-time decision-making and action in embedded systems.

DESIGN GUIDELINES FOR ISRS

When writing ISRs, the following best practices should be followed:

Use Stack for Register Protection: Save and restore any registers used in the ISR to prevent data corruption.

```
PUSH PSW
```

```
; ISR code
```

```
POP PSW
```

Keep ISRs Short: Long ISRs can cause system delay or lead to missed interrupts. Avoid complex processing.

Avoid Nested Interrupts Unless Necessary: While the 8085 allows nested interrupts (higher-priority interrupts interrupting a running ISR), this should be handled carefully to avoid system instability.

End with RETI Instruction: This ensures the processor returns properly to the main program and updates the interrupt control logic.



ADVANTAGES OF USING ISRS

The use of ISRs brings several advantages to microprocessor-based systems:

Improved Responsiveness: Real-time events are handled immediately without the need for polling.

Efficient CPU Utilization: The processor can continue executing useful instructions instead of waiting for events.

Modular Program Design: Each interrupt can be handled by a dedicated routine, improving program readability and structure.

Support for Multitasking: ISRs allow multiple hardware devices to interact with the processor without conflict.

ISRS IN EMBEDDED SYSTEMS

ISRs are integral to the operation of various embedded and real-time systems, such as:

Alarm Systems: Triggering alerts on door or window breach.

Industrial Control Systems: Reacting to sensor inputs in automation equipment.

Medical Devices: Responding to heart rate changes or abnormal signals.

Consumer Electronics: Handling button presses on a microwave or remote control.

Communication Devices: Managing data reception over serial communication.

In each of these cases, ISRs provide the necessary responsiveness to ensure safety, functionality, and user interaction.

REAL-TIME CONSTRAINTS AND ISR MANAGEMENT

In real-time systems, ISRs must meet deadline constraints. A delay in responding to an interrupt could lead to data loss, performance degradation, or even system failure (e.g., in automotive systems). Therefore:

ISRs should be prioritized according to the criticality of the events they handle.

Interrupts with lower urgency can be masked when high-priority ISRs are running.

Non-essential tasks should be handled outside the ISR to avoid delays.



KEY TAKEAWAYS

- Contains code executed when an interrupt occurs.
- Should be short and fast.
- Must preserve CPU context if needed.

OUTCOMES OF CHAPTER 12 (8085 INTERRUPTS)

- Understand the concept of interrupts and explain how the 8085 microprocessor handles asynchronous events using hardware and software interrupt mechanisms.
- Differentiate among types of interrupts including maskable/non-maskable and vectored/non-vectored, and describe their roles and priority structure in real-time systems.
- Explain the interrupt processing cycle in 8085 including context saving, ISR execution, and program resumption using control instructions like EI, DI, SIM, and RIM.
- Design and implement efficient Interrupt Service Routines (ISRs) to respond to external events, ensuring reliable and responsive microprocessor operation in embedded applications.



COMPUTER ORGANIZATION



SUB LESSON 13.1

INTRODUCTION

OBJECTIVES OF CHAPTER 13 (COMPUTER ORGANIZATION)

1. To explain the classification of microprocessors based on architecture and bit-width.
2. To provide an understanding of instruction sets, types of instructions, and assembly language programming.
3. To describe microprocessor bus organization and its role in data communication and interfacing.
4. To explore addressing modes and the programming model of the 8085 microprocessor in detail.

INTRODUCTION

Computer Organization is a foundational concept in computer science and engineering that focuses on the operational structure of a computer system. It explores how the different hardware components of a computer function and interact to execute programs and process data. Unlike computer architecture, which deals with the conceptual design and fundamental operational structure of a computer system, computer organization zooms into the implementation details of these structures, such as how data flows, how instructions are executed, and how control signals manage operations.

OVERVIEW OF COMPUTER ORGANIZATION

The study of computer organization addresses the behavior and structure of computer systems at the hardware level. It provides insight into how various components such as the central processing unit (CPU), memory, input/output devices, and control units collaborate to perform computing tasks. Understanding these elements is essential for both hardware designers and



software developers to create efficient and optimized systems. Key elements of computer organization include:

The Central Processing Unit (CPU) – Often referred to as the brain of the computer, the CPU carries out arithmetic, logic, and control operations.

- Memory Systems – These store instructions and data temporarily or permanently.
- Input and Output Units – These facilitate communication between the user and the computer.
- Control Unit – This component directs the operation of the processor.

Each of these parts plays a vital role in the overall function of a computer system, and their coordination is what enables efficient processing and computing.

BASIC FUNCTIONAL UNITS

The functional structure of a computer system is composed of several units working together. These include:

- Input Unit – Receives data from external sources (keyboard, mouse, sensors, etc.) and converts it into a form the computer can process.
- Memory Unit – Stores both data and instructions required during computation. This includes:
 - Primary memory like RAM and ROM
 - Secondary memory such as hard drives and solid-state drives
- Arithmetic Logic Unit (ALU) – Executes arithmetic and logical operations.
- Control Unit – Directs all activities within the computer by interpreting instructions and generating control signals.



- Output Unit – Translates and presents processed data in human-readable formats (monitor, printer, etc.).

These units are the basic building blocks of any computer system, and understanding their functions provides insight into how computers work internally.

REGISTER TRANSFER LANGUAGE (RTL)

Register Transfer Language (RTL) is a symbolic language used to describe operations within a computer's CPU. It plays a key role in specifying the micro-operations performed on data stored in registers.

In RTL, operations are described in terms of data movement between registers. For example, the expression $R1 \leftarrow R2 + R3$ indicates that the sum of the contents of registers R2 and R3 is calculated and the result is stored in R1. This form of notation is crucial for designing control logic in digital circuits and understanding how data flows through a system.

RTL is particularly useful in:

Defining internal CPU operations

Describing control steps in instruction execution

Designing control units and datapaths

It forms the bridge between high-level algorithmic design and low-level hardware implementation.

REGISTERS AND NOTATION

Registers are small, fast storage locations within the CPU used to hold temporary data and instructions during execution. They serve as essential components for quick data access, reducing the time needed to fetch data from main memory.

Commonly used registers include:



R1, R2, etc. – General-purpose registers

PC (Program Counter) – Holds the address of the next instruction to be executed

IR (Instruction Register) – Stores the currently executing instruction

MAR (Memory Address Register) – Contains the address of memory to be accessed

MDR (Memory Data Register) – Holds data read from or written to memory

Typical operations performed using registers include:

Transfer – Moving data from one register to another

Increment – Increasing a register value by one

Clear – Resetting a register value to zero

Load – Storing new data into a register

Understanding the role of registers is essential for writing low-level programs and optimizing CPU performance.

MEMORY ORGANIZATION

Memory organization describes how the computer's memory is structured and accessed. Memory is typically arranged in a hierarchical model that balances speed and storage capacity:

- Cache Memory – Small and fast, located close to the CPU; stores frequently used data and instructions.
- Main Memory (RAM) – Moderate speed and size; stores programs and data currently being processed.
- Secondary Storage – Large capacity but slower; includes hard drives, SSDs, and other permanent storage solutions.



Memory is divided into cells arranged in a matrix, with each cell uniquely addressable. The system uses addresses to read from and write to these cells efficiently.

MEMORY TYPES

Memory types are classified based on volatility and use:

PRIMARY MEMORY

- RAM (Random Access Memory) – Volatile; stores data temporarily while the computer is on.
- ROM (Read-Only Memory) – Non-volatile; used to store firmware and bootstrapping code.
- Cache – Very fast volatile memory used by the CPU for high-speed data access.

SECONDARY MEMORY

HDD (Hard Disk Drive) and SSD (Solid State Drive) – Non-volatile; store data permanently.

Volatile memory loses its contents when the power is turned off, while non-volatile memory retains data without power.

BUS DESIGN CONCEPTS

A bus is a shared communication path used to transfer data between different components of a computer. There are three main types of buses:

- Data Bus – Carries actual data.
- Address Bus – Carries addresses of memory locations.
- Control Bus – Carries control signals to coordinate operations.



Since multiple devices might attempt to use the bus simultaneously, bus arbitration is used to manage access, ensuring that only one device controls the bus at any given time

MULTIPLEXERS IN BUS DESIGN

A multiplexer is a hardware component that selects one of many input lines and forwards it to a single output line. In bus systems, multiplexers simplify design by reducing the number of data lines needed for communication.

They operate under the control of selection lines, which determine which input is selected at any time. This mechanism helps in optimizing the layout and complexity of digital circuits, especially in data transfer operations.

TRI-STATE BUFFERS

A tri-state buffer is a type of digital circuit that can exist in one of three states: logic high, logic low, or high-impedance (Z-state). The high-impedance state effectively disconnects the buffer from the bus, allowing other devices to use it without interference.

Tri-state buffers are crucial in systems where multiple devices share a common communication line, such as a system bus. Controlled by an enable signal, these buffers prevent data collision and ensure smooth operation.

MEMORY TRANSFERS

The movement of data between the CPU and memory is facilitated through two special registers:

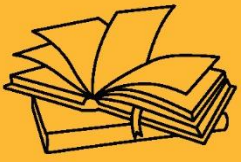
MAR (Memory Address Register) – Specifies the address in memory where data is to be accessed.

MDR (Memory Data Register) – Holds the data that is being transferred to or from the memory.



KEY TAKEAWAYS

- Computer organization focuses on internal structure and control.
- Includes memory, control units, ALU, and buses.
- Supports efficient program execution.



COMPUTER ORGANIZATION





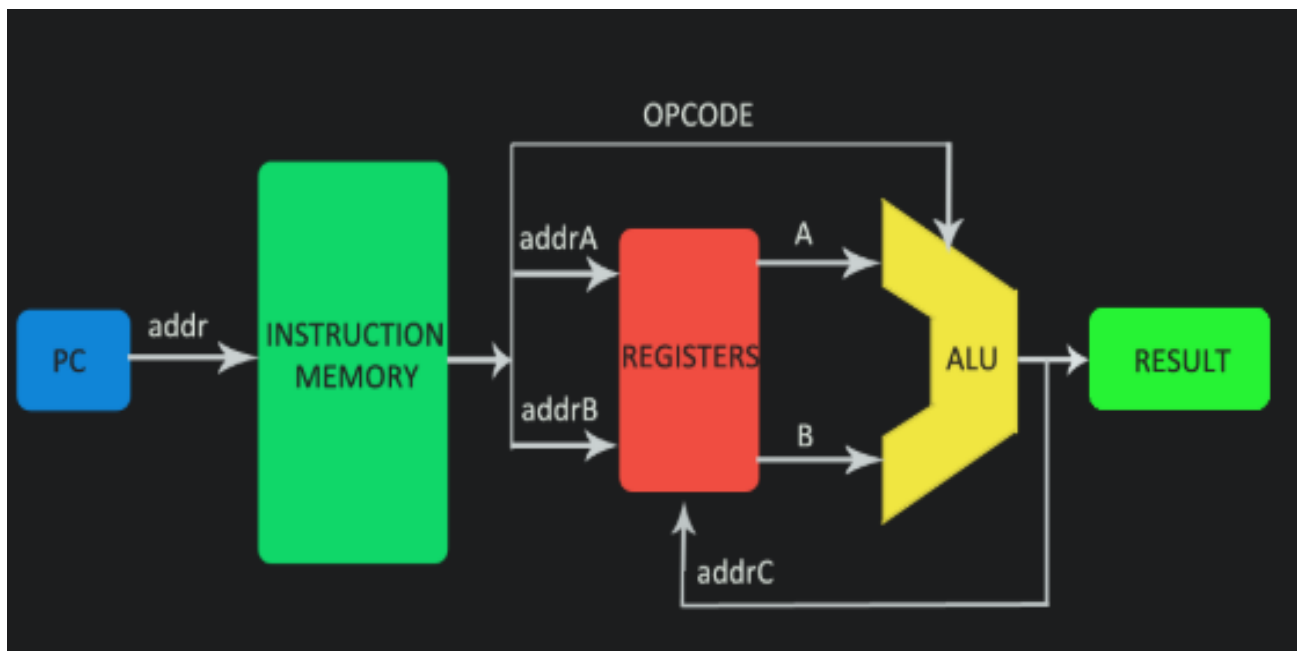
SUB LESSON 13.2

REGISTER TRANSFER LANGUAGE

REGISTER TRANSFER LANGUAGE

Register Transfer Language (RTL) plays a foundational role in understanding the internal operations of a computer's central processing unit (CPU). It provides a symbolic representation of the data flow between registers and describes how micro-operations are carried out within a processor. RTL serves as a blueprint for both the design and the functional understanding of the control logic and data manipulation performed by digital computers.

WHAT IS RTL?



RTL, or Register Transfer Language, is a low-level notation that is used to describe the operations that occur during the execution of instructions in a digital computer. These operations typically involve the transfer of data between registers, the performance of arithmetic or logical functions, and the control of timing and sequencing of these actions.



RTL is often used in the design and simulation of computer systems and is vital for defining the architecture and behavior of the datapath and control unit of the CPU. It allows computer engineers to specify precisely how data moves through the system during each clock cycle.

PURPOSE AND IMPORTANCE OF RTL

The primary purpose of RTL is to provide a clear and concise method to express the internal data transfers and micro-operations within a computer. It serves as a communication bridge between hardware designers and computer engineers by offering a systematic way to visualize and define how the components of a CPU interact.

THE IMPORTANCE OF RTL INCLUDES:

- **Design Verification:** RTL helps verify that each component and operation in the CPU behaves as expected.
- **Micro-operation Planning:** It outlines the exact steps involved in executing complex instructions by breaking them into fundamental operations.
- **Hardware Design:** It provides the foundation for designing digital logic circuits that form the CPU.
- **Control Logic Development:** It is instrumental in defining control signals that manage the sequence and timing of operations.

REGISTERS AND RTL NOTATION

Registers are small, fast memory elements located within the CPU, used to temporarily hold data and instructions during processing. Since memory access is comparatively slower, registers are used for immediate access and quick execution of operations.

COMMON REGISTERS INCLUDE:

R1, R2, R3, etc. – General-purpose registers



PC (Program Counter) – Holds the address of the next instruction

IR (Instruction Register) – Holds the current instruction being executed

MAR (Memory Address Register) – Holds memory addresses

MDR (Memory Data Register) – Holds data to be transferred to or from memory

RTL notation uses these register names along with operational symbols to describe what happens during data transfers. For example:

$R1 \leftarrow R2$: This means the contents of R2 are copied into R1.

$R1 \leftarrow R2 + R3$: The sum of R2 and R3 is calculated, and the result is stored in R1.

$R2 \leftarrow R2 + 1$: Increment operation.

$R1 \leftarrow 0$: Clear operation, resetting the register.

This symbolic language ensures clarity in the description of data manipulation and movement inside the CPU.

RTL IN MICRO-OPERATIONS

Micro-operations are the fundamental actions that occur inside a CPU to execute an instruction. Each instruction in a computer's instruction set is broken down into smaller steps called micro-operations. RTL helps in expressing these steps.

For example, the instruction ADD R1, R2 may involve the following RTL micro-operations:

Fetch: $IR \leftarrow M[PC]$, $PC \leftarrow PC + 1$

Decode: Decode the operation from IR

Execute: $R1 \leftarrow R1 + R2$

Here, the RTL notation captures the exact operations needed to execute the instruction at the hardware level.



RTL AND CONTROL DESIGN

Another major use of RTL is in control logic design. The control unit of a CPU needs to generate the appropriate signals to perform each micro-operation at the right time. Using RTL, designers can specify the sequence of operations and control signal requirements for each instruction.

For example, to execute a LOAD R1, M instruction, RTL might describe:

$MAR \leftarrow \text{Address}$

$MDR \leftarrow M[MAR]$

$R1 \leftarrow MDR$

This sequence defines how the control unit should behave and what signals it must generate to successfully complete the instruction.

RTL IMPLEMENTATION IN HARDWARE

In actual hardware, RTL operations are realized through logic gates, multiplexers, decoders, flip-flops, and other digital components. The logic design translates the symbolic RTL operations into combinational and sequential logic circuits.

FOR INSTANCE:

Transfer operations (e.g., $R1 \leftarrow R2$) are implemented using buses and tri-state buffers.

Arithmetic operations (e.g., $R1 \leftarrow R2 + R3$) are executed using arithmetic logic units (ALUs).

Control of sequencing is handled by finite state machines (FSMs) or microprogrammed control units.

RTL serves as the intermediate step between the abstract architecture and concrete hardware implementation.

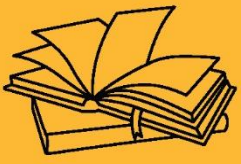


SUMMARY OF RTL

In summary, Register Transfer Language is essential for understanding and designing the internal operations of digital computers. It provides a symbolic method to define how data flows between registers, how it is processed through micro-operations, and how control logic sequences these actions.

KEY TAKEAWAYS

- RTL is used to describe the internal data movements in the CPU.
- It forms the foundation of micro-operation design and CPU instruction execution.
- Registers play a central role in RTL, acting as temporary data holders.
- RTL simplifies the design and analysis of control units and datapaths.
- It is a critical tool for translating instruction sets into executable hardware logic.
- By mastering RTL, one gains a deeper understanding of how computers execute instructions at the hardware level, which is essential for computer engineers, system designers, and anyone interested in low-level computing systems.



COMPUTER ORGANIZATION





SUB LESSON 13.3

MEMORY ORGANIZATION

MEMORY ORGANIZATION

Organization refers to the way data is stored, accessed, and managed within a computer system. It is a fundamental concept in computer organization because memory plays a crucial role in storing both instructions and data required for processing. The structure and behavior of memory determine how efficiently a computer performs tasks, which directly impacts overall system performance.

INTRODUCTION TO MEMORY IN COMPUTER SYSTEMS

In a computer, memory is essential for holding programs and data that the processor needs while executing instructions. Memory systems vary in speed, size, and function. Based on these characteristics, memory is organized into a hierarchical structure to optimize performance and cost.

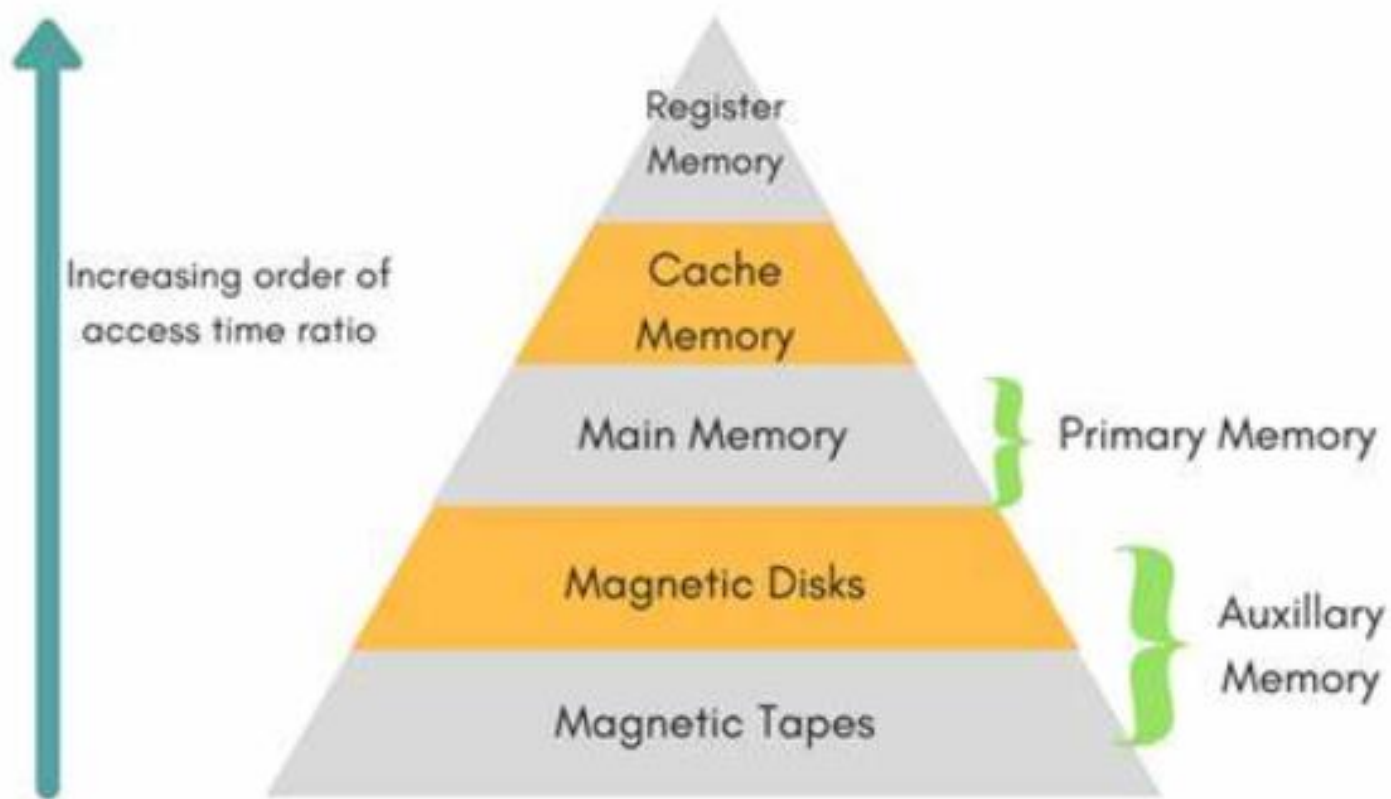
The study of memory organization involves understanding how data is stored in different types of memory, how these types interact, and how the system accesses and manages stored information. The three key components in this organization are:

Primary Memory – Directly accessible by the CPU.

Secondary Memory – Used for long-term data storage.

Cache Memory – A high-speed intermediary between primary memory and the processor.

HIERARCHICAL STRUCTURE OF MEMORY



Modern computer systems adopt a memory hierarchy to balance speed and cost. This hierarchy ranges from the fastest and smallest memory units close to the processor to the largest and slowest ones used for mass storage. The general structure is as follows:

Registers (inside the CPU) – Smallest and fastest; used for immediate data processing.

Cache Memory – Small, fast memory placed close to the CPU to store frequently accessed data.

Main Memory (RAM) – Medium-speed memory used for storing active programs and data.

Secondary Storage (HDDs, SSDs) – Large and slower; used for long-term data storage.

This layered approach ensures that the most frequently used data is available quickly, while less frequently used data resides in slower, but larger and cheaper storage.

MEMORY CELLS AND ADDRESSING

Memory in a computer is made up of millions (or billions) of memory cells arranged in a grid or matrix. Each cell is capable of storing one bit of data – either a 0 or a 1.

To make data accessible, every memory cell or group of cells is associated with a unique identifier called a memory address. These addresses are used by the processor to locate and access specific

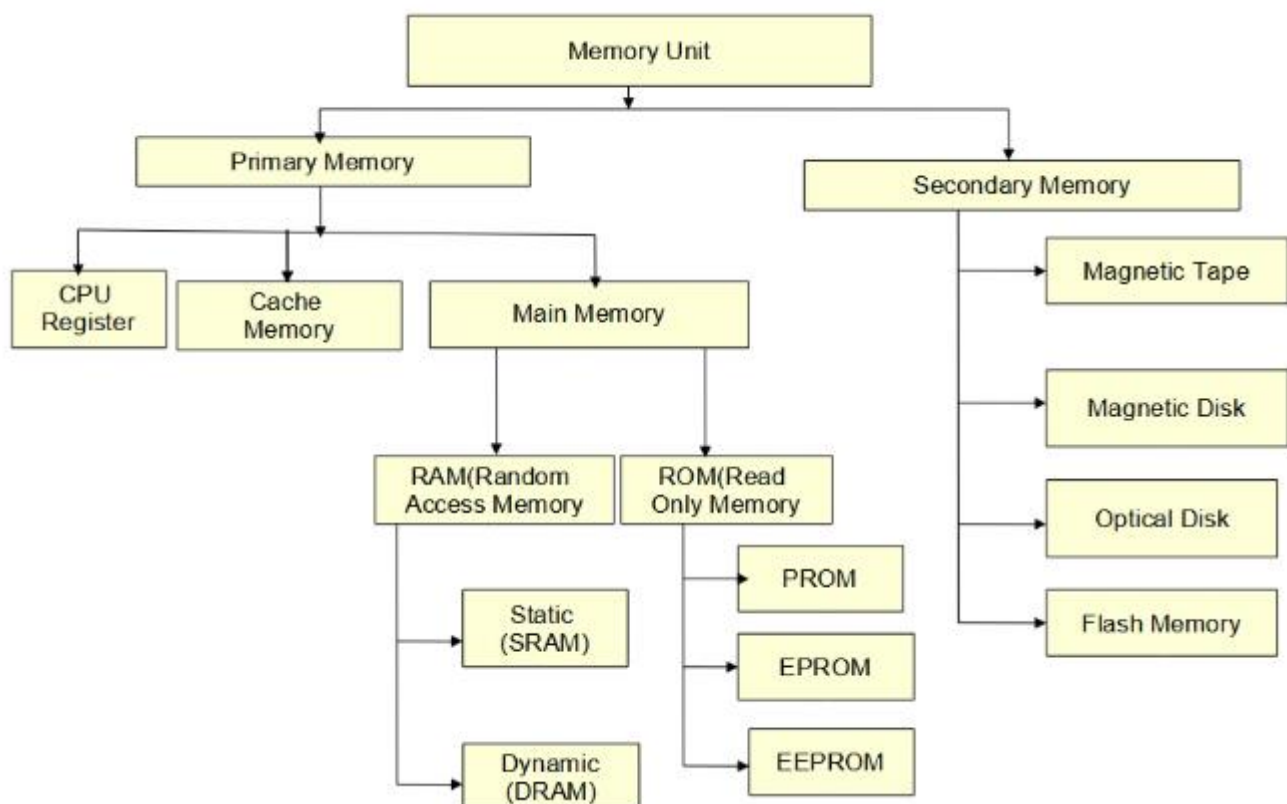
data during program execution. The process of accessing memory using these addresses is referred to as addressing.

TYPES OF ADDRESSING INCLUDE:

- Direct Addressing – The memory address is given explicitly.
- Indirect Addressing – The address is obtained from a register or another memory location.
- Indexed Addressing – A base address is combined with an index to access arrays or sequential data.

TYPES OF MEMORY

Memory in a computer system is broadly categorized into Primary and Secondary types. Each type plays a distinct role in the processing ecosystem.



PRIMARY MEMORY

Primary memory is directly accessible by the CPU. It includes:



Random Access Memory (RAM) – This is a volatile memory that temporarily stores data and instructions that the CPU needs while performing tasks. RAM allows both read and write operations and is cleared when the system is powered off.

READ-ONLY MEMORY (ROM) – This is non-volatile and stores critical programs such as the BIOS. Data in ROM is permanent and generally cannot be modified.

CACHE MEMORY – This is a small and fast type of volatile memory that stores frequently used data and instructions to speed up processing. It is located either on or close to the CPU chip.

Secondary Memory

Secondary memory stores data permanently and is not directly accessible by the CPU. It includes:

- Hard Disk Drives (HDDs) – Mechanical storage devices with large capacities.
- Solid-State Drives (SSDs) – Faster and more durable than HDDs, but typically more expensive.
- Optical Drives and USB Drives – Used for external storage and data transfer.

VOLATILE VS NON-VOLATILE MEMORY

Another important classification of memory is based on data retention:

VOLATILE MEMORY – Requires power to maintain stored information. Data is lost when the device is turned off. Examples: RAM and cache.

NON-VOLATILE MEMORY – Retains data even when power is off. Examples: ROM, SSDs, HDDs.

- The choice between volatile and non-volatile memory depends on the specific application. Volatile memory is suitable for temporary processing tasks, while non-volatile memory is used for permanent storage.
- Memory Access Time and Performance
- The speed at which data can be read from or written to memory is known as memory access time. Faster access times lead to better performance. In a memory hierarchy:
- Registers have the fastest access time.
- Cache memory is slower than registers but faster than RAM.
- RAM is significantly faster than secondary storage.
- HDDs and SSDs have the slowest access time but offer large storage capacities.



- Latency and bandwidth are two important performance metrics:
- Latency is the delay between a request for data and its delivery.
- Bandwidth is the amount of data that can be transferred in a given time.
- Optimizing these metrics is crucial in designing efficient memory systems.

MEMORY MANAGEMENT IN SYSTEMS

Efficient memory management ensures that the processor can access necessary data quickly and that memory is used effectively. This includes:

Memory Allocation – Assigning portions of memory to various programs.

Paging and Segmentation – Techniques used to manage large memory efficiently.

Virtual Memory – Uses secondary storage to simulate additional RAM, allowing larger applications to run smoothly even with limited physical memory.

Operating systems are responsible for managing memory dynamically, ensuring that processes do not interfere with each other and that memory is recycled as needed.

CONCLUSION

Memory Organization is a vital component of computer architecture and plays a central role in determining system performance, efficiency, and capability. Understanding the hierarchical structure of memory, various types of memory, addressing mechanisms, and how data is accessed and managed enables engineers to design faster and more efficient computing systems.



KEY TAKEAWAYS

- Memory is organized hierarchically to balance speed, size, and cost
- Each memory type—registers, cache, RAM, and secondary storage—has specific roles.
- Data access depends on unique addresses assigned to memory locations.
- Efficient memory management techniques are essential for multitasking and system stability.
- The distinction between volatile and non-volatile memory helps define use-cases and system behavior.

By mastering memory organization, students and professionals gain deeper insights into how computers operate internally and how to improve performance through architectural choices.



COMPUTER ORGANIZATION





SUB LESSON 13.4

BUS DESIGN USING MULTIPLEXER AND TRI-STATE BUFFER

BUS DESIGN USING MULTIPLEXER AND TRI-STATE BUFFER

In digital computer systems, the bus plays a central role in facilitating communication between the various internal components such as the CPU, memory, and input/output devices. Bus design is a critical aspect of computer organization because it affects system speed, complexity, and scalability. One of the most efficient ways to implement bus systems is through the use of multiplexers and tri-state buffers. These components help manage multiple data sources trying to access the same communication lines while maintaining signal integrity and reducing hardware complexity.

WHAT IS A BUS?

A bus is a collection of wires or conductors used to transmit data, addresses, and control signals between different components of a computer. It is a shared communication pathway, meaning multiple devices use the same set of lines to send or receive information. Because the bus must support data exchange between multiple units, it requires careful design to avoid data collisions and ensure efficient communication.

There are three primary types of buses in a computer system:

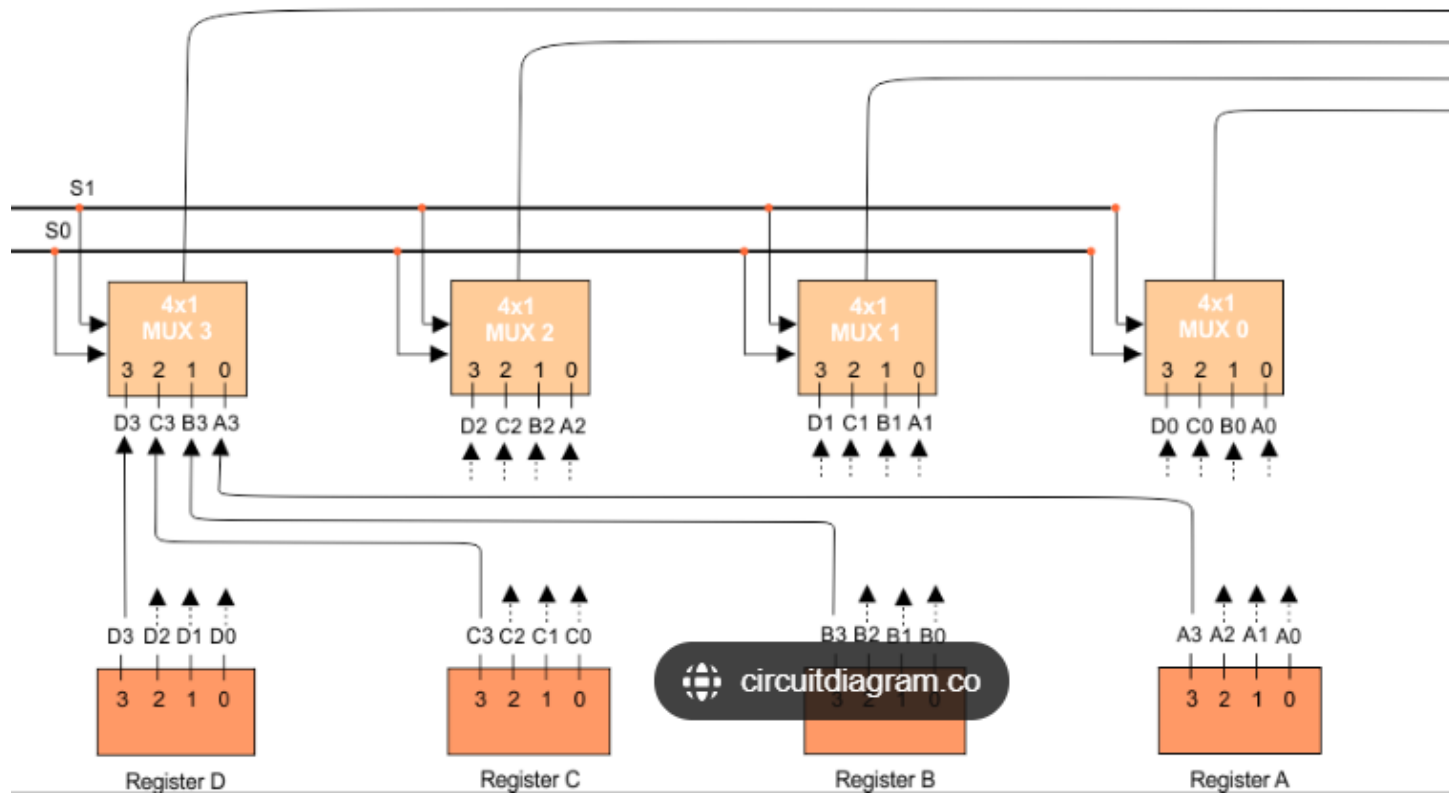
Data Bus: Carries the actual data being processed.

Address Bus: Carries the address of the memory or I/O device involved in the operation.

Control Bus: Carries control signals (e.g., read, write, enable) that manage operations.

Since the bus is shared, mechanisms are needed to control access. This is where bus arbitration, multiplexers, and tri-state buffers come into play.

BUS ARBITRATION



Bus arbitration is the process of managing access to the bus when multiple components request its use simultaneously. Without proper arbitration, multiple devices might attempt to communicate over the bus at the same time, causing signal conflict or data corruption.

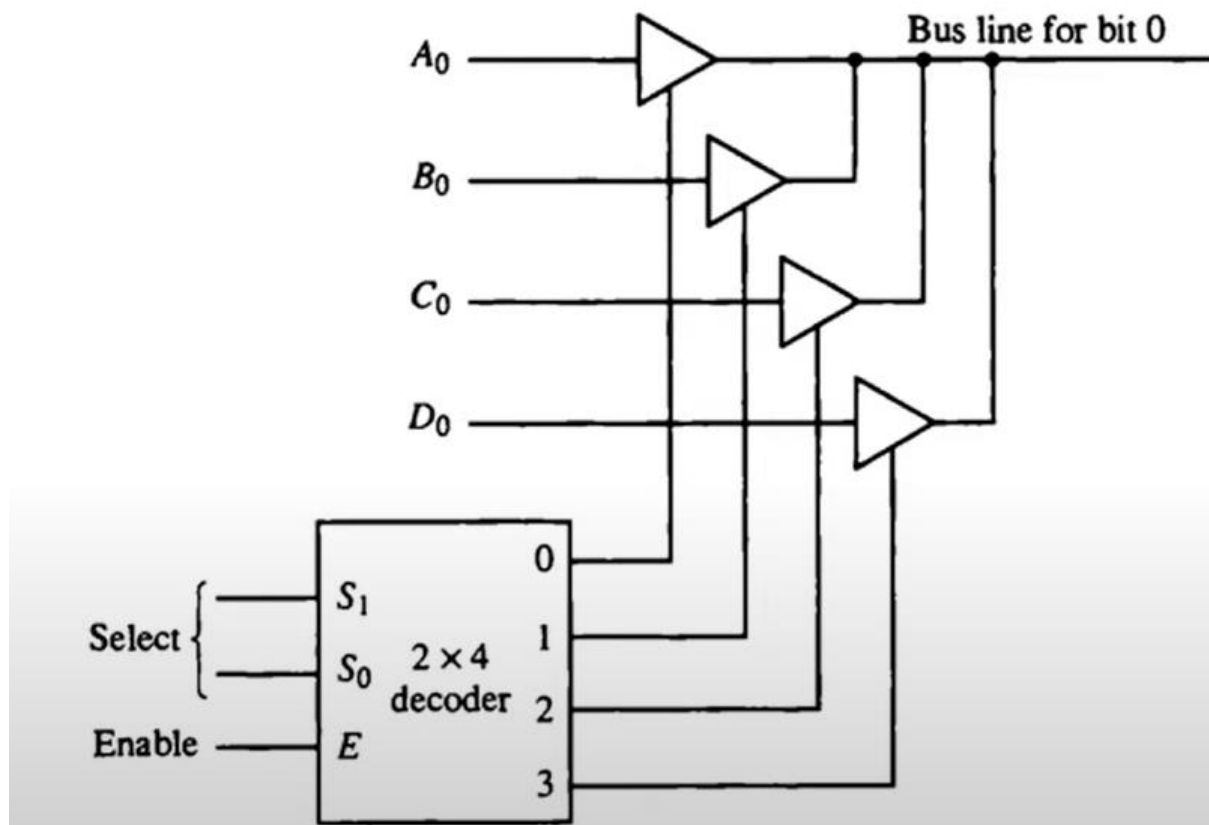
BUS ARBITRATION TECHNIQUES INCLUDE:

Centralized arbitration, where a single controller decides who gains bus access.

Distributed arbitration, where all devices participate in the decision-making process.

Although arbitration is essential for bus operation, the physical components like multiplexers and tri-state buffers are equally vital in the hardware design of the bus system.

MULTIPLEXERS IN BUS DESIGN



A multiplexer (MUX) is a digital switch that selects one of several input signals and forwards it to a single output line. In bus design, multiplexers allow multiple data sources to share a single line or set of lines without interference.

How It Works: A multiplexer takes multiple input lines (e.g., data from registers R1, R2, R3) and routes only one of them to the output based on the selection lines.

The selection lines determine which input is active at any given time.

For example, if a 4-to-1 multiplexer is used, four input lines (I0 to I3) can be controlled using two selection lines (S0 and S1). The output will reflect only the selected input.



ADVANTAGES IN BUS DESIGN

- Reduces the number of physical wires needed, minimizing hardware complexity.
- Allows controlled and conflict-free access to the bus from multiple sources.
- Enables time-shared communication on a single bus line.
- Multiplexers are commonly used in data routing, particularly in systems where simultaneous access to the bus is not required.

TRI-STATE BUFFERS IN BUS DESIGN

A tri-state buffer is a special kind of logic circuit that can exist in three states:

Logic High (1)

Logic Low (0)

High Impedance (Z) – effectively disconnects the output from the bus.

The high impedance state allows the output line of the buffer to be inactive, which means it neither drives the line high nor low. This property is essential for allowing multiple devices to share a single communication line without interference.

HOW IT WORKS

The tri-state buffer is controlled by an enable signal.

When the enable signal is active, the buffer passes the input signal to the output.

When disabled, the output enters a high-impedance state, effectively removing the device from the bus.



ADVANTAGES IN BUS DESIGN:

- Allows multiple devices to connect to a common bus without conflict.
- Devices can “take turns” using the bus by enabling/disabling their buffers.
- Simplifies bus arbitration by allowing hardware-level control of line access.
- Tri-state buffers are especially useful in open bus architectures, where multiple memory modules, processors, or I/O devices share the same data lines.

COMPARING MULTIPLEXER VS TRI-STATE BUFFER APPROACHES

Both multiplexers and tri-state buffers serve the same purpose—allowing multiple devices to share a bus—but they do so differently.

Feature	Multiplexer	Tri-State Buffer
Control	Selection lines	Enable signal
Output	One output from selected input	Any number of outputs, but only one active
Scalability	Adds complexity with more inputs	Easier to scale by adding buffers
Use Case	Controlled access from one source at a time	Multiple potential sources, one active at a time

Multiplexers are better suited for centralized, controlled access scenarios, while tri-state buffers are ideal for decentralized access with simple hardware logic.

Design Considerations



- When designing a bus system, engineers must take into account:
- Bus width: Number of bits that can be transferred at once.
- Bus speed: How quickly data is moved across the bus.
- Latency: Time delay between request and response.
- Bus contention: Situations where multiple devices attempt to use the bus simultaneously.
- Power consumption and physical layout: Important in embedded systems and mobile devices.
- Using multiplexers and tri-state buffers smartly can significantly reduce bus contention, simplify control logic, and improve overall system reliability and performance.

Bus design is an integral part of computer architecture that facilitates communication between different subsystems. Efficient design using multiplexers and tri-state buffers allows multiple components to interact without interference while minimizing complexity and maximizing performance.

Multiplexers simplify wiring by selecting one source out of many, while tri-state buffers enable multiple sources to share a common bus line safely. Together, these components form the backbone of a flexible and scalable communication infrastructure within digital systems.

Understanding how these elements function and how they are implemented in bus systems gives students and professionals the tools to build high-performance computing systems with optimized data flow and minimal hardware overhead.

KEY TAKEAWAYS

- Multiplexers select data from multiple sources.
- Tri-state buffers manage shared bus access.
- Prevents bus conflicts in complex systems.



COMPUTER ORGANIZATION





SUB LESSON 13.5

MEMORY TRANSFER

MEMORY TRANSFER

In computer systems, memory transfer refers to the process of moving data between the CPU (Central Processing Unit) and main memory. This is one of the most fundamental operations in computing, as it enables the execution of programs by allowing the processor to fetch instructions and manipulate data stored in memory. Without memory transfer mechanisms, the CPU would be isolated from data storage, rendering it ineffective for most computational tasks. This chapter explores how memory transfers are performed, the key components involved, the control signals that govern these operations, and specific memory transfer instructions used in microprocessor programming.

KEY COMPONENTS INVOLVED IN MEMORY TRANSFER

- Two critical hardware elements are used for memory transfers:
- Memory Address Register (MAR): This register holds the address of the memory location that is to be accessed, whether for a read or a write operation. It points to the exact location in memory where data is to be fetched from or stored.
- Memory Data Register (MDR): Also known as the Memory Buffer Register (MBR), this register temporarily holds the data being transferred to or from memory. It acts as a bridge between the memory and the CPU's internal registers.
- In a typical memory operation:
 - The MAR specifies where the data is located.
 - The MDR contains the actual data to be written or fetched.
 - These two registers, along with the control unit and the system buses, work together to ensure smooth data movement between memory and the processor



READ AND WRITE OPERATIONS

Memory transfer can occur in two directions:

Read Operation (CPU $\leftarrow$ Memory):

The CPU fetches data from a specific memory location.

The address of the desired memory cell is loaded into the MAR.

A control signal is issued to perform the read operation.

The data at that address is then loaded into the MDR.

Finally, the data is transferred from the MDR to the CPU register.

Write Operation (Memory $\leftarrow$ CPU):

The CPU sends data to be stored in memory.

The target memory address is loaded into the MAR.

The data to be written is placed in the MDR.

A write control signal is sent to initiate the transfer.

The data is then written from the MDR into the specified memory location.

Control signals like Read and Write are essential for determining the direction of the data flow and initiating the correct operation.

MEMORY TRANSFER INSTRUCTIONS IN MICROPROCESSORS

Memory transfer operations are implemented through instructions that are part of a microprocessor's instruction set. In the 8085 microprocessor, several instructions are designed specifically for transferring data between registers and memory.

Here are some commonly used memory transfer instructions:

MOV: Used to transfer data between registers or between a register and a memory location.

Example: MOV A, M – Transfers data from the memory location addressed by the HL pair to the accumulator A.

MVI: Move immediate instruction loads a register or memory location with an immediate value (constant).

Example: MVI B, 32H – Loads the hexadecimal value 32 directly into register B.



LDA: Load Accumulator Direct – Loads the accumulator with the content of a specific memory address.

Example: LDA 2500H – Loads the accumulator with the contents of memory location 2500H.

STA: Store Accumulator Direct – Stores the content of the accumulator into a specific memory address.

Example: STA 2600H – Stores the content of the accumulator into memory location 2600H.

These instructions are essential for loading data to be processed and storing results after computation. They provide the necessary control for moving data across different levels of memory hierarchy.

ROLE IN PROGRAMS

Memory transfer instructions are among the most frequently used commands in microprocessor programming. They form the foundation for:

Data initialization: Loading variables or constants into registers for computation.

Operand movement: Bringing data from memory to registers for arithmetic or logic operations.

Result storage: Writing processed results back to memory.

Program flow control: Fetching instruction codes and addresses from memory.

They enable flexibility, data management, and modularity in software design. Whether for simple arithmetic tasks or complex algorithm execution, efficient use of memory transfer instructions ensures optimal program performance and resource utilization.

Conclusion

Memory transfer is a fundamental aspect of computer operation, enabling the CPU to interact with memory efficiently. By using the MAR and MDR, the processor can accurately locate and move data. Read and write operations, controlled by specific signals, form the basis for data flow in any computing system.

In microprocessor programming, especially in the 8085, memory transfer instructions like MOV, MVI, LDA, and STA play a crucial role in executing meaningful tasks. Mastering these instructions and understanding the internal mechanisms of memory transfer is essential for anyone studying computer architecture or low-level programming.

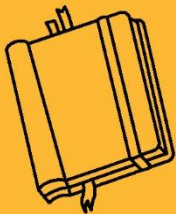


KEY TAKEAWAYS

- Involves reading/writing data between memory and CPU.
- Controlled by read/write signals and address bus.
- Efficient transfer is crucial for performance.

OUTCOMES CHAPTER 13 (COMPUTER ORGANIZATION)

- Understand the Structure and Operation of a Computer System
- Gain knowledge of the fundamental components such as CPU, memory, input/output units, and control units, along with their interactions.
- Apply Register Transfer Language (RTL) for Internal Operations
- Learn how to describe and analyze data flow and micro-operations within the CPU using RTL notation.
- Comprehend Memory Hierarchy and Organization
- Understand various memory types (RAM, ROM, cache, secondary storage), their characteristics, and how memory is addressed and managed.



COMPUTER ORGANIZATION





SUB LESSON 14.1

ARITHMATIC MICRO OPERATION

OBJECTIVES OF CHAPTER 14 (MICRO OPERATION)

- To introduce the concept of micro-operations and classify them into arithmetic, logical, and shift categories within digital systems.
- To explain the execution of arithmetic micro-operations such as addition, subtraction, increment, and decrement using registers.
- To describe logical and shift micro-operations, including bitwise manipulation and various shift techniques like logical, arithmetic, and circular shifts.
- To understand the structure and functioning of the Arithmetic Logic Unit (ALU) and its role in performing combined micro-operations under the control of the CPU.

ARITHMATIC MICRO OPERATION

In digital computer systems, operations are performed at multiple levels of abstraction. At the most fundamental level within the processor, operations are carried out using micro-operations. These are simple, atomic operations executed directly on the data stored in registers. Micro-operations are essential for the execution of more complex instructions and serve as the basic building blocks of processor functionality.

This chapter focuses specifically on arithmetic micro-operations, which are operations related to arithmetic calculations such as addition, subtraction, increment, and decrement. These operations are carried out inside the Arithmetic Logic Unit (ALU), which is the part of the CPU responsible for performing mathematical and logical operations.

INTRODUCTION TO MICRO-OPERATIONS

A micro-operation is a low-level operation performed on the contents of one or more registers. These are not instructions by themselves but rather the sub-steps of executing a machine-level



instruction. Micro-operations operate on the smallest units of data storage — registers — and modify their contents.

There are various types of micro-operations, including:

Arithmetic Micro-Operations – Perform basic arithmetic functions.

Logic Micro-Operations – Carry out bitwise logic functions like AND, OR, XOR.

Shift Micro-Operations – Shift the bits of a register left or right.

Control Micro-Operations – Manage the control signals and sequencing within the processor.

Among these, arithmetic micro-operations form the foundation of most computational tasks.

WHAT ARE ARITHMETIC MICRO-OPERATIONS?

Arithmetic micro-operations are micro-level operations that perform arithmetic calculations on numerical data stored in registers. These are executed within the Arithmetic Logic Unit (ALU) and involve operands fetched from CPU registers. The result of an arithmetic micro-operation is usually stored in another register.

Typical arithmetic micro-operations include:

- Addition
- Subtraction
- Increment
- Decrement
- Negation (Unary minus, i.e., two's complement)
- Transfer (Simple copying, sometimes considered part of arithmetic group)

These operations are vital to carrying out more complex program instructions like looping, counting, and numerical calculations.

ROLE OF ALU IN ARITHMETIC OPERATIONS

The Arithmetic Logic Unit (ALU) is a digital circuit that performs arithmetic and logical operations. It is a core component of the CPU and interacts with the registers and control unit to perform micro-operations.

For arithmetic operations:



The ALU takes inputs from two registers.

It performs the specified operation (e.g., addition).

The result is stored in another register or fed back into the ALU for further processing.

Control signals direct the ALU to perform a specific operation, and a multiplexer is often used to select the correct function within the ALU.

COMMON ARITHMETIC MICRO-OPERATIONS AND THEIR FUNCTIONS

Let's explore the most common arithmetic micro-operations in more detail:

ADDITION

Addition is the most frequently used arithmetic operation and forms the basis for many higher-level functions such as multiplication, accumulation, and indexing.

Example:

$$R3 \leftarrow R1 + R2$$

In this operation, the contents of registers R1 and R2 are added, and the result is stored in register R3. This requires a binary adder circuit within the ALU.

Addition must also consider carry conditions when the result exceeds the maximum bit width, and may trigger a carry flag.

SUBTRACTION

Subtraction is typically implemented using addition and two's complement representation.

Instead of direct subtraction, the ALU adds the complement of one operand to the other.

Example:

$$R3 \leftarrow R1 - R2$$

This is implemented as:

scss

CopyEdit

$$R3 \leftarrow R1 + (2's \text{ complement of } R2)$$

The ALU may include a subtractor circuit or perform subtraction using adder logic and complement techniques.



INCREMENT

The increment operation increases the content of a register by one. It is useful in loops, counters, and pointer manipulations.

Example:

$$R1 \leftarrow R1 + 1$$

A binary counter or an adder with a hardwired input of '1' is used for this operation. It is simple and fast as no operand needs to be fetched from another register.

DECREMENT

The decrement operation decreases the content of a register by one. It is the counterpart to increment and is also widely used in looping and counting mechanisms.

Example:

$$R1 \leftarrow R1 - 1$$

This operation typically uses the two's complement method to perform the subtraction internally within the ALU.

NEGATION

Negation changes the sign of a number. For signed binary numbers, negation is done using two's complement representation.

Example:

$$R1 \leftarrow -R1$$

This operation computes the two's complement of the register contents, flipping all bits and adding one.

CLEAR OR TRANSFER

Although not arithmetic in nature, operations like:

$$R1 \leftarrow 0$$

$$R1 \leftarrow R2$$

are essential for initializing values or transferring them across registers. These often accompany arithmetic functions in instruction sequences.



Example Use in Programs

Arithmetic micro-operations are used frequently in processor instructions to support:

Counting loops (using increment/decrement)

Array processing (using index register addition)

Conditional operations (e.g., subtracting to compare values)

Accumulator updates (e.g., in summing algorithms)

Mathematical computations (addition, subtraction chains)

In a microprogrammed control unit, each high-level instruction like ADD, SUB, or INR is broken into a series of micro-operations.

HARDWARE IMPLEMENTATION OF ARITHMETIC MICRO-OPERATIONS

To implement these operations in hardware, digital circuits such as adders, subtractors, counters, and multiplexers are used.

Adders: Perform both addition and subtraction using two's complement.

Incrementers/Decrementers: Use cascaded logic gates or modified adders.

ALU Multiplexers: Select which arithmetic operation to execute based on control input.

Control signals determine which operation is selected, and the data path within the processor directs the operand flow and result storage.

SIGNIFICANCE IN DIGITAL SYSTEM DESIGN

Arithmetic micro-operations form the basis for executing almost all meaningful computational tasks. Whether it is numerical processing, data management, or control decision-making, arithmetic operations are involved at some level.

They also influence:

Instruction set design: Every instruction set must support basic arithmetic.

Pipelining: Arithmetic units are optimized in modern CPUs for high-speed parallel operations.

ALU optimization: Performance of a processor is often gauged by how efficiently it handles arithmetic operations.



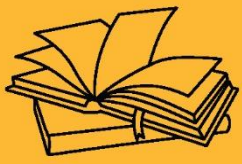
CONCLUSION

Arithmetic micro-operations are fundamental operations in digital computing systems. They enable the processor to perform essential tasks such as addition, subtraction, incrementing, and decrementing values stored in registers. These operations are executed within the Arithmetic Logic Unit and are critical for nearly every computing application.

By mastering arithmetic micro-operations, one gains a clearer understanding of how CPUs process instructions at the lowest level, making it easier to understand and design efficient programs, processors, and digital systems.

KEY TAKEAWAYS

- Includes add, subtract, increment, and decrement.
- Performed in ALU using control logic.
- Fundamental to computation tasks.



COMPUTER ORGANIZATION



SUB LESSON 14.2

LOGICAL MICRO OPERATION

LOGICAL MICRO OPERATION

In the architecture of digital systems, micro-operations are fundamental low-level actions that manipulate data stored in registers. These micro-operations form the basis of instruction execution inside the CPU. Among the various categories of micro-operations—arithmetic, logic, and shift—logical micro-operations play a vital role in bitwise data manipulation and control processing.

Logical micro-operations allow direct manipulation of individual bits within a register. They are used extensively for modifying data, testing conditions, setting or clearing flags, and simplifying control logic in processors. This chapter explores the types, syntax, hardware behavior, and applications of logical micro-operations in digital systems.

INTRODUCTION TO LOGICAL MICRO-OPERATIONS

Logical micro-operations are performed directly on the binary representation of data stored in registers. Unlike arithmetic operations, which involve numerical calculations, logical operations manipulate data at the bit level. They apply Boolean algebra rules to execute operations such as:

- AND
- OR
- XOR (exclusive OR)
- NOT (complement)

These operations are extremely fast and are executed using logic gates within the Arithmetic Logic Unit (ALU). They are crucial for tasks like masking data, toggling bits, implementing control flow decisions, and simplifying comparisons between data.



TYPES OF LOGICAL MICRO-OPERATIONS

Let's break down the primary logical operations used in micro-operations, including their syntax and function:

AND OPERATION

SYNTAX:

$R1 \leftarrow R1 \text{ AND } R2$

In this operation, each bit of register R1 is ANDed with the corresponding bit of register R2. The result is stored in R1. The output bit is 1 only if both input bits are 1; otherwise, it is 0.

EXAMPLE:

If $R1 = 1101$ and $R2 = 1011$, then:

$R1 \text{ AND } R2 = 1001$

USE CASES:

Masking bits (e.g., keeping only selected bits of data)

Clearing specific bits

Checking bit status

OR OPERATION

SYNTAX:

$R1 \leftarrow R1 \text{ OR } R2$

Here, a bitwise OR is performed between R1 and R2. The result is 1 if at least one of the corresponding bits is 1.

EXAMPLE:



If $R1 = 1100$ and $R2 = 1010$, then:

$R1 \text{ OR } R2 = 1110$

USE CASES:

Setting specific bits

Combining bit patterns

Ensuring at least one condition is true

XOR OPERATION (EXCLUSIVE OR)

SYNTAX:

$R1 \leftarrow R1 \text{ XOR } R2$

XOR outputs 1 if the bits are different and 0 if they are the same. It is often used for comparing two binary values or toggling specific bits.

EXAMPLE:

If $R1 = 1100$ and $R2 = 1010$, then:

$R1 \text{ XOR } R2 = 0110$

USE CASES:

Comparing two values

Bitwise inversion or toggling

Data scrambling in encryption

NOT OPERATION (COMPLEMENT)

SYNTAX:



$R1 \leftarrow \text{NOT } R1$

The NOT operation inverts each bit in the register R1. 0 becomes 1, and 1 becomes 0.

EXAMPLE:

If $R1 = 1100$, then:

$\text{NOT } R1 = 0011$

USE CASES:

- One's complement operations
- Generating inverse masks
- Control logic inversion

APPLICATIONS OF LOGICAL MICRO-OPERATIONS

Logical operations are widely used in system-level programming and digital design. Their ability to manipulate data at the bit level enables numerous critical functionalities:

1. BIT MASKING

Masking involves using a logical AND operation with a specific bit pattern to isolate or clear certain bits. For example, to extract only the lower four bits of a byte, a mask of 00001111 can be used.

2. SETTING AND CLEARING BITS

To set a specific bit, an OR operation with a bit mask is used. To clear a bit, an AND operation with the complement of a bit mask is performed.

Example:

Set bit 2: $R \leftarrow R \text{ OR } 00000100$



Clear bit 2: $R \leftarrow R \text{ AND } 11111011$

3. BITWISE COMPARISONS

Using XOR, differences between two data values can be detected. If the result is 0, the two operands are identical.

4. CONTROL FLAGS AND CONDITION CHECKING

Logical micro-operations are used to check and manipulate flags (such as zero, carry, or overflow flags) that control the execution flow in CPUs.

5. EFFICIENT DATA MANIPULATION

Logical operations enable fast and efficient modification of data patterns, making them invaluable in encryption, data compression, graphics processing, and more.

HARDWARE IMPLEMENTATION

Logical micro-operations are implemented using basic digital logic gates:

AND gates for AND operation

OR gates for OR operation

XOR gates for XOR operation

Inverters for NOT operation

These gates are part of the ALU, and control signals determine which operation is executed. A multiplexer is usually used to select between various logic operations based on control input from the instruction decoder or control unit.

IMPORTANCE IN INSTRUCTION EXECUTION

- Logical micro-operations are often part of complex instruction execution. For instance:



- In conditional instructions (if, switch), bits are tested using AND/OR.
- In jump or branch instructions, logical conditions are evaluated before altering program flow.
- In operating system kernels and embedded systems, logical micro-operations enable fine-grained hardware control.
- These operations are foundational not only in processor logic but also in software systems that interact with hardware at a low level.

CONCLUSION

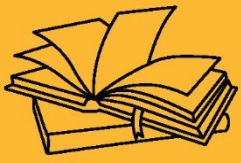
Logical micro-operations are an essential part of digital system design and microprocessor operation. By performing bitwise operations such as AND, OR, XOR, and NOT, these micro-operations offer powerful tools for data manipulation, condition checking, and control signal handling.

Their implementation within the ALU enables fast and efficient processing of logical instructions, and their applications extend across virtually every level of computer system design — from microarchitecture to operating system kernels and embedded control systems.

By understanding and utilizing logical micro-operations, engineers and students alike can gain deeper insight into how digital systems function at their core.

KEY TAKEAWAYS

- Includes AND, OR, NOT, XOR operations.
- Used for decision-making and bit manipulation.
- Support masking, setting, and clearing bits.



COMPUTER ORGANIZATION





SUB LESSON 14.3

SHIFT MICRO OPERATION

SHIFT MICRO OPERATION

In digital systems, micro-operations are basic actions performed on data stored in registers. These micro-operations are categorized into arithmetic, logical, and shift types. Shift micro-operations are used to move bits within a register to the left or right, playing a crucial role in data manipulation, arithmetic computations, and optimization techniques in computer architecture. Shift micro-operations do not change the actual value significantly in most cases but alter the bit positions of data within a register. Depending on the type of shift, they may affect the carry flag or preserve the sign bit, making them critical for both unsigned and signed data handling.

INTRODUCTION TO SHIFT MICRO-OPERATIONS

Shift micro-operations are used to move bits within a register either to the left or to the right. These operations are particularly useful in arithmetic calculations (especially multiplication and division by powers of 2), bit manipulation, cryptographic routines, and efficient data processing. There are three main types of shift operations:

- Logical Shift
- Arithmetic Shift
- Circular Shift (Rotate)

Each type of shift affects the data differently, depending on the intent and the nature of the bits (signed or unsigned).

LOGICAL SHIFT

A logical shift moves bits left or right and fills the vacated position with a zero. It is typically used for unsigned binary numbers.

Logical Shift Left (LSL): All bits are shifted one position to the left. A 0 is inserted into the least significant bit (LSB), and the most significant bit (MSB) is discarded.



Logical Shift Right (LSR): All bits are shifted one position to the right. A 0 is inserted into the MSB, and the LSB is discarded.

Example:

Consider an 8-bit register with contents: 10110010

Logical Shift Left: 01100100 (MSB '1' is dropped, '0' added to LSB)

Logical Shift Right: 01011001 (LSB '0' is dropped, '0' added to MSB)

APPLICATIONS:

Multiplication/division of unsigned integers by 2

Binary data alignment

Fast binary calculations

ARITHMETIC SHIFT

An arithmetic shift is similar to a logical shift but takes into account the sign of the binary number, which is essential in signed binary number representations, such as two's complement.

Arithmetic Shift Left: Behaves like logical shift left (fills LSB with 0 and discards MSB).

Arithmetic Shift Right: Retains the sign bit (MSB) during the shift to preserve the number's sign.

The LSB is discarded.

Example:

For signed 8-bit binary number 11010010 (which is negative in two's complement):

Arithmetic Shift Right: 11101001 (MSB '1' retained to preserve negative sign)

This helps preserve the integrity of signed values during right shifts.

Applications:

Multiplication/division of signed integers by powers of 2

Sign-preserving shifts

Arithmetic computations in fixed-point arithmetic

CIRCULAR SHIFT (ROTATE)

Circular shifts, also known as rotations, move bits around the register in a circular manner, with no bits lost. Bits shifted out from one end are inserted back at the other end.

Rotate Left (ROL): Bits are shifted left, and the MSB is placed in the LSB.



Rotate Right (ROR): Bits are shifted right, and the LSB is placed in the MSB.

Example:

For 10110010:

Rotate Left: 01100101 (MSB '1' is wrapped to LSB)

Rotate Right: 01011001 (LSB '0' is wrapped to MSB)

Applications:

Used in cryptographic algorithms for data scrambling

Bitwise operations in encoding and decoding routines

Efficient circular queue implementations

Hardware logic optimizations

SUMMARY OF SHIFT OPERATIONS

Type	Shift Direction	Bit Inserted	Preserves Sign	Data Loss
Logical Shift	Left/Right	0	No	Yes
Arithmetic Shift	Left	0	No	Yes
Arithmetic Shift	Right	Sign Bit	Yes	Yes
Circular Shift	Left/Right	Rotated Bit	Not Applicable	No

HARDWARE IMPLEMENTATION

In hardware, shift operations are performed using shifting circuits, multiplexers, or barrel shifters. These circuits are usually a part of the ALU (Arithmetic Logic Unit) or separate shift registers.

Shift Registers: Designed specifically to move data left or right, bit by bit.

Barrel Shifters: Allow multi-bit shifts in a single clock cycle, improving performance.

Control logic selects the type and direction of shift based on the instruction. In microprocessor instructions, these are typically invoked using shift mnemonics such as SHL, SHR, SAR, ROL, and ROR.



EFFECTS ON PROCESSOR FLAGS

Shift operations often impact the flags in the status register of a processor:

Carry Flag (CF): Set if a bit is shifted out from the register.

Zero Flag (ZF): Set if the result after the shift is zero.

Sign Flag (SF): Reflects the MSB (important in arithmetic shift).

Overflow Flag (OF): May be affected in some shift instructions based on sign changes.

These flags are important for conditional branching and further operations in assembly-level programming.

REAL-WORLD APPLICATIONS

Shift micro-operations are widely used in:

Embedded Systems: Efficient bit manipulation for device control.

Cryptography: Rotation is used in encryption algorithms like AES and DES.

Graphics Processing: Pixel manipulation and color component extraction.

Data Compression: Bit-level packing and unpacking operations.

Signal Processing: Fast arithmetic operations using shifts instead of multipliers.

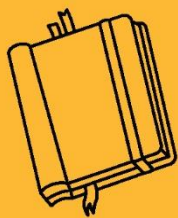
CONCLUSION

Shift micro-operations are an essential part of register-level data manipulation in digital systems. By moving bits left or right within a register, these operations enable a variety of functions including arithmetic scaling, bit-level masking, encryption, and system control.

Understanding the differences between logical, arithmetic, and circular shifts is crucial for anyone involved in system programming, digital circuit design, or computer architecture. These operations enhance performance, simplify instruction sets, and enable highly optimized computational solutions at the hardware level.

KEY TAKEAWAYS

- Shift operations move bits left/right.
- Used in multiplication/division and data manipulation.
- Logical and arithmetic shifts differ in sign bit handling.



COMPUTER ORGANIZATION



SUB LESSON 14.4

ARITHMETIC LOGIC SHIFT UNIT

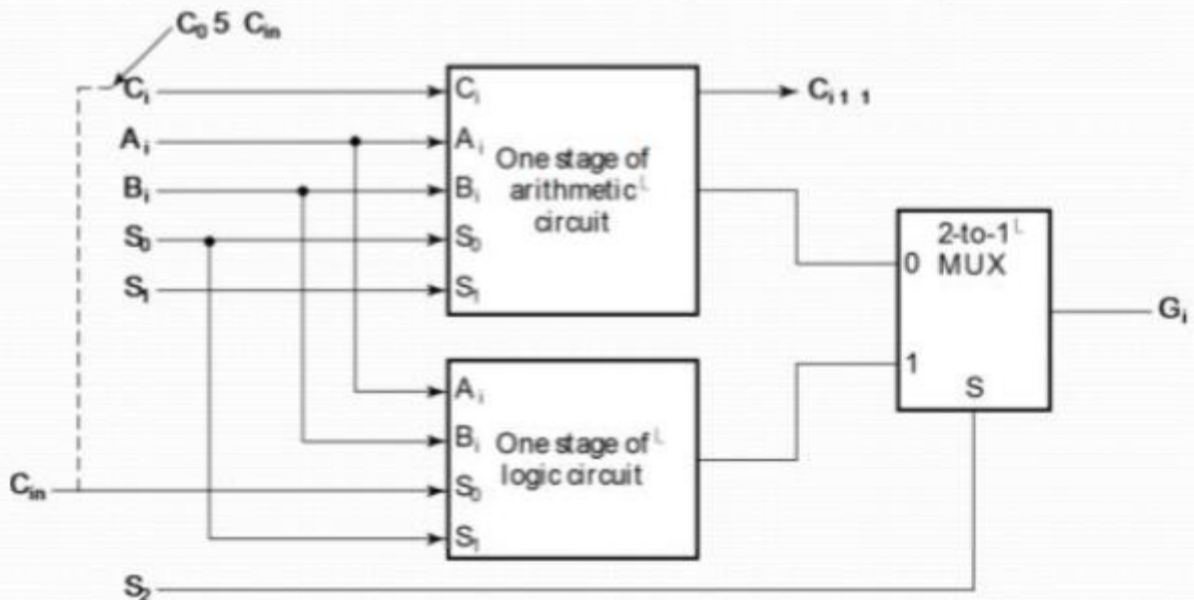
ARITHMETIC LOGIC SHIFT UNIT

The Arithmetic Logic Unit (ALU) is one of the most critical components within a computer's Central Processing Unit (CPU). It is the part of the processor that performs all arithmetic and logic operations. Without the ALU, a computer would not be able to process mathematical calculations or make logical decisions. This chapter delves into the functions, design, control mechanisms, and significance of the ALU in executing micro-operations.

WHAT IS THE ALU?

The Arithmetic/Logic Unit

- ALU performs arithmetic/logic micro-operations.



The Arithmetic Logic Unit is a combinational digital circuit that executes arithmetic operations like addition, subtraction, increment, decrement, and logical operations such as AND, OR, NOT,



and XOR. It processes data inputs from CPU registers, performs the required operation, and sends the result back to a register or memory location.

The ALU receives instructions from the control unit, which sends control signals to determine the type of operation that should be performed. Thus, the ALU acts on data but does not decide what to do with it on its own—it operates under the direction of the control unit.

Operation Select				Operation	Function
S_2	S_1	S_0	C_{in}		
0	0	0	0	$G=A$	Transfer A
0	0	0	1	$G=A+1$	Increment A
0	0	1	0	$G=A+B$	Addition
0	0	1	1	$G=A+B+1$	Add with carry input of 1
0	1	0	0	$G=A+\overline{B}$	A plus 1s complement of B
0	1	0	1	$G=A+\overline{B}+1$	Subtraction
0	1	1	0	$G=A-1$	Decrement A
0	1	1	1	$G=A$	Transfer A
1	X	0	0	$G=A \wedge B$	AND
1	X	0	1	$G=A \vee B$	OR
1	X	1	0	$G=A \text{ xor } B$	XOR
1	X	1	1	$G=\overline{A}$	NOT (1s complement)

ROLE OF THE ALU IN A COMPUTER SYSTEM

The ALU serves as the execution core of the CPU. It supports various types of micro-operations, such as:

Arithmetic Micro-Operations: Addition, subtraction, increment, decrement, and sometimes multiplication and division.

Logical Micro-Operations: Bitwise operations like AND, OR, XOR, and NOT.

Shift Micro-Operations: Logical or arithmetic shift left and right.



Comparison Operations: Greater than, less than, or equal to, often using subtraction and flag analysis.

Every computational instruction in a program is ultimately broken down into one or more micro-operations handled by the ALU.

COMPONENTS OF THE ALU

An ALU is composed of several smaller logic circuits, each designed to perform specific operations. These components typically include:

- **Adder/Subtractor:** A circuit that performs binary addition and subtraction using logic gates and two's complement arithmetic.
- **Logic Gates:** Implements bitwise operations like AND, OR, XOR, and NOT.
- **Multiplexers:** Used to select between multiple operation results based on control inputs.
- **Shift Units:** Handle shift operations like logical and arithmetic left or right.
- **Flags/Status Bits:** Indicates the result of the operation (e.g., zero, carry, overflow, sign).

Each operation path in the ALU is connected to a multiplexer so the correct result can be routed to the output, depending on the control signal.

SELECT INPUTS AND CONTROL SIGNALS

The operation performed by the ALU is determined by select inputs. These inputs are a set of binary control lines sent from the control unit. For example, a 3-bit select input can determine up to 8 different operations (e.g., 000 = ADD, 001 = SUB, 010 = AND, etc.).

The control unit orchestrates this process by:

- Decoding the current instruction
- Generating appropriate control signals
- Routing data from registers to the ALU inputs
- Enabling the desired operation through select inputs
- Storing the ALU output back into a register or memory

Control signals also manage auxiliary operations such as enabling data buses, triggering status flags, or synchronizing with the system clock.



FUNCTION TABLE FOR ALU OPERATIONS

To understand how the ALU operates, it's helpful to visualize a function table, which maps select inputs to the specific operation. Here's a simplified example:

Select Input (S2 S1 S0)	Operation	Description
000	ADD	$R \leftarrow A + B$
001	SUB	$R \leftarrow A - B$
010	AND	$R \leftarrow A \text{ AND } B$
011	OR	$R \leftarrow A \text{ OR } B$
100	XOR	$R \leftarrow A \text{ XOR } B$
101	NOT	$R \leftarrow \text{NOT } A$
110	Shift Left	$R \leftarrow A \ll 1$
111	Shift Right	$R \leftarrow A \gg 1$

This function table is implemented using logic gates and multiplexers. It helps the control unit route the correct output to the destination register.

ARITHMETIC LOGIC SHIFT UNIT

In many processors, the ALU is extended to become an Arithmetic Logic Shift Unit, which integrates arithmetic, logical, and shift operations in a single functional block. This integrated design:

- Reduces hardware complexity

- Enhances performance by avoiding separate functional units

- Simplifies control unit design

The use of multiplexers plays a key role in selecting the correct output from multiple operation paths within the ALU. These selection mechanisms allow dynamic control based on the instruction currently being executed.

ALU FLAGS AND STATUS BITS

After executing an operation, the ALU often updates several status flags in the processor's status register. These include:



- Zero Flag (Z): Set if the result is zero.
- Carry Flag (C): Set if there's a carry out from the MSB (in addition) or borrow (in subtraction).
- Overflow Flag (V): Indicates signed overflow.
- Sign Flag (S): Reflects the sign of the result (0 for positive, 1 for negative).
- Parity Flag (P): Indicates whether the number of set bits in the result is even or odd.

These flags are essential for conditional instructions, such as IF, JUMP, or CALL, enabling decision-making based on previous operations.

DESIGN CONSIDERATIONS

Designing an ALU requires attention to several factors:

Bit Width: ALUs are designed based on the CPU word size (e.g., 8-bit, 16-bit, 32-bit, 64-bit).

Speed: Must execute operations in as few clock cycles as possible.

Complexity: Multiplexers simplify the design but may increase propagation delay.

Power Efficiency: Especially important in embedded and mobile processors.

Scalability: ALU design should accommodate instruction set expansion.

Modern processors may also include dedicated hardware for complex operations such as floating-point arithmetic, which are handled by the Floating Point Unit (FPU).

APPLICATIONS OF THE ALU

The ALU plays a role in virtually every digital device that processes data. Common applications include:

Microprocessors and Microcontrollers: All arithmetic and logical computations.

Digital Signal Processors (DSPs): Fast arithmetic operations on audio, video, and sensor data.

Embedded Systems: Efficient control and decision-making tasks.

Supercomputers: High-performance computing using advanced ALU arrays for parallel computation.

In all these cases, the ALU is essential for enabling computational logic and processing efficiency.



CONCLUSION

The Arithmetic Logic Unit (ALU) is the computational heart of any processor. It performs the core operations that allow a computer to process data, make decisions, and control external devices. By combining arithmetic, logical, and shift operations into a unified architecture, the ALU enables efficient execution of instructions.

Its design—relying on multiplexers, control signals, and logic circuits—provides flexibility and scalability while maintaining performance. The ALU's close integration with the control unit and memory ensures that every instruction in a program is effectively translated into executable actions within the CPU.

A deep understanding of the ALU's functioning, structure, and interaction with the rest of the computer system is essential for students, hardware designers, and computer engineers aiming to build efficient and powerful computing devices.

KEY TAKEAWAYS

- Integrates arithmetic and logical shift capabilities.
- Supports efficient data manipulation.
- Key component in ALU design for modern CPUs.

OUTCOMES CHAPTER 14 (MICRO OPERATION)

- Understand the different types of micro-operations (arithmetic, logic, and shift) and their role in executing low-level operations within the CPU.
- Describe the design and working of the Arithmetic Logic Unit (ALU), including how it performs various micro-operations using logic circuits and control signals.
- Explain the implementation and application of arithmetic, logical, and shift micro-operations, including operations like ADD, SUB, AND, OR, shift left/right, and rotate.
- Analyze the use of control signals and multiplexers in selecting and executing specific micro-operations within a unified ALU structure.